

Sviluppo Applicazioni Per Dispositivi Mobili

simi

February 2024

Contents

1	Augmented Reality	3
1.1	Mixed Reality	4
1.2	Cosa ci aspettiamo dall'AR?	5
1.3	Visori VR	5
1.4	AR in altri media	5
1.5	Quale HW su dispositivi mobili	6
1.6	AR solo visuale?	6
1.7	Cosa studieremo	6
1.8	Registration	7
1.8.1	Esempio della rotazione	7
1.8.2	Registration	10
1.8.3	Registration usando segnale radio	10
1.8.4	AR diretta	11
1.8.5	Geometria prospettica	11
1.8.6	Registration con marcatori espliciti	12
1.8.7	Registration con marcatori impliciti	15
1.9	Tracking	18
1.9.1	Registration ripetuta	19
1.9.2	Tecniche visuo-inerziali	19
1.9.3	Visual-inertial odometry	23
1.9.4	Registration e tracking combinate	24
1.9.5	Esempi	27
1.10	Display	27
1.10.1	Display visuale	29
1.10.2	Sistema di riferimento	29
1.11	Understanding	30
1.11.1	Riconoscimento dei piano	30
1.11.2	Riconoscimento di template 2D	30
1.11.3	Riconoscimento di template 3D	31
1.11.4	Identificazione della sorgente luminosa	31

1.11.5	Sovrapposizione di oggetti virtuali	32
1.11.6	Problema dell'occlusione	32
1.11.7	Altri vincoli fisici	32
1.11.8	Riconoscimento della scena	33
1.11.9	Problema degli oggetti riflettenti	33
1.11.10	Diminished reality	33
1.12	Interazione	34
1.12.1	Conversione di coordinate da 2D a 3D	34
1.12.2	Manipolazione di oggetti virtuali	34
1.12.3	Esempi	36
1.13	Panoramica delle librerie di AR	37
1.13.1	Scene/Session	37
1.13.2	Configuration	37
1.13.3	Ancore	38
1.13.4	hitTest	38
1.13.5	Riconoscimento oggetti	38
1.13.6	Registrazione su scala globale	38
1.13.7	AR multiuser	38
1.13.8	Scene geometry	39
1.13.9	Motion Capture	39
1.13.10	La realtà aumentata e i dispositivi mobili	39
2	Indoor Positioning	40
2.1	Concetti preliminari	41
2.1.1	Cosa vogliamo localizzare	42
2.1.2	Chi effettua il calcolo della posizione	42
2.1.3	Applicazioni principali	42
2.1.4	Paradigmi di navigazione	43
2.1.5	Caratteristiche dei sistemi di posizionamento indoor	43
2.1.6	Discipline coinvolte	44
2.1.7	Tecniche di posizionamento indoor che approfondiamo	45
2.2	Calcolo della posizione con tecniche di immagini	45
2.2.1	Marcatori visivi	46
2.2.2	Marcatori impliciti	46
2.2.3	Senza marcatori	48
2.2.4	Recap	48
2.3	Calcolo della posizione basato su segnale radio	48
2.3.1	Posizionamento per prossimità	49
2.3.2	Multilaterazione	50
2.3.3	WiFi Round Trip Time	51
2.3.4	Fingerprinting	51

2.3.5	Recap tecniche basate su segnale radio	54
2.4	Calcolo dello spostamento	54
2.5	Tecniche ibride	54
2.5.1	Segnale radio + computer vision	55
2.5.2	Marcatori visivi + calcolo dello spostamento	55
2.6	Tecniche di aggregazione di dati soggetti a rumore	55
2.6.1	Modello Bayesiano	56
2.6.2	Modello Markoviano	57
2.6.3	Esempio	59
2.7	Calcolo della posizione adottando il modello Markoviano	61
2.7.1	Particle Filter	62
2.7.2	Quanta incertezza?	65
3	Human Activity Recognition	67
3.1	Panoramica delle tecniche di riconoscimento delle attività	70
3.1.1	Tecniche analitiche	70
3.1.2	Uso del Machine Learning	70
3.2	Acquisizione e pre-processing	74
3.2.1	Allineamento temporale	75
3.2.2	Pre-processing	75
3.3	Acquisizione e annotazione dei dati	80
3.3.1	Quali attività?	80
3.3.2	Quali dispositivi?	80
3.3.3	Quante persone coinvolgere?	80
3.3.4	Dove svolgere le acquisizioni?	81
3.3.5	Quali azioni?	81
3.3.6	Come annotare i dati	81
3.3.7	Quantità di dati da raccogliere	81
3.3.8	Annotazione delle finestre	82
3.3.9	Cosa abbiamo ottenuto	82
3.4	Caratteristiche ed uso degli algoritmi di classificazione	83
3.4.1	Apprendimento e generalizzazione	83
3.4.2	Addestramento offline vs online	84
3.4.3	Inferenza offline vs online	84
3.4.4	Dove avviene la computazione	85
3.4.5	Deploy su dispositivo mobile	85
3.4.6	Creazione del modello	86
3.4.7	Creazione e specializzazione di modelli	86
3.5	Tuning e valutazione dei modelli	86
3.5.1	Suddivisione fissa	87
3.5.2	Cross-validation	87

3.5.3	Metriche di valutazione per i classificatori binari	88
3.5.4	Matrice di confusione	89
3.5.5	Altre metriche	90
3.5.6	Metriche di valutazione per classificatori multi-classe .	90
3.5.7	Validazione dei modelli e semantica dei dati	91
3.5.8	Processo di tuning	91
3.6	Il problema dei dati	92
3.6.1	Reperimento	92
3.6.2	Annotazione dei dati	93
3.7	Activity recognition con tecniche di visione	96
3.7.1	Tipi di attività	97
3.7.2	Quali parti del corpo	97
3.7.3	Classificazione vs Detection	98
3.7.4	Rappresentazione spaziale	98
3.7.5	Uso delle tecniche	99
3.8	Deep learning per activity recognition	99
3.8.1	Il problema dei dati	100
3.8.2	Estrazione delle feature	100
3.8.3	Uso delle reti deep per serie di dati	101
3.8.4	Limiti	102
3.9	Casi di studio	102
3.9.1	SmartWheels	102
3.9.2	Estrazione delle feature	104
3.9.3	Tuning dei parametri	104
3.9.4	Risultati	104
3.10	Play4Physio	104
3.10.1	Riconoscimento degli esercizi	105
4	iOS	106
4.1	Programmazione procedurale in Swift	107
4.1.1	Assegnazione del tipo ad una variabile	107
4.1.2	Tipi	108
4.1.3	Tuple	108
4.1.4	Valori opzionali	109
4.1.5	Gestione dell'overflow	112
4.1.6	Range operator	112
4.1.7	Collections	113
4.1.8	Costrutti di selezione	115
4.2	Funzioni e closures	116
4.2.1	Valori di ritorno multipli	117
4.2.2	Augments lables e parameters name	117

4.2.3	Parametri opzionali	118
4.2.4	Passaggio dei parametri per riferimento	119
4.2.5	Le funzioni come tipi	119
4.2.6	Un esempio esteso	120
4.2.7	Funzioni innestate	122
4.2.8	Closures	122
4.3	Object Oriented Programming	126
4.3.1	Costruttori (inizializzatori)	126
4.3.2	Copie e riferimenti	127
4.3.3	Cosa scelgo? Struct o class	129
4.3.4	Proprietà (attributi)	129
4.3.5	Metodi	130
4.3.6	Computed properties VS Metodi	131
4.3.7	Ereditarietà	131
4.3.8	Estensione di classi	131
4.3.9	Controllo del tipo	132
4.3.10	Protocolli	133
4.3.11	Extension	134
4.3.12	Gestione della memoria	136
4.3.13	Manual Retain Release	136
4.3.14	Automatic Reference Counting	137
4.4	iOS Programming	139
4.4.1	Xcode	140
4.4.2	iOS vs Android	140
4.4.3	Gestione delle view	140
4.4.4	Approccio delle UI: approccio dinamico	140
4.4.5	Approccio statico	141
4.4.6	Esempio dichiarativo (con sistema reattivo)	142
4.4.7	UIKit	142
4.4.8	SwiftUI	143
4.5	Approfondimento su SwiftUI	145
4.5.1	Property Wrappers	146
4.5.2	@State	149
4.5.3	Altri property wrapper	149
4.5.4	Projected values e @Binding	149
4.5.5	Observable objects	153
4.5.6	I tipi opachi	154
4.5.7	Result builders	157

Nel corso di magistrale affronteremo problemi che non hanno una soluzione certa, ma ci sono anche questioni aperte. Infatti durante la magistrale dovremmo cominciare a leggere qualche articolo scientifico.

Le risorse se non indicate come opzionali sono materiale d'esame.

Lunedì iniziamo alle 15.00 e Venerdì iniziamo alle 14.30.

Chapter 1

Augmented Reality

Cos'è l'Augmented Reality, esistono molte definizioni, useremo questa definizione presa da un paper: *Augmented Reality (AR) is a live, direct or indirect, view of a physical, real-world environment whose elements are augmented by computer-generated sensory input such as sound, video, graphic or GPS data*

è una vista del mondo fisico che può essere live diretta o indiretta e in questa vista del mondo reale abbiamo elementi del mondo reale che sono aumentati da degli elementi generati dal computer che l'utente percepisce tipo suono, video, grafica ed altri dati.

- Live: significa che è in tempo reale
- Direct: significa che si può sovrapporre o sostituire al mondo reale (ad esempio tramite un device che mostra quanto ripreso dalla telecamera)
- Indirect significa che può aumentare una rappresentazione del mondo reale (ad esempio una mappa)

E' importante specificare quali sono gli elementi del mondo reale e quelli del mondo virtuale.

1.1 Mixed Reality

L'AR fa parte di un contesto più ampio chiamato Mixed Reality, che indica gli ambienti in cui si mescolano elementi del mondo reale e altri del mondo virtuale:

- Augmented reality
- Augmented Virtuality: immagini del mondo reale messe nel mondo virtuale

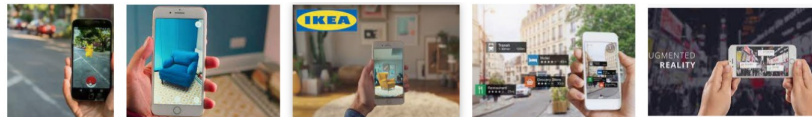
È importante che quando si descrive un problema di mixed reality specificare quali elementi sono del mondo reale a quali sono virtuali.

Possiamo interpretare la mixed reality in uno spettro che va dal reality al virtual reality:

- Reality: solo l'ambiente reale
- Mixed Reality:
 - Augmented reality
 - Augmented virtuality
- Virtual reality: solo mondo virtuale

1.2 Cosa ci aspettiamo dall'AR?

Vengono fuori da google tutte applicazioni su dispositivi mobili, tutti di AR visuale, tutte con viste dirette ed interattive.



In realtà l'AR non è solo su dispositivi mobili, ma è nata ben prima. Già negli anni 40 negli areri si usavano dei visori che facevano vedere delle informazioni virtuali.

1.3 Visori VR

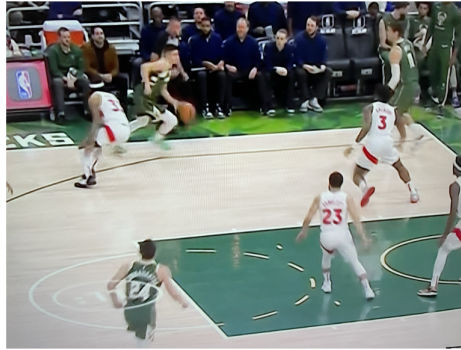
I recenti visori per la realtà virtuale possono essere utilizzati anche per la realtà aumentata e sono molto realistici. Questi dispositivi sono abbastanza di nicchia per i costi e la limitata accettazione (per ora).

1.4 AR in altri media

In NBA la utilizzano massicciamente mettendo scritte sul campo.



Adesso meno ma all'inizio c'erano degli errori, ad esempio il segna tempo si sovrappone al giocatore (problema dell'occlusion).



1.5 Quale HW su dispositivi mobili

L'AR ci richiede di comprendere l'ambiente circostante, per fare questo useremo varie risorse HW dei dispositivi mobili:

- Le telecamere: più di una e anche per comprendere com'è fatto il mondo
- Sensori inerziali che ci aiutano a capire come si sta spostando l'utente
- Sensori di profondità: permettono di costruire un'immagine di profondità (TrueDepth e Lidar per iOS)

1.6 AR solo visuale?

No, l'AR può essere con l'audio spazializzato, ad esempio può usare degli effetti su come percepiamo l'audio proveniente dall'ambiente.

Noi ci concentreremo su AR per dispositivi mobili, direct e visuale.

Abbiamo:

- Una telecamera che riprende il mondo
- Uno schermo che proietta il mondo con aggiunta gli elementi virtuali

Il nostro scopo è fare in modo che gli elementi virtuali sembrino fare parte del mondo.

1.7 Cosa studieremo

4 problemi principali:

- Registration

- Tracking
- Display
- Interaction

1.8 Registration

A volte per spiegare qualcosa è utile spiegare il contrario di quel qualcosa. Per spiegare cos'è l'AR partiamo da un esempio di applicazione che non fa AR.



Sticker è un applicazione in cui ho un pinguino appiccicato al centro dello schermo, ovunque mi muovo lo vedo sempre nello stesso punto in coordinate schermo (sono diverse da coordinate mondo). Se volessi simulare che il pinguino faccia parte del mondo reale dovrei farlo spostare in direzione opposta rispetto a come mi muovo nel mondo reale. Quindi il motivo per cui il pinguino non sembra fare parte del mondo reale è perchè è fermo sulle coordinate della camera ma continua a muoversi in coordinate mondo, che è il contrario di quello che vorremmo noi. Inoltre non poggia da nessuna parte, non ha ombra. Dobbiamo conoscere un sacco di cose dell'ambiente, inoltre dobbiamo sapere come si muove la camera perchè devo continuamente fare variare la posizione del pinguino sulle coordinate schermo in base a come la camera si muove.

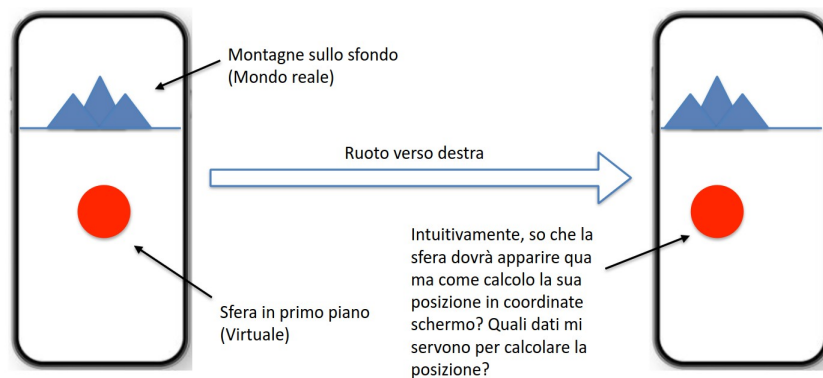
L'esempio del dinosauro ci sembra più verosimile, perchè cambia la visualizzazione se mi avvicino. Analogo se mi sposto attorno (vedo angoli diversi. Poggia per terra e proietta un'ombra.

1.8.1 Esempio della rotazione

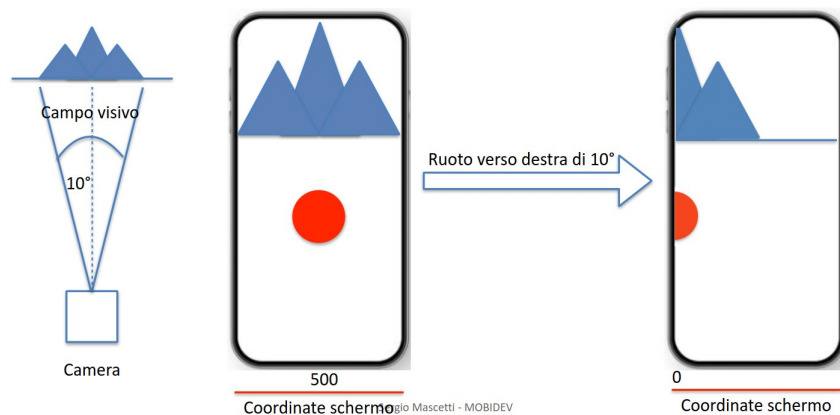
Proviamo a capire cosa si aspetta il nostro sistema. Abbiamo delle montagne e un oggetto virtuale che abbiamo posizionato e stiamo visualizzando, facciamo girare la camera verso destra, ci aspettiamo intuitivamente che la sfera



si sposti verso sinistra (rimane fisso in coordinate mondo, quindi si sposta a sinistra in coordinate schermo), ma di quanti pixel dobbiamo spostare l'oggetto a sinistra? mi interessa la rotazione. Non basta però.



Dobbiamo introdurre un concetto non intuitivo, il campo visivo, la telecamera guarda il mondo con un angolo visivo. Quindi se ci spostiamo di 10 gradi se prima era al centro deve spostarsi sul bordo sinistro.



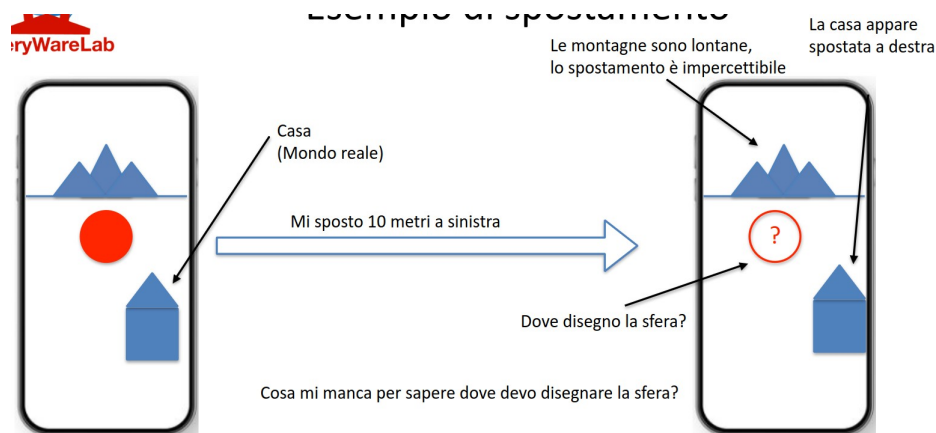
Se invece il campo visivo è più ampio, se ruoto di 10 gradi la posizione sullo schermo sarà a metà strada tra il bordo sinistro ed il centro.



Quindi dobbiamo tenere traccia di come ci spostiamo rispetto alle caratteristiche della nostra camera.

Facciamo finta di spostarci 10 metri a sinistra vedrò la casa spostata e destra, mentre la palla dipende dalla distanza, se era molto vicina può uscire dal campo visivo, mentre se non era tanto vicina potremmo vederla spostata a destra.

L'intuizione è che se vogliamo visualizzare in modo realistico sullo schermo gli oggetti virtuali dobbiamo conoscere le proprietà della camera, che di fatto sono fisse per ogni tipo di camera. L'altra cosa che dobbiamo fare è uscire da un'abitudine tipica di chi fa grafica 2D che è quello di rappresentare gli oggetti sulle loro coordinate sullo schermo, questo in AR non può essere fatto, dobbiamo rappresentare tutti gli oggetti non come loro si proiettano sullo schermo, ma sulla loro posizione sul mondo reale, come loro si proiettano sullo schermo è una conseguenza di come loro si posizionano nel mondo reale.



Dobbiamo inoltre capire dove si trova il dispositivo nel mondo, nel sistema di riferimento del mondo, potrebbe anche essere un sistema locale (una stanza).

In realtà ci sono tante altre cose che dobbiamo capire (la luce, qualucno che passa nel mezzo...)

1.8.2 Registration

Abbiamo due sistemi di riferimenti:

- Sistema di coordinate nel mondo reale
- Sistema di coordinate a schermo

Abbiamo un sistema che ci converte le coordinate del mondo reale in coordinate schermo (potrebbe servirci anche il contrario). Vogliamo capire se dato un oggetto questo si proietta sullo schermo e se si dove?

Abbiamo un sistema, ma questo sistema dipende dalla posizione della camera. Registration è il processo attraverso il quale calcoliamo un'equazione che ci permette di convertire le coordinate del mondo reale in coordinate schermo (ovvero passare da un sistema all'altro).

La registration ha molti aspetti in comune con il calcolo della posizione (che vedremo dopo).

1.8.3 Registration usando segnale radio

Usando segnale radio (bluetooth, wifi...) si può capire dove si trova l'utente. Quindi supponiamo di sapere dove si trova l'utente e com'è orientato, abbiamo degli oggetti nel mondo e vogliamo farli vedere in una visione indiretta,

ovvero abbiamo una rappresentazione di cosa vede l'utente (una mappa che ci fa vedere l'utente visto dall'alto). Vogliamo fare packman in AR, usando le librerie delle mappe viene gratis, ma supponiamo di voler rappresentare la posizione dei mostri senza mappa. Quindi conosco la posizione dei mostri e dell'utente, occupiamoci solo dell'asse x.

Dobbiamo capire se il mostro deve essere visualizzato sullo schermo e se si dove (a quale pixel metto il mostro?).

Facciamo finta che l'utente si trovi a longitudine 10 e che lo schermo mi rappresenti 2 gradi e che abbia 1000 pixel (quindi ai bordi dello schermo ci sono i pixel 9 e 11) a questo punto basta fare una proporzione tra dimensioni.

Se il sistema fosse questo la registration sarebbe capire la posizione dell'utente.

In una vista diretta è più complicato, perchè c'è anche una proiezione prospettica.

1.8.4 AR diretta

Quando ho l'AR diretta il problema è identico, la soluzione è più complicata a livello matematico. La registration è sempre capire i dati che mi servono per convertire la posizione da mondo reale a schermo. Ho sempre una funzione e devo sostituire i parametri per farla funzionare nel caso giusto.

In generale non ci interessa la posizione dell'utente nel mondo reale, ci interessa un sistema di riferimento tridimensionale e per gli scopi di questo insegnamento non distingueremo tra sistema di coordinate camera e schermo (non è detto che sia sempre così, può essere che la camera si muova indipendentemente dallo schermo). Il dispositivo si muove nell'ambiente con 6 gradi di libertà (su giu dx sx e rotazioni lungo i tre assi) ed una camera che riprende il mondo secondo le leggi della geometria prospettica.

1.8.5 Geometria prospettica

Quando abbiamo una visione prospettica del mondo questa viene distorta, le rette rimangono tali, ma gli angoli e distanze vengono distorti.

Esiste un modello (formula matematica) che mi dice come convertire i punti visti in visione prospettica e convertirli in punti del mondo reale (convertire punti del mondo in coordinate schermo e viceversa), è un modello di proiezione prospettica che rappresenta come l'immagine del mondo reale si proietta sull'immagine catturata dalla camera. Non vediamo come si arriva a questo modello di visione prospettica.

- le coordinate nel sistema di riferimento della camera che sono associate tramite una serie di passaggi alle coordinate del mondo reale

Coordinate immagine camera		Parametri camera ("Matrice di calibrazione" o "matrice intrinseca")		Matrice di roto-traslazione		Coordinate mondo reale
	$= s$				$*$	
$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$		$\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$		$\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}$		$\begin{bmatrix} U \\ V \\ W \\ 1 \end{bmatrix}$

- fattore s di scala che per il momento ignoriamo
- Parametri della camera, viene anche detta matrice di calibrazione o intrinseca, che viene calcolata una volta per ogni camera
- La matrice di roto-traslazione, che rappresenta tre valori di traslazione e tre valori di rotazione, di fatto ci codifica 6 valori che sono i t che rappresentano la traslazione sui 3 assi e gli altri 9 valori rappresentano la rotazione. Di fatto questo ci rappresenta la posizione della camera nel mondo reale

Quindi se conosco i parametri della camera e la sua posizione nel mondo reale, dato un punto reale posso capire dove si proietta sullo schermo.

Il problema è calcolare la matrice di roto-traslazione, che è il problema della registration.

Tramite il modello di proiezione prospettica possiamo associare punti nel piano dell'immagine 2D con punti del mondo 3D. Abbiamo capito meglio il problema di registration.

Vediamo 3 soluzioni:

- Marcatori espliciti
- Marcatori impliciti
- Markerless

1.8.6 Registration con marcatori espliciti

I marcatori sono degli oggetti bi-dimensionali che sono stati progettati per essere facilmente riconoscibili con tecniche di computer vision. Sono fatti per permettere di calcolare la posizione e l'orientamento della camera rispetto al marcatore con 6 gradi di libertà.

Potremmo posizionare questi marcatori in una posizione nota rispetto all'origine degli assi, oppure potremmo posizionare questi marcatori sugli

assi della scena reale (usare il marcatore come origine degli assi nel sistema di riferimento mondo è quello che si fa). Mettiamo un marcatore fisico e ci costruiamo la scena attorno, questo perchè il marcatore delimita la posizione degli assi della scena. La domanda è come capiamo la rototraslazione rispetto a questi marcatori.

In robotica si usano i marcatori aruco. Abbiamo l'immagine del marcatore (visto perfettamente da sopra) e abbiamo l'immagine ripresa dalla camera, vogliamo calcolare se nell'immagine è incluso il marcatore e se c'è qual'è la rototraslazione tra le due immagini, se riesco a fare questo ho trovato la rototraslazione della camera ed ho risolto il problema della registration. Ovviamente tutto rispetto ad una scala.

Omografia

L'omografia è una relazione tra punti di due spazi, tale per cui un punto di uno spazio corrisponde ad uno ed uno solo punto dell'altro spazio. Di solito si intende omografia bidimensionale, quando i due spazi sono dei piani. Il problema dell'omografia è trovare una trasformazione in grado di mappare tutti i punti di un piano in quelli dell'altro e viceversa.

Anche qui c'è una soluzione basata su un'equazione matriciale:

$$s \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

ho un fattore di scala che moltiplicato per i punti di un piano è messo in eguaglianza ai punti di un altro piano prima moltiplicati per una matrice, detta matrice omografica.

Le app che fanno le scansioni dei documenti che tolgono la rotazione prospettica applicano la matrice omografica.

Come calcolo la matrice omografica, per calcolare questa matrice bisogna avere quattro coppie di punti, ciascuna coppia è formata da un punto del primo piano (immagine) e un punto del secondo piano (immagine).

Un oggetto bidimensionale di forma rettangolare quando inquadrato da una camera appare distorto in base alla visione prospettica, se però riesco ad identificare i suoi vertici e so che è un quadrato posso rimappare l'immagine in cui ho rimosso la distorsione prospettica (rimappo i vertici in un'immagine più grossa, come si vede dall'immagine nuova che ha del nero), quello che faccio è creare un nuovo piano in cui posizioni i 4 vertici in posizioni note di forma rettangolare, tutto funziona a meno di un fattore di scala che vedremo in seguito.



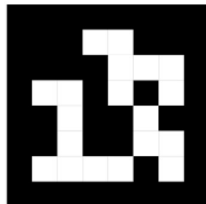
Di fatto la matrice omografica rappresenta una roto-traslazione, ci indica come spostare la camera tra le due visioni che ho del mio piano.

Dalla matrice omografica posso calcolare la matrice di roto-traslazione.

Quindi ora vedremo una procedura per calcolare la matrice omografica rispetto ad un marcatore.

Calcolo della posizione con marcatori ARUCO

Hanno forma quadrata (è comodo per la matrice omografica), il bordo è sempre nero e dentro includono una serie di grossi pixel bianchi ed il disegno non deve essere simmetrico (articolo su come sono stati definiti gli ARUCO, solo le parti evidenziate).



C



A



Dobbiamo calcolare se C contiene A e nel caso quale sia l'omografia che associa l'ARUCO nell'immagine C con quello in A. E' difficile confrontare

l'ARUCO C con A, perchè in C c'è distorsione prospettica per toglierla però ho bisogno dell'omografia che è la soluzione che devo trovare. Prendo una serie di candidati ARUCO in C e per quelli calcolo una matrice omografica temporanea e la uso per togliere la distorsione prospettica, una volta che l'ho tolta li confronto, se il confronto ha successo ho trovato la matrice omografica, altrimenti vado avanti.

1. Trasformo l'immagine in scala di grigi
2. Uso una serie di tecniche ben note che mi tirano fuori i segmenti di retta e vado a vedere se ho 4 segmenti di retta collegati tra di loro ho un quadrilatero e quello è un sospetto ARUCO (mi interessano in quadrilateri perchè c'è distorsione prospettica nonostante stia cercando un quadrato)
3. Avendo un quadrilatero posso calcolare l'omografia, perchè ho i vertici, se quello non era un quadrato originariamente me lo trasformerà in modo strano, mentre se quello in visione prospettica una volta trasformato ottengo un quadrato ho trovato l'ARUCO. Non è esattamente l'ARUCO perchè ci sono degli artefatti legati all'illuminazione che però se ribinarizziamo dovremmo togliere.
4. A questo punto confronto l'immagine con l'ARUCO originale, cosa che non è difficile fare semplicemente scorrendo l'immagine. Se è uguale ho trovato l'ARUCO ed ho trovato l'omografia e la matrice di roto-traslazione, altrimenti vado avanti.

Fino ad ora non abbiamo tenuto conto dell'orientamento del marcatore, possiamo per ogni quadrilatero calcolare 4 omografie (una per ogni possibile rotazione) oppure quando vado a confrontare, provo 4 volte, ruotando l'immagine trasformata in tutte le possibili direzioni.

1.8.7 Registration con marcatori impliciti

Introduce un concetto importante da capire. Possiamo definire un oggetto, tipicamente un oggetto bidimensionale, e dire al sistema "guarda riconosci questo oggetto, ti dico io a tempo di set up del sistema che esiste questo oggetto e voglio che lo trovi nella scena".

I primissimi sistemi di AR funzionavano solo con i marcatori espliciti, nel senso che si usava un oggetto noto a priori e si costruiva la scena su quell'oggetto. (esempio di Ikea, usavano come marcatore il depliant cartaceo). Si usano queste tecniche ad esempio per riconoscere l'opera in un museo e per riconoscere come è orientata la camera.

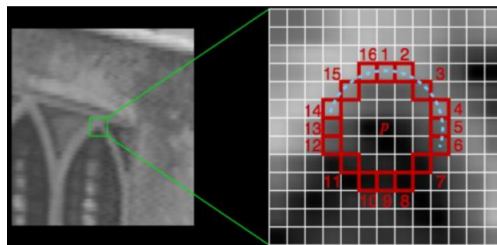
Ci sono dei problemi nell'utilizzare dei marcatori che non sono espliciti (ad esempio una scatola di biscotti), in questo caso non è detto che abbiamo un quadrato (a differenza degli ARUCO), o quantomeno non possiamo sapere che l'immagine contenga un quadrato facilmente riconoscibile. Inoltre gli ARUCO sono costruiti in modo tale da poter essere binarizzati facilmente per togliere il rumore, mentre in questo caso se c'è tanto rumore potremmo non riuscire a toglierlo.

Bisogna usare una tecnica diversa, detta template matching, data un'immagine senza distorsione prospettica capire se quell'immagine compare in un'immagine presa da una camera con distorsione prospettica ed in tal caso capire qual'è l'omografia che le collega.

Anche in questo caso dobbiamo calcolare la matrice omografica per rimuovere la distorsione, ma è la soluzione che vogliamo trovare, quindi possiamo immaginare che la soluzione non sarà troppo diversa.

Feature Points

Il feature point è una piccola parte dell'immagine, tipicamente addirittura un pixel, che ha delle caratteristiche tali per cui mi aspetto di poterlo riconoscere in varie immagini diverse anche cambiando angolatura, luminosità e così via.



Se vogliamo ricordarci di un particolare di una stanza e prendiamo un quadratino di muro bianco è difficile che ci aiuti a capire dove siamo, se invece prendiamo un punto caratteristico è facile che ci aiuti a capire dove siamo.

È quindi un punto con delle caratteristiche, feature descriptions, che sono ad esempio i pixel circostanti.

I feature points, generalmente, vengono individuati dove ci sono dei cambi di colore e di forma molto particolari nell'immagine.

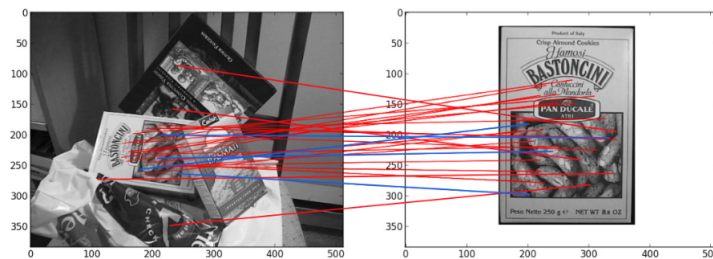
Siccome abbiamo i feature descriptors, possiamo dati due feature points capire che possono essere lo stesso punto ma visto da due punti di vista diversi.

Ci saranno dei casi in cui dovremo calibrare una tolleranza nei confronti dei feature points.

L'intuizione è che i feature point estratti non si rimapperanno nel mondo in mille punti possibili, ma ci sarà un set limitato di punti.

Idea della soluzione

Identifico i feature point in entrambe le immagini, poi trovo un modo per far combaciare (match) i feature points delle due immagini, poi devo trovare quattro coppie di punti tra i feature point che hanno matchato correttamente. Ora il problema è capire quali sono i punti che devo usare, una possibile soluzione è provare tutte le combinazioni di punti (ma sono troppe). Faccio una tecnica che si chiama RANSAC, scelgo un'associazione casuale di 4 punti in T e C e calcolo l'omografia, a questo punto posso rimuovere la distorsione prospettica solo sui feature points usando l'omografia che ho calcolato. Una volta che facciamo l'omografia vediamo se gli altri feature points si rimappano nei punti giusti. A questo punto iteriamo finché non troviamo l'omografia corretta, ovvero non otteniamo un certo livello minimo di punti che combaciano.



Tutto questo funziona a meno di una scala, perchè non gli abbiamo detto quando è grande in centimetri il template. Questo serve per capire dov'è posizionata la camera, in modo da calcolare quant'è ruotata la camera rispetto a quel marcatore.

1.9 Tracking

Abbiamo appena finito di vedere che quello che facciamo è associare una posizione bidimensionale ad una posizione tridimensionale attraverso l'equazione prospettica di cui dobbiamo determinare la matrice di roto-traslazione.

$$\begin{array}{c} \mathbf{x} \\ \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \end{array} = \begin{array}{c} \mathbf{K} \\ \begin{bmatrix} f & 0 & p_x & 0 \\ 0 & f & p_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{array} \overbrace{\begin{array}{c} \mathbf{T} \\ \mathbf{H} = [\mathbf{R} \mid \mathbf{t}] \\ \begin{bmatrix} R_{11} & R_{12} & R_{13} & t_1 \\ R_{21} & R_{22} & R_{23} & t_2 \\ R_{31} & R_{32} & R_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{array}}^{\mathbf{H}} \begin{array}{c} \mathbf{X} \\ \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \end{array}$$

Problema: se abbiamo una registration visuale non basta sapere in un certo istante com'è posizionata la camera nel mondo e pochi secondi dopo non saperlo più. Perché nel momento in cui il sistema sbaglia a calcolare il sistema della camera rispetto al mondo l'utente vede una cosa posizionata nel posto sbagliato. In altre parole se il device si muove, la matrice di roto-traslazione non è più corretta.

In generale il device continua a muoversi, dunque bisogna tenere aggiornata la matrice, questo è il problema del tracking.

Matematicamente possiamo avere una matrice di roto-traslazione e poi possiamo continuare a calcolare lo spostamento tra la matrice iniziale ed una matrice calcolata istante per istante, il prodotto tra le due matrici mi dà la matrice di roto-traslazione aggiornata.

$$\begin{array}{c} \mathbf{x} \\ \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \end{array} = \begin{array}{c} \mathbf{K} \\ \begin{bmatrix} f & 0 & p_x & 0 \\ 0 & f & p_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{array} \overbrace{\begin{array}{c} \mathbf{H}_{\text{tot}} \\ \mathbf{H} = [\mathbf{R} \mid \mathbf{t}] \quad \mathbf{H}' = [\mathbf{R}' \mid \mathbf{t}'] \\ \begin{bmatrix} R_{11} & R_{12} & R_{13} & t_1 \\ R_{21} & R_{22} & R_{23} & t_2 \\ R_{31} & R_{32} & R_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R'_{11} & R'_{12} & R'_{13} & t'_1 \\ R'_{21} & R'_{22} & R'_{23} & t'_2 \\ R'_{31} & R'_{32} & R'_{33} & t'_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{array}}^{\mathbf{H}_{\text{tot}}} \begin{array}{c} \mathbf{X} \\ \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \end{array}$$

Le soluzioni sono tre:

- Ripetere la registration, non fare tracking
- Calcolare la matrice di spostamento usando tecniche visuo-inerziali per calcolare lo spostamento

- Cambiare approccio: non considerare registration e tracking come due problemi distinti ma risolvere, con una sola tecnica, il problema combinato di registrazione e tracking. Fa registration markerless e utilizza i feature points per tenere traccia di come è fatto l'ambiente.

1.9.1 Registration ripetuta

Vediamo perchè non è una buona soluzione, innanzitutto ha un certo costo computazionale quindi non è efficiente. In realtà non può essere utilizzata come soluzione perchè c'è un problema sostanziale di user experience, potrebbe essere che l'utente non inquadri più il marcatore ed in quel caso non è più possibile fare la registration quindi non saremmo più in grado di posizionare l'oggetto nel punto giusto aggiornando la posizione correttamente.

1.9.2 Tecniche visuo-inerziali

Si sono utilizzate tecniche visuo-inerziali, nei dispositivi mobili ci sono dei sensori che servono anche per calcolare lo spostamento (accelerometro, giroscopio...), l'accelerometro calcola l'accelerazione sugli assi, quindi in teoria per calcolare lo spostamento dall'accelerazione basterebbe fare un doppio integrale, non è così semplice.

Quindi le tecniche visuo-inerziali calcolano lo spostamento (inteso come variazione di posizione e orientamento) e combinano:

- Tecniche inerziali
- Tecniche visuali

Si usano tecniche di dead reckoning già utilizzate in diversi ambiti, ad esempio un sottomarino usa accelerazione e velocità di rotazione angolare per capire dove si trova.

Il problema è che il sistema è soggetto ad errori, abbiamo un errore che cresce nel tempo, viene detto errore di drift. Se c'è un errore di registrazione dell'accelerazione mi può portare a sbagliare il calcolo della velocità, questo poi comporterebbe un continuo calcolo di posizioni sbagliate.

Nei sottomarini tutto è mitigato, mentre per noi anche un errore di mezzo metro da una brutta UX.

Nonostante ciò nel breve periodo per alcune applicazioni potrebbe anche andare bene, magari per poco il calcolo della posizione usando tecniche inerziali è corretto, l'utente muove il device generando un drift, poi quando inquadra nuovamente il marcatore potremmo rifare la registration (fare un fix di posizione).

Sostanzialmente quello che succede è questo:

- All'inizio viene generato un sistema di riferimento locale al device, conoscendo la direzione della forza di gravità grazie agli accelerometri (assumendo che il device sia fermo)
- Durante gli spostamenti integriamo il valore dei giroscopi e doppio-integriamo quello degli accelerometri per calcolare la posizione rispetto al nostro sistema di riferimento iniziale
- Queste operazioni di integrazione aumentano l'errore, che causa drift e dunque richiede un frequente fix della posizione

Librerie che usano queste tecniche:

- <https://developer.apple.com/documentation/coremotion>
- https://developer.android.com/develop/sensors-and-location/sensors/sensors_motion

Tecniche visuali

Noi abbiamo anche la camera, possiamo mettere insieme alle tecniche inerziali lo spostamento che vedo nella camera per capire come si sta spostando la camera stessa? il problema si chiama optical flow: è l'insieme di spostamenti appartenenti di oggetti in una scena causato dallo spostamento della camera che riprende la scena. Questo problema prende il nome di optical flow: insieme di spostamenti apparenti di oggetti in una scena causato dallo spostamento della camera che riprende la scena.

Dato un insieme di punti in un certo frame al tempo $t-1$ vogliamo capire qual'è lo spostamento che hanno fatto i punti per arrivare ad una certa posizione al tempo t . (l'input che abbiamo è frame e frame e noi vogliamo calcolare lo spostamento che ci è stato).

Facciamo un esempio: supponiamo di avere un'immagine perfettamente bianca, con un solo pixel verde. È molto difficile capire come si sono spostati i pixel bianchi, invece posso facilmente capire come si è spostato il pixel verde. Ma potrebbero comunque esserci molti casi particolari: es il pixel verde rappresenta un led che si spegne e un altro led, in posizione differente si accende.



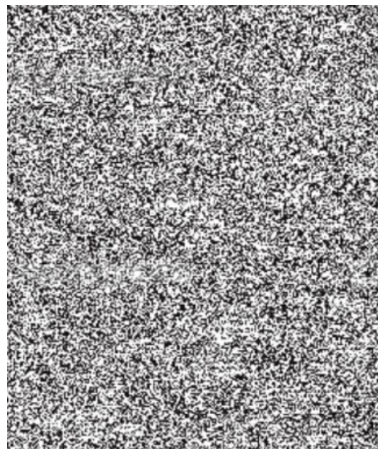
Due approcci:

- Denso: facendo tutti i punti dell'immagine
- Usando i feature points e lo facciamo solo su questo

Problemi che potremmo avere:

- Tra un frame e l'altro potrebbe cambiare improvvisamente l'illuminazione
- I punti potrebbero uscire dall'immagine e potrebbero comparirne di nuovi.
- Non ci è data nessuna garanzia che i punti rimangano fermi, gli oggetti potrebbero spostarsi (avere un movimento proprio)

Ad esempio se inquadrriamo uno schermo che ha with noise non possiamo capire come ci stiamo spostando. Un altro caso comune, anche noi sbagliamo, è il classico esempio del treno, non capiamo se siamo noi a spostarci oppure è l'altro treno che si sta spostando.



Dobbiamo quindi fare delle assunzioni per rendere il problema risolvibile in modo efficace in molti casi pratici:

- Small motion: i punti non si muovono molto lontano, è una conseguenza del fatto che rispetto alla velocità con cui mi sposto passa del tempo
- Brightness constancy: l'illuminazione non cambia troppo
- Spatial coherence: i punti si muovono più o meno come i loro vicini, è un problema quando abbiamo degli oggetti in prospettiva, errore di parallasse

In generale la soluzione non è banale, la prima soluzione è il noto algoritmo di "Lucas-Kanade" dell'81 (da leggere articolo).

La cosa più difficile da capire è il problema e la formalizzazione dello stesso, poi la soluzione in mono-dimensionale è abbastanza semplice.

Abbiamo sempre immaginato l'immagine come un array bidimensionale di pixel, proviamo a vedere un'immagine come una funzione matematica, quindi abbiamo un dominio discreto su x e su y (coordinate del piano) e per ogni coppia di coordinate abbiamo la funzione dell'immagine ci dice il valore dell'immagine in quel punto (profondità di colore in scala di grigi). Nell'esempio di adesso pensiamo ad un'immagine monodimensionale, è una riga di pixel (la prima riga di pixel dell'immagine). Abbiamo due righe di pixel che più o meno sono uguali (non identiche), che sono le funzioni $F(x)$ e $G(x)$, che sono un po' traslate su x di una certa distanza h , quindi matematicamente vediamo le due strisce di pixel come due funzioni, vogliamo calcolare h . Sappiamo inoltre, come detto, che tolta la traslazione h le due funzioni sono simili, quindi $G(x) \sim F(x + h)$

Per definizione di derivata per h piccoli (e noi come da ipotesi stiamo assumendo che h sia piccolo):

$$F'(x) \sim \frac{F(x + h) - F(x)}{h} = \frac{G(x) - F(x)}{h}$$

dunque:

$$h \sim \frac{G(x) - F(x)}{F'(x)}$$

Purtroppo abbiamo una similitudine e non un'uguaglianza, quindi dobbiamo trovare un modo per stimare il valore di h .

Un modo semplice per calcolare h è discretizzare x e fare una media (posso farlo perchè ho funzioni discrete, prendo tutti i punti e ne faccio una media):

$$h \sim \left(\sum_x \frac{G(x) - F(x)}{F'(x)} \right) / \sum_x 1$$

Questo approccio funziona bene quando $F(x)$ è lineare (o non troppo distante dal lineare), il tutto andrebbe ricalcolato cercando di dare peso maggiore ai punti in cui la funzione è lineare e peso minore ai punti in cui non lo è, per capire quando la funzione è lineare calcoliamo la derivata seconda.

$$F''(x) = \frac{G'(x) - F'(x)}{h}$$

Quindi per dare un peso ai punti definiamo un peso per la media, il peso è proporzionale quando la funzione è più vicino al lineare (il peso è 1 fratto

la derivata seconda per ogni punto, maggiore quando la derivata seconda è piccola):

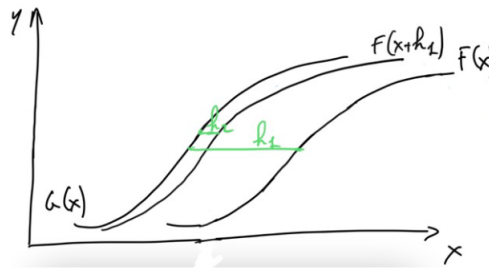
$$w(x) = \frac{1}{|G'(x) - F'(x)|}$$

ora possiamo ridefinire il calcolo di h assegnando il corretto peso ad ogni punto:

$$h \sim \left(\sum_x \frac{w(x) \cdot [G(x) - F(x)]}{F'(x)} \right) / \sum_x w(x)$$

Tutto funziona bene quando h è piccola, allora al posto che calcolarla una volta sola lo calcoliamo in modo iterativo per ridurre gli errori di stima, prima confrontiamo g ed f per calcolare un h' , poi definiamo una nuova funzione $f(x+h')$ a questo punto la distanza sarà più piccola e quindi il calcolo sarà più preciso.

Iteriamo un tot (come dicono nell'articolo) e ci dà un'approssimazione buona di h , a questo punto sommiamo tutti gli h e troviamo l' h originale.



Recentemente sono state sviluppate delle soluzioni per risolvere il problema dell'optical flow basate su machine learning.

1.9.3 Visual-inertial odometry

Posso unire la maniera inerziale e la maniera visuale per avere una maggiore robustezza nel calcolo? sì, anche se non è per niente semplice. Come faccio a mettere insieme due misure diverse con errori? lo vedremo con i filtri a particelle.

Il vantaggio di usare la combinazione è di sorpassare il limiti di uno con l'altro.

Esperimento per capire l'errore drift (errore nel tempo) nell'uso delle tecniche VIO, marker da inquadrare per capire dove sono veramente (non faccio fix di posizione), più è nuovo il dispositivo minore è l'errore, ma comunque la tendenza è a crescere.

1.9.4 Registration e tracking combinate

Un modo alternativo per fare registration è non fare la registration, o quantomeno la faccio talmente banale che non serve a niente, quando parte la session di visione artificiale metto il device in 0,0,0. Il problema è quando mi sto spostando, perchè si accumula l'errore nel tempo.

Se non abbiamo più quei marcatori impliciti o espliciti (i marcatori li definiamo prima), riusciamo a fare un fix di posizione senza aver definito a priori i marcatori che sono nella scena. Il problema può sembrare tipo è nato prima l'uovo o la gallina.

Supponiamo di conoscere la mappa di questa stanza (conosciamo tutti i feature points della mappa) inquadrando una porzione della stanza con dei feature points riusciamo a capire la posizione in cui siamo, ci ricorda il problema del template matching.

Supponiamo di avere un sistema che ci da perfettamente la posizione dell'utente, riusciamo ad andare in giro e fare una mappa dei feature points della stanza.

Posso però fare le due cose assieme? Questo è noto in letteratura e prende il nome di simultaneous location and mapping (SLAM).

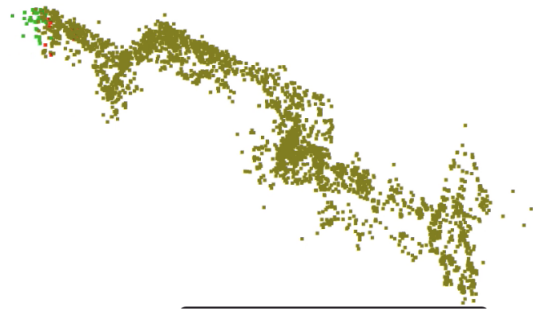
Supponiamo di usare un approccio markerless, non abbiamo definito a priori dei marker che ci sono nella scena, il sistema parte e mette il device in coordinate 0,0,0 e a questo punto comincia a tracciarmi usando tecniche di visione inerziale, riconosce una serie di feature points, si accumula errore e potrebbe capitare che l'utente inquadri nuovamente i feature points che il device ha salvato. Potrei usare quella serie di feature points come marcatore per fare un fix di posizione, ma non lo avevo definito prima, li ho scoperti mentre navigavo l'ambiente.

La differenza rispetto all'uso dei marcatori è che i feature points non li avevo scelti prima, li abbiamo trovati navigando l'ambiente.

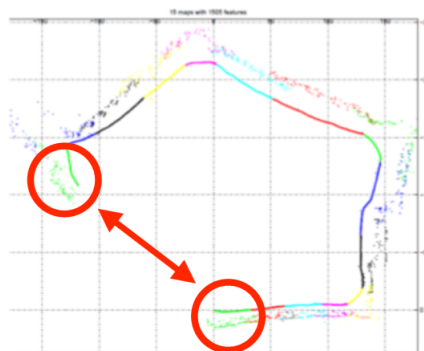
Una nuvola di feature points si chiama point cloud.

Quello che vogliamo noi è creare una mappa di tutto l'ambiente attraverso i suoi feature point, questo ci garantisce che se poi andiamo a inquadrare di nuovo i feature points possiamo fare una sorta di fixing.

Le tecniche di SLAM risolvono contemporaneamente il problema di costruire una mappa e di calcolare la posizione di un oggetto che si muove al suo interno.



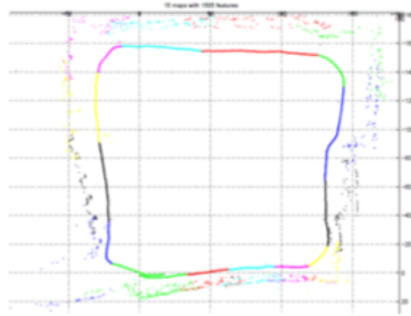
Questa tecnica non serve a nulla se continuiamo a spostarci senza inquadrare di nuovo i feature points. Di fatto quando ci spostiamo le librerie costruiscono una mappa con i feature points. Usando tecniche SLAM quando chiudo un cerchio (la mappa) posso correggere anche le tecniche visio inerziali che ho usato correggendo anche la mappa che aveva creato. Come nell'esempio, supponiamo di girare attorno ad un edificio, mentre lo facciamo, identifichiamo i feature points, l'edificio è rettangolare, ma il sistema visuo-inerziale sta sbagliando, ha calcolato gli angoli in modo errato e dunque il rettangolo non si chiude.



Se però riusciamo ad accorgerci che siamo tornati nello stesso punto della partenza (perchè riconosciamo gli stessi feature points) possiamo correggere l'errore, non solo nella posizione corrente, ma anche in tutte quelle precedenti.

Riuscire a calcolare in modo robusto lo spostamento tramite lo spostamento visio-inerziale meglio riuscirò a costruire la mappa e calcolare la posizione tramite slam.

Video che dovevamo vedere sullo SLAM: consideriamo di avere un approccio probabilistico ai problemi di SLAM, non sappiamo esattamente dove siamo in ogni istante, ma abbiamo una gaussiana (approccio probabilistico) che ci da una distribuzione di probabilità di dove siamo in ogni istante. In pratica vogliamo una distribuzione di probabilità che stima il percorso e la



mappa dell'ambiente dati dei controls command e delle osservazioni dati dai sensori. Esistono anche problemi di Online SLAM, ovvero quando non vogliamo costruire l'intera mappa ma solamente cosa fare dopo (la prossima posizione). Da un punto di vista matematico basta integrare tutte le vecchie posizioni del robot da un problema di full SLAM (con tutta la mappa).

La difficoltà del problema SLAM è che il percorso da fare e la mappa sono entrambi sconosciuti e sono correlati. Inoltre il movimento aumenta l'Instabilità del sistema, abbiamo un errore crescente dato dalle misurazioni dei sensori, possiamo ridurre l'incertezza quando aggiorniamo la mappa e riosserviamo del landmark che conosciamo, questo permette di stimare meglio la posizione. È difficile anche perché c'è un problema di data association, dobbiamo mettere insieme cosa vediamo (osserviamo) in un istante con quello che abbiamo visto prima, l'errore nello stimare la posizione impatta la data association. Esistono diverse mappe che si creano con gli approcci SLAM, dal 3D ad una mappa basata solo sulle features (feature base approach).

Handle wrong data association. Ci sono approcci in cui dobbiamo assumere di essere in un mondo dinamico, nei mondi dinamici (dove gli oggetti si spostano) è più difficile da risolvere.

Abbiamo 3 approcci per risolvere il problema:

- Kalman filter
- Particle filter
- Graph-based

Tutti questi approcci si basano su due modelli, come ci siamo muovendo (motion model) e cosa stiamo vedendo (observation model). Il motion model mi dice qual'è la distribuzione di probabilità della nuova posizione data la posizione vecchia e lo spostamento. Possiamo modellarla come una distribuzione Gaussiana o come una nuvola di punti.

1.9.5 Esempi

Quando si parla di queste tecnologie è interessante, oltre agli aspetti teorici, è interessante capire qualche applicazione pratica.

È stato fatto un progetto di navigazione in ambito industriale, l'idea era data una mappa di rischio di guidare l'utente percorrendo la strada più sicura. L'AR erano dei palloni gialli che indicavano la strada da seguire.

La stessa tecnologia è stata impiegata in un progetto museale, l'idea era di rendere accessibile le opere d'arte alle persone con difficoltà visive. Usa tecniche di template matching per riconoscere le opere d'arte, si supponeva di sapere la posizione delle opere d'arte, grazie a questo era possibile sapere anche la posizione dell'utente nell'ambiente. Per risolvere il problema della scala nel template matching è fornire la dimensione in centimetri del template (potremmo inquadrare il template a distanze diverse). Questa tecnica ha un limite, andiamo a dire al sistema qual'è la posizione del marcatore nell'ambiente, quando lo inquadrano fa il fix di posizione, quando ci spostiamo un po' di errore lo accumula. Quindi se volessimo mostrare delle cose nell'ambiente (un segno di attenzione per l'utente) sarebbe un casino per la precisione. L'idea è quella di utilizzare la capacità delle librerie di AR di creare la mappa e di salvarla, così salviamo dei feature points e non dei marker, così rifacciamo il fix di posizione con i feature points.

1.10 Display

Come abbiamo visto l'AR è divisa in quattro fasi, cominciamo a vedere la fase di display e di interaction.

Dal punto di vista del prof l'AR è fatta però da 5 fasi, c'è una fase anche di understanding, se vogliamo realizzare un sistema realistico dobbiamo comprendere com'è fatto l'ambiente. Per fare questo dobbiamo risolvere problemi non banali di computer vision.

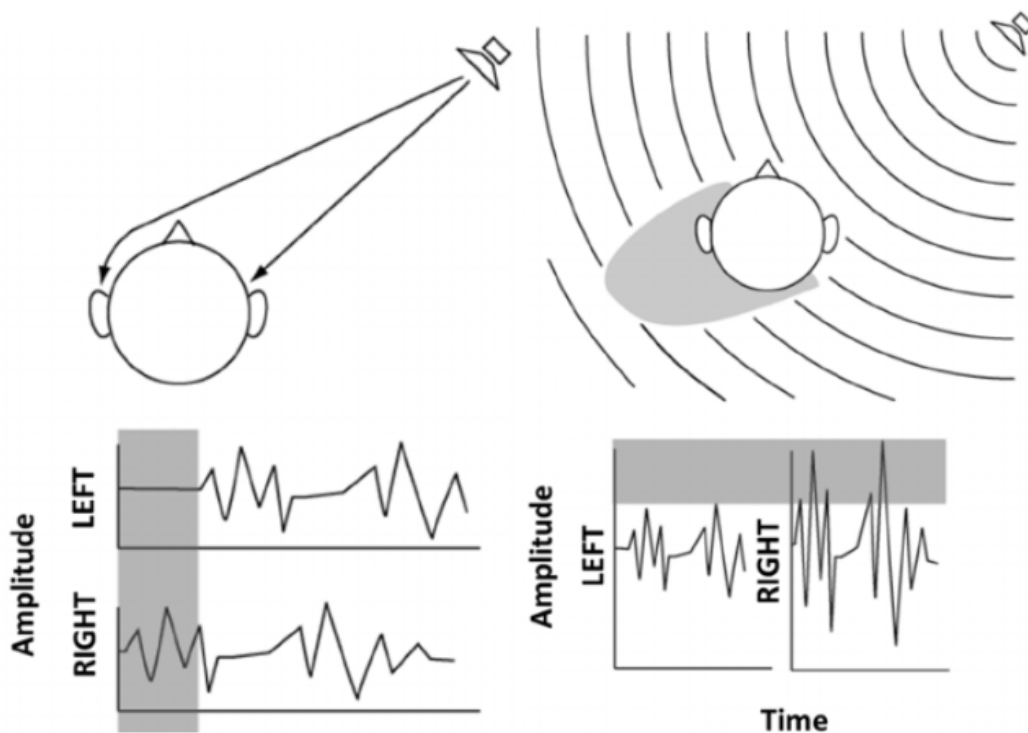
Rientra nel display anche la parte non visuale, la creazione di audio che dovrebbe fare parte dell'ambiente. Dobbiamo creare audio che sembra provenire dall'ambiente, possiamo fare spazializzazione del suono (fare arrivare il suono da una parte dell'ambiente).

Per fare il suono spazializzato serve un sistema stereo per riprodurre il suono (due canali separati come due cuffie). Partiamo dagli aspetti fisici del problema. Quando sentiamo un rumore riusciamo a capire se arriva da destra o sinistra, è più difficile davanti o dietro (perché abbiamo le orecchie a dx e sx). Il nostro cervello usa due trucchi:

- il suono arriva una frazione di secondo prima nell'orecchio che è in di-

rezione del suono. Ovviamente dipende da un sacco di fattori, cos'ho intorno, di che materiali sono fatte le pareti, qual'è la forma del padiglione auricolare. Potremmo però avere delle misure standard. La differenza di tempo di arrivo si chiama Interaural Time Difference.

- guardare la differenza di volume. Il volume sarà più alto nell'orecchio in cui ho sentito il suono. La differenza di volume si chiama Interaural Level Difference.



Per spazializzare il suono possiamo alterarlo per sfruttare queste caratteristiche sui due canali, modificando il tempo o modificando il volume.

Ci sono una serie di difficoltà aggiuntive, una di queste è la posizione della testa, la registration è fatta rispetto al device, che però non è solidale rispetto alla posizione della testa dell'utente. Possiamo usare dell'HW che calcoli la posizione del volto dell'utente rispetto alla posizione del device, le cuffie di Apple che hanno il suono spazializzato funziona così. L'altra possibilità è usare tecniche di riconoscimento del volto dalla camera frontale.

Un' altro problema è quello dell'ambiente. I sistemi che fanno generazione dell'audio sono talmente sofisticati che in un sistema di realtà virtuale si possono associare i suoni anche ai materiali, non solo ai luoghi. Perché il materiale diverso avrà un impatto diverso sulla diffusione del suono. Il

problema è che mentrei nei sistemi di realtà virtuale tutti gli elementi sono virtuali, in sistemi di augmented reality dovrei capire di che materiali sono fatti gli oggetti reali.

L'altro problema è che molte delle proprietà tipiche utilizzate per generare suono spazializzato dipendono in modo marcato dalla forma del padiglione auricolare dell'utente. Come è intuibile questo è un casino. Ci sono delle applicazioni che richiedono la scansione dell'orecchio per personalizzare il modo in cui il suono venga riprodotto. Questi modelli sono chiamati Head Related Transfer Functions.

Nelle librerie di iOS e Android sono disponibili delle funzioni per la spazializzazione del suono.

(letture: Articolo di Harma et al. Leggere introduzione e sezioni 2 e 3, il resto è facoltativo)

1.10.1 Display visuale

Non parleremo del problema del display visuale, perchè è un problema di rendering grafico e ci sono altri corsi che trattano questo problema. L'idea è che il sistema sa dove si trova l'oggetto virtuale e com'è fatto l'oggetto (ne conosce le proprietà). Il problema in realtà aumentata è uguale a quello che abbiamo in realtà virtuale, la differenza sta nella camera, in quanto in un mondo virtuale abbiamo una camera virtuale che possiamo spostare, mentre in realtà aumentata la posizione della camera dobbiamo stimarla ed è fissa.

Guardiamo come si fa a mostrare un oggetto in una libreria di AR. Ci sono due modi:

- Costruiamo un oggetto 3D statico a tempo di programmazione. Quindi prendiamo un modello che rappresenta un oggetto e me lo mettiamo in quella posizione secondo una scale.
- Scrivo del codice che mi crea degli oggetti virtuali all'interno di una scena AR.

Solitamente quando abbiamo un oggetto complesso creiamo un modello e lo carichiamo mentre quando abbiamo un oggetto semplice lo creiamo dinamicamente.

1.10.2 Sistema di riferimento

Come facciamo a posizionare gli oggetti virtuali? Esiste un sistema che semplifica il procedimento con la matrice di roto-traslazione. Abbiamo un sistema di riferimento locale basato su una gerarchia di nodi. Il primo nodo è

l'origine degli assi, i sottostanti definiscono la propria posizione nello spazio in termini di una roto-traslazione rispetto al padre. Quindi quello che faccio è crearmi un nodo all'interno di questa alberatura e posizionare gli oggetti nel sistema di riferimento locale che ho creato con il nodo, in pratica l'effetto finale è di posizionare un oggetto in coordinate del mondo, non in coordinate schermo.

1.11 Understanding

Addentriamoci un po' nella parte che non è standard nelle definizioni di AR, questa è una delle parti su cui ci sono state più modifiche negli ultimi anni. Il problema della visualizzazione è quello di mostrare oggetti in modo realistico. Questo richiede di comprendere l'ambiente. Per questo motivo le librerie di AR includono una serie di funzionalità.

1.11.1 Riconoscimento dei piani

Il riconoscimento dei piani è importante perchè molti degli oggetti virtuali che vogliamo creare devono essere collegati in qualche nodo ad un piano. Tipicamente gli oggetti stanno appoggiati a qualcosa. Questo vale sia per i piani orizzontali sia per i piani verticali. Le tecniche di computer vision per riconoscere i piani sono note da tanti anni e abbiamo un esempio per capire come si fa.

Il Lidar aiuta il riconoscimento dei piani, tramite le immagini di profondità.

1.11.2 Riconoscimento di template 2D

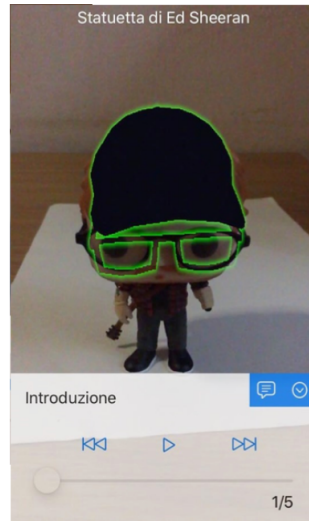
Abbiamo parlato di riconoscimento di template come tecnica di registration, poi abbiamo capito che non si fa così la registration. Però il riconoscimento di template rimane molto utile in alcuni contesti applicativi, come l'esempio di riconoscimento di opere d'arte. Si fa come abbiamo visto.

Nelle librerie ci sono delle funzionalità che chiamano delle funzioni di callback quando un certo template è stato riconosciuto, questa funzione ci dà anche la roto-traslazione.

Tendenzialmente dobbiamo fornire alla libreria l'immagine da riconoscere.

1.11.3 Riconoscimento di template 3D

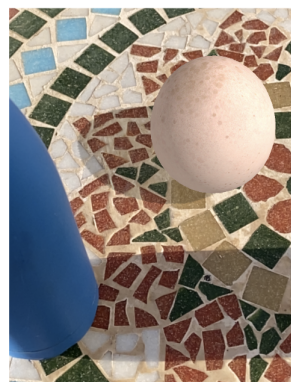
Bisogna fare una scansione 3D dell'oggetto, crea una nuvola di feature points e poi lo riconosce tramite questa cloud, funziona più o meno. Nell'esempio l'app riconosce il template fisico e disegna oggetti visuali sopra di esso.



1.11.4 Identificazione della sorgente luminosa

In un ambiente virtuale è facile perchè la sorgente luminosa la creiamo noi e sappiamo tutte le caratteristiche. Nell'AR è più difficile, possiamo simulare da dove arriva la luce, ma poi potrebbe non essere coerente con quello che succede nell'ambiente.

Ora stanno provando varie tecniche per stimare rispetto all'ambiente dov'è la sorgente luminosa.

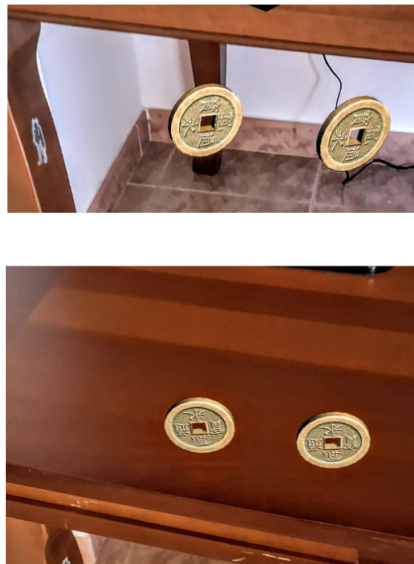


1.11.5 Sovrapposizione di oggetti virtuali

C'è un'altra questione molto rilevante per rendere una sessione di AR credibile. Se ho due oggetti virtuali è facile capire quale deve stare davanti (in quanto so la loro distanza dalla camera), perchè sono entrambi virtuali quindi posso calcolare quale dei due è più vicino per nascondere la parte di quello che sta dietro.

1.11.6 Problema dell'occlusione

Il problema vero nasce quando abbiamo degli oggetti virtuali in un ambiente reale, nelle immagini vediamo le monete sia sopra che sotto il tavolo, non è realistico. Per nasconderle quando è sopra il tavolo il sistema deve capire che c'è un oggetto in mezzo che oscura la visione.



L'hanno risolto per i piani, in generale no. Anche se in realtà lo hanno risolto con il Lidar, perchè siamo in grado di stimare la profondità a questo punto è facile capire cosa sta davanti.

Il problema si ha quando un oggetto fisico si frappone tra la camera e l'oggetto virtuale.

1.11.7 Altri vincoli fisici

Dobbiamo evitare comportamenti irrealistici di compenetrazione tra oggetti fisici e oggetti virtuali. È più difficile ma si possono implementare delle leggi fisiche che permettano ad esempio di gestire le collisioni.

1.11.8 Riconoscimento della scena

Si crea una mesh di copertura dell'ambiente e si usa per capire come interagire con l'ambiente. Con il Lidar ARKit riesce a riconoscere in automatico una serie di oggetti come porte, scrivanie...

1.11.9 Problema degli oggetti riflettenti

Ancora una volta in un ambiente virtuale possiamo tranquillamente posizionare uno specchio, in un ambiente di AR è più difficile, ci sono situazioni in cui è impossibile sapere cosa deve proiettare un oggetto virtuale su una superficie riflettente perchè proprio non conosciamo l'ambiente. Si cerca di stimare, tramite tecniche ML, che caratteristiche possa avere.

(articolo di Ropinski et al (Intro e Sez 3.1 sono obbligatorie))

1.11.10 Diminished reality

Una forma di AR in cui non voglio aggiungere gli oggetti virtuali ma in cui voglio togliere degli oggetti fisici. Con i sistemi di ML attuali si riesce a fare qualcosa di abbastanza realistico per stimare cosa c'è dietro l'oggetto che vogliamo sapere.



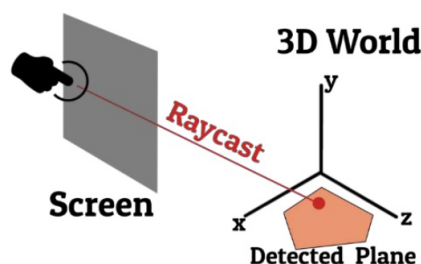
1.12 Interazione

La parte di interazione riguarda come fa un utente ad interagire con un sistema di realtà aumentata. Quando parliamo di dispositivi mobili abbiamo una serie di movimenti ben consolidati su come interagire con il dispositivo. Nell'AR ci sono problemi diversi, in quanto dobbiamo interagire con oggetti virtuali che fanno finta di essere in un ambiente reale. Inoltre di base le applicazioni di AR possono permettere di interagire con due tipi di oggetti:

- Virtuali
- Reali: ad esempio se voglio creare un oggetto virtuale in un punto devo interagire con il punto fisico

1.12.1 Conversione di coordinate da 2D a 3D

Se lavoriamo attraverso lo schermo siamo in bidimensionale, ma il mondo è tridimensionale. Come facciamo a mappare i punti del mondo reale sullo schermo? come faccio a toccare un punto sullo schermo e trasformarlo in punto 3D? C'è una funzione che si chiama Hit Test, tocco un punto dello schermo (corrisponde a quello che viene inquadrato dalla camera) e questo viene proiettato nel mondo reale, mi ritorna un elenco di oggetti reali o virtuali che sto toccando. Avendo un'apertura focale fissa basta fare delle proiezioni per capire dove tracciare la linea, con gli oggetti virtuali è facile capire se un oggetto virtuale tocca questo raggio. Per gli oggetti fisici è più complesso, infatti non si ritornano tutti gli oggetti, ma solo gli oggetti che sono stati riconosciuti, ad esempio la scrivania, se non riconosce niente non ritorna niente.



1.12.2 Manipolazione di oggetti virtuali

Può avvenire con 9 gradi di libertà (in linea di principio, possiamo anche ridurli), 3 posizione, 3 rotazione, 3 per la dimensione sui tre assi. Quali

sono gli strumenti che uso per permettere all'utente di fare queste 9 operazioni sull'oggetto? Qui gli approcci sono due, spesso le soluzioni sono la combinazione dei due:

- **Integrality:** l'utente può modificare tutte le dimensioni contemporaneamente, possiamo definire delle gesture per gestire le dimensioni allo stesso tempo.
- **Separability:** l'utente può modificare una dimensione alla volta

Tipicamente si trova un compromesso permettendo di modificare un insieme di dimensioni alla volta.

Quali sono i paradigmi di interazione che possiamo utilizzare:

- Tramite schermo
- Proxy fisici
- Movimenti delle mani nello spazio
- Tracciamento della vista

Tramite schermo

È forse uno degli approcci più comuni ma è molto delicata, bisogna separare bene integrality e separability, bisogna il più possibile limitare i gradi di libertà in modo che i gesti facciano quello che si aspetta l'utente, per non dovergli spiegare nulla.

Ovviamente si pone il problema di capire come gestire la dimensione perpendicolare allo schermo. Se limitiamo le dimensioni a 2 è facile per l'utente. Per la terza dimensione bisogna aggiungere dei gesti che potrebbero non essere facili per l'utente.

In realtà fino adesso abbiamo dato per scontato che vogliamo avere un interazione diretta, ovvero vogliamo interagire con l'oggetto interagendo direttamente con l'oggetto stesso. In alcune applicazioni è scomodo questo approccio, forse conviene usare altri elementi di UI (non AR) per interagire con l'oggetto, questo è un approccio con interazione indiretta.

Interazione tramite proxy fisici

Un'altra possibilità che è poco utilizzata è l'interazione tramite proxy fisici. A volte può essere molto comodo per l'utente non fare cose strane sullo schermo ma muovere oggetti fisici per fare cose sugli oggetti virtuali.



I controller dei visori sono proxy fisici. Il sistema traccia la posizione dei controller nello spazio con anche un insieme di tasti.

L'uso di marcatori è un altro sistema di interazione con proxy fisici.

Si potrebbero usare anche dei marcatori per capire come posizionare degli oggetti. Ad esempio è stato realizzato un sistema per migliorare il gioco di ruolo su ogni pedina veniva messo in realtà aumentata il personaggio e veniva poi spostata la pedina per muovere il personaggio.

Movimento delle mani nello spazio 3D

Si chiama anche interazione naturale. Il problema con gli smartphone è che posso inquadrare una mano sola perchè con l'altra devo tenere il telefono.

Utilizzo di eye tracking

Non nasce adesso ma c'è da tanti anni e veniva utilizzato per:

- Ricerca: capire dove stava guardando l'utente, ad esempio su un sito
- Sviluppo di tecnologie assistive per persone con disabilità motorie

Ultimamente è stato utilizzato nell'Apple vision pro, unisce anche gesti con la mano per generare l'interazione.

1.12.3 Esempi

Abbiamo visto un esempio in cui tramite un template mettiamo un oggetto virtuale in una scena e vediamo anche delle debug options.

Vediamo ad esempio l'origine degli assi in cui ha creato la scena, vediamo anche l'oggetto. Vediamo anche dei puntini gialli che sono i feature points che ha individuato ARKit.

L'altro esempio riconosce un piano e disegna sopra al piano. L'oggetto configuration definisce i parametri della sessione AR, configuration.planeDetection dice che voglio fare una plan detection.

Faccio una callback che viene chiamato quando trova un oggetto. L'idea è che disegno un piano virtuale sopra quello fisico che ho riconosciuto, cambio il colore se è orizzontale o verticale.

Creo un nodo in cui metto un piano e in cui imposto la posizione del nodo andando a prendere i dati del piano. Lo ruotiamo di 90 gradi sull'asse x. Per aggiornare i piani gestiamo la callback `rendered didUpdate`. Il metodo sostanzialmente è lo stesso, aggiungo un nuovo piano ma prima di aggiungerlo tolgo dal nodo quello precedente.

1.13 Panoramica delle librerie di AR

Le librerie più diffuse per mobile sono ARKit e ARCore rispettivamente per iOS e Android.

Ci sono delle librerie anche cross platform che risultano essere molto indietro rispetto a le librerie per il nativo, come Vuforia.

Offrono gratis le funzionalità di registration e tracking, understanding, display e interaction.

ARKit e ARCore condividono alcuni concetti di base:

- Scene / sessione
- Configuration
- Anchor
- HitTest / RayCast

1.13.1 Scene/Session

Rappresentano l'ambiente rispetto al quale effettuiamo la registration, gli oggetti virtuali e reali sono posizionati nel sistema di riferimento della scena secondo una rototraslazione. Fornisce le chiamate per accedere agli altri elementi di AR.

1.13.2 Configuration

È un oggetto che definisce le proprietà della scena. In ARKit abbiamo diversi oggetti base da cui partire:

- `ARWorldTrackingConfiguration`: viene creato un sistema di riferimento in una posizione iniziale e si tiene traccia dello spostamento rispetto a questo sistema
- `ARImageTrackingConfiguration`: viene creato un sistema di riferimento in corrispondenza di un oggetto

- `ARFaceTrackingConfiguration`: il sistema di riferimento viene creato sul volto di un utente

1.13.3 Ancore

Rappresentano riferimenti agli oggetti virtuali o a quelli reali identificati, quando un oggetto viene riconosciuto, viene associato ad un'ancora. Attraverso l'ancora possiamo risalire alla posizione e rotazione dell'oggetto rispetto alla scena.

1.13.4 `hitTest`

Si tratta di un'operazione per convertire coordinate schermo in un punto 3D.

1.13.5 Riconoscimento oggetti

Per riconoscere un oggetto 3D ARKit necessita di un modello 3D dell'oggetto da riconoscere.

1.13.6 Registrazione su scala globale

Esiste una configurazione `ARGeoTrackingConfiguration` che usa il riferimento su scala globale, non crea l'origine degli assi dove siamo ma da la posizione in termini di latitudine, longitudine e rotazione. Si poteva fare anche prima ma era necessario fare a mano una conversione sapendo le coordinate dell'origine degli assi. Aveva la limitazione di fare la registrazione in termini di posizionamento outdoor.

Hanno implementato una funzionalità che usa la street view per fare la registration visuale unendo il segnale GNSS. C'è una funzionalità simile in Google Maps in cui inquadrando attorno e riconosce delle cose che conosce e mi dà la posizione in AR.

1.13.7 AR multiuser

Sia in ARKit e ARCore è possibile creare sessioni condivise tra più utenti. Due utenti condividono la stessa sessione e possono metterci vari oggetti virtuali con cui possono interagire. L'idea concettuale è che usando informazioni visuali riesco a fare in modo che due (o più) device si registrino rispetto allo stesso sistema di riferimento.

Tipicamente AR multiuser è pensato per essere usato allo stesso tempo. Però si può anche realizzare in tempi diversi. Esempio di applicazione: creo

un percorso che vorrei far seguire ad un altro utente. Salvo questo percorso assieme alla sessione/scena che ho usato per crearlo (che contiene tutti i riferimenti agli oggetti dell'ambiente) così quando lo stesso utente (o uno diverso) torna nello stesso posto, potrà seguire lo stesso percorso usando le informazioni già note della sessione/scena.

1.13.8 Scene geometry

Tramite il Lidar sappiamo la profondità quindi possiamo creare una ricostruzione topologica dell'ambiente. Questo rende possibile:

- Riconoscere oggetti
- Calcolare quando un oggetto si interpone tra la camera e un oggetto virtuale, così da risolvere il problema dell'occlusione
- Permettere agli oggetti virtuali di rispettare (alcune) regole della fisica nell'interazione con oggetti del mondo reale

1.13.9 Motion Capture

Permette di riconoscere le principali articolazioni del corpo e averne informazioni tramite una modalità chiamata Body tracking.

1.13.10 La realtà aumentata e i dispositivi mobili

La limitazione è che non è immersiva, c'è anche il problema che il device va tenuto in mano e questo diventa scomodo.

I sistemi di AR necessitano di soluzioni evolute in varie discipline, tra cui:

- Data management
- Grafica
- HCI
- Computer vision
- AI / ML

In molti di questi campi la ricerca e lo sviluppo sono ancora in corso per riuscire a migliorare le soluzioni esistenti.

Chapter 2

Indoor Positioning

È una tematica che il prof ha affrontato parecchio, sia in ambito di sviluppo industriale e anche in ambito di ricerca per fare applicazioni assistive.

Lo studio del problema dell'indoor positioning aiuta a capire che ci sono problemi complessi e poi capire caso per caso, qual'è la soluzione migliore da adottare. Non c'è una soluzione che è emersa come stato dell'arte per l'indoor positioning, ci sono tante soluzioni possibili con pro e contro.

Quindi questa sezione avrà una parte introduttiva molto lunga, per capire cosa dobbiamo affrontare.

2.1 Concetti preliminari

Nella posizione indoor solitamente è richiesta una precisione maggiore nel calcolo. Ad esempio se siamo in strada e l'indicazione è sbagliata di un paio di metri tendenzialmente non succede niente, mentre se in indoor sbagliamo di due metri le informazioni di navigazione cambiano notevolmente.

Molte delle tecniche adottate per l'outdoor non possono essere adottate per l'indoor. Il motivo principale è che non c'è una situazione in cui possiamo ricevere il segnale satellitare, quindi si usa il satellite quando abbiamo una condizione di Line-of-Sight (possiamo vedere direttamente il satellite). Inoltre nell'indoor il segnale radio è disturbato da oggetti che potrebbero impedire di calcolare la posizione tramite la rete cellulare, è vero che anche nell'outdoor ci sono delle interferenze. Non si può neanche usare il posizionamento con il wifi perchè anche nell'outdoor per essere utilizzata ha bisogno di un approccio crowdsourcing che a sua volta ha bisogno che qualcuno conosca la posizione precisamente con il satellite.

Ci sono altre differenze tra outdoor e indoor:

- L'area di copertura è ridotta
- C'è un minore impatto dei fenomeni meteorologici, anche se in realtà ci sono sempre dei fattori esterni incontrollabili
- Abbiamo delle risorse che in outdoor non abbiamo, ad esempio potremmo avere accesso ad una rete cablata, alla rete elettrica ...
- Solitamente una minore rapidità di spostamento degli utenti. Questo è un vantaggio perchè l'utente si muove più lentamente quindi frequenze di aggiornamento più lente potrebbero non essere così influenti.
- La proprietà è tipicamente privata: potrebbero esserci degli edifici pubblici, ma quasi sempre quando abbiamo un edificio c'è bisogno di inter-

agire con il gestore dell'edificio che ha la planimetria per permetterci di stabilire quali sono le rotte migliori

Per tutti questi motivi l'indoor positioning è ancora poco comune, esistono delle implementazioni, ma sono tutte tecnologie diverse che adottano implementazioni diverse. Questo ci fa capire che non esiste una soluzione migliore per tutti.

2.1.1 Cosa vogliamo localizzare

Il soggetto da localizzare è l'utente tramite il suo dispositivo mobile. In realtà molta della letteratura disponibile è stata fatta per il posizionamento di robot che si muovono in maniera autonoma per l'ambiente. La differenza sostanziale è che del robot (su ruote) si può stimare molto bene quanto si sta spostando, inoltre sui robot possiamo utilizzare dell'HW apposito.

L'altra possibilità è avere degli oggetti, che vengono spostati, esempio del carrello per le emergenze in ospedale, anche in questo caso si può attaccare dell'HW apposito.

Non tratteremo soluzioni che implicano l'aggiungimento di HW ai dispositivi mobili.

2.1.2 Chi effettua il calcolo della posizione

Tecniche attive sono le tecniche in cui il dispositivo mobile calcola la sua posizione in locale. Esiste tutta una serie di tecniche, passive, per effettuare il calcolo della posizione a livello del sistema, è il sistema che calcola la posizione, possibilmente con la collaborazione dei dispositivi (ma non è detto). Ad esempio è possibile tracciare il flusso di persone in ambienti tramite l'utilizzo di wifi. Noi ci focalizzeremo sulle tecniche attive.

2.1.3 Applicazioni principali

- Navigazione
- Indoor location based service: ad esempio applicazioni di proximity marketing
- Home automation: accensione e spegnimento automatica delle luci, porta che si apre automaticamente quando ti avvicini
- Context detection: la posizione, anche indoor, è un aspetto importante del contesto dell'utente

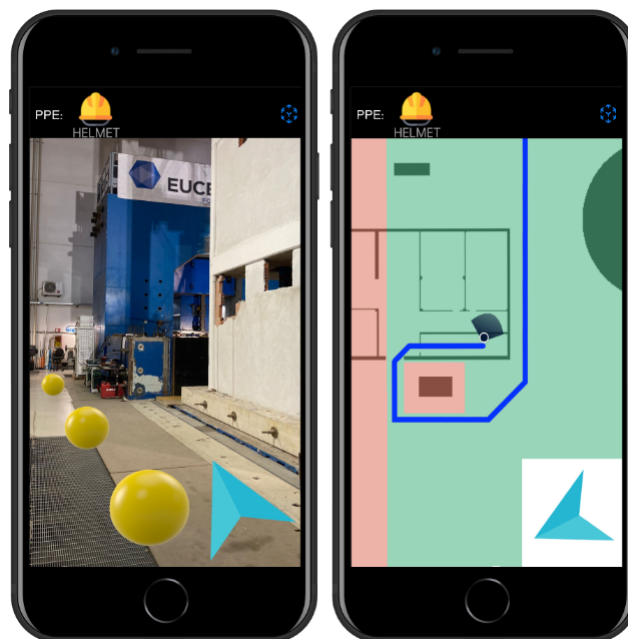
- Social networking: friend finder, dove sono gli amici attorno a noi
- Emergenze: in caso di incendio i vigili del fuoco sanno dove sono le persone da salvare

2.1.4 Paradigmi di navigazione

Sono due:

- Sistema egocentrico: ci fa vedere le informazioni da dove sei tu. Ovviamente devo capire come sei orientato. Il vantaggio è che spesso richiede uno sforzo cognitivo minore, è di più immediata comprensione.
- Sistema allocentrico: è la classica vista dall'alto. Il vantaggio è che si può usare anche senza sapere l'orientamento dell'utente.

Non c'è un sistema che va male, bisogna capire in base alla applicazione quale dei è il più adatto.



2.1.5 Caratteristiche dei sistemi di posizionamento indoor

Che caratteristiche ci interessano:

- Quanto è preciso il sistema nel calcolo della posizione?
- Area di copertura (stanza, edificio...)
- Costo (installazione, manutenzione, HW, setup ...)
- Infrastruttura (nessuna, trasmettitori sparsi nell'ambiente...)
- Notorietà della tecnologia
- Tipo di informazione prodotta: può essere prodotta solo informazione 2D oppure anche 3D, ovvero l'altezza. (a che piano sono)
- Privacy: nei sistemi passivi (dove l'ambiente calcola la posizione)
- Frequenza di update: ci sono sistemi in cui è possibile calcolare la frequenza di update quasi in modo continuo ed altri in cui è richiesta l'intervento dell'utente
- Interfaccia
- Robustezza della misura di calcolo: bisogna misurare il sistema quando l'ambiente è affollato
- Praticità della soluzione: le batterie dei beacon si scaricano, i marcatori si staccano
- Scalabilità: ci sono alcuni sistemi che funzionano bene in un ambiente piccolo, quando allarghiamo il sistema rischia di aumentare molto il costo o l'imprecisione del calcolo
- Impatto sull'ambiente: i marcatori visivi rovinano l'aspetto estetico

2.1.6 Discipline coinvolte

Il calcolo della posizione è principalmente un problema di data management. Tuttavia sono coinvolte altre discipline:

- Reti
- Computer vision
- Machine learning
- HCI

2.1.7 Tecniche di posizionamento indoor che approfondiamo

- Basate su immagini: come abbiamo visto in AR
- Basate su segnale radio
- Basate su sistemi inerziali
- Sistemi ibridi:
 - Incluse tecniche visuo-inerziali basate su AR

Ci sono altre tecniche che non affronteremo:

- Basate su infrarosso: in teoria potrebbero funzionare su dispositivi mobili, anche se probabilmente non si possono acquisire le immagini infrarosso della telecamera
- Basate su suono: se in una stanza c'è un suono ripetuto possiamo riconoscerlo per capire in che stanza siamo. Ultrasuono così il sistema non lo sente ma noi sì. L'altra possibilità è che il nostro dispositivo emetta un suono e il sistema con microfoni capisce dove siamo.
- Basate su RFID: range molto limitato
- Ultra Wide Band: è una tecnologia radio. Gli air tag funzionano così.
- GNSS pseudo-satellites o ripetitori GNSS: esistono dei sistemi che permettono di o ripetere o simulare un segnale GNSS in modo che con l'antenna GNSS posso riceverlo.

2.2 Calcolo della posizione con tecniche di immagini

Come visto in AR possiamo usare dei marcatori espliciti ed impliciti. Vedremo due versioni basate sui marcatori espliciti. Poi vedremo quelle con marcatori visivi impliciti e poi le tecniche senza marcatori.

2.2.1 Marcatori visivi

L'idea è che ho un marcatore e so dove l'ho posizionato nell'ambiente, quindi nella versione base so che quando l'utente inquadra con il suo dispositivo quel marcatore deve essere lì nei paraggi.

Questa tecnica per quanto banale è stata utilizzata, ad esempio dal sistema mapwize la cui idea era molto semplice, l'addetto tramite un'interfaccia web caricava la mappa dell'ambiente, posizionava dei marcatori di posizione (dei qr code) il quale ha scritto un valore unico, quando viene inquadrato sappiamo circa dove si trova l'utente.

Dal punto di vista architetturale possiamo mettere nel qr code la posizione (latitudine e longitudine) oppure mettere un id che dice in che posizione è.

Il vantaggio principale di questa soluzione è che è semplice, inoltre i marcatori sono molto economici e la tecnica di data management è banale. Ha degli svantaggi:

- La precisione è legata alla distanza alla quale il device riesce a riconoscere il marcatore. Tipicamente un errore di qualche metro
- Siccome l'utente può inquadrare il marcatore con diversi angoli, di fatto questa tecnica non permette di calcolare l'orientamento
- È richiesta un'azione esplicita all'utente
- Non adatto se l'utente ha difficoltà visive
- Tra due marcatori non sappiamo dov'è l'utente
- È necessario disporre nell'ambiente vari cartelli e questo può creare problemi estetici

Una soluzione più avanzata è che conoscendo la dimensione fisica del marcatore potrei utilizzare tecniche di AR e calcolare la matrice di roto-traslazione, quindi dato che conosciamo la posizione e l'orientamento del marcatore nello spazio, possiamo risalire facilmente alla posizione e all'orientamento dell'utente nello spazio. Problema se non usiamo sessioni AR non sappiamo la matrice intrinseca della camera. Se usiamo librerie di AR ci viene tutto gratis, ma se dobbiamo implementare noi il calcolo della matrice di roto-traslazione è un casino.

2.2.2 Marcatori impliciti

Si potrebbero utilizzare dei marcatori impliciti, ma quali marcatori si possono usare in un ambiente indoor? o abbiamo casi particolari, ad esempio

un museo, che ha tanti quadri che solitamente non si spostano. In un caso generico gli oggetti che possono fungere da template sono i cartelli, ad esempio quelli di emergenza sono messi in modo tale che siano facilmente visibili. Il problema è che potrebbero essere troppo semplici per essere riconosciuti con tecniche di pattern matching. L'altro problema è che potrei averne più di uno, in quel caso cosa faccio?

Disambiguazione dei marcatori impliciti

Ci sono due tecniche sostanziali:

- Lo faccio in una tecnica ibrida, usando il segnale readio calcolo che l'utente si trova in una stanza, ma non so dove e non so come sia orientato, e poi inquadrare un cartello so dove si trova l'utente e come è orientato
- Uso tecniche per calcolare lo spostamento: l'idea è che veniamo catapultati in un ambiente che non conosciamo niente e vediamo solo i cartelli, ne vediamo uno ci spostiamo e ne vediamo un altro, facendo così combinando la posizione di cartelli con le loro posizioni relative posso capire dove sono. È una soluzione difficile da implementare, perchè dobbiamo tenere conto dell'errore, in realtà applicando tecniche di particle filter possiamo risolvere il problema. Il problema nasce se abbiamo edifici in cui la posizione dei cartelli si ripete su tutti i piani, inoltre non è immediato dobbiamo inquadrare molti cartelli.

Pro:

- Non devo installare altri segnali nell'ambiente

Contro:

- Le tecniche di riconoscimento sono più complicate
- Se i marcatori non sono univoci, devo combinarla con altre tecniche

Limiti:

- L'utente deve inquadrare il marcatore
- Devo definire a priori i marcatori che andrò ad utilizzare, è un lavoro lungo e noioso perchè devo andare a trovare i marcatori e fare le misurazioni. È la fase di setup.

2.2.3 Senza marcatori

L'idea è di costruire una mappa solitamente 3D dell'ambiente, non deve essere una ricostruzione di tutti i pixel che vedo, ma solo dei feature points. Quando poi l'utente è nell'ambiente, vengono identificati alcuni punti di riferimento e poi vengono confrontati con quelli ottenuti nella fase di costruzione della mappa.

Questa è una soluzione, ormai datata, che veniva fatta utilizzando dell'HW apposito.

È una soluzione più facile dello SLAM, perchè si usava dell'HW apposito per fare la mappa, poi con il dispositivo mobile si faceva solo la localizzazione.

Ad oggi si può fare lo SLAM direttamente su dispositivi mobili, soprattutto quelli con il LIDAR.

Rendiamoci conto però che se ci appoggiamo su librerie di AR ci viene gratis, mentre se dobbiamo implementare tutto da zero è parecchio lavoro.

2.2.4 Recap

	<i>Errore ODG (Ordine Di Grandezza)</i>	<i>Orientamento?</i>	<i>Complessità</i>	<i>Costo (setup, hardware)</i>	<i>Note</i>
<i>Marcatori espliciti Base</i>	<i>metri</i>	<i>No</i>	<i>Molto bassa</i>	<i>Basso</i>	<i>1. Azione esplicita 2. Riconoscimento sporadico 3. Impatto visivo sull'ambiente</i>
<i>Marcatori espliciti Avanzata</i>	<i>decimetri</i>	<i>Sì</i>	<i>Bassa</i>	<i>Basso</i>	<i>Come sopra</i>
<i>Marcatori impliciti</i>	<i>decimetri</i>	<i>Sì</i>	<i>Media, (e va combinata con altre tecniche)</i>	<i>Basso</i>	<i>1. e 2. come sopra + 4. richiede altre tecniche per disambiguare la posizione</i>
<i>Senza marcatori</i>	<i>decimetri</i>	<i>Sì</i>	<i>Elevata (e va combinata con altre tecniche)</i>	<i>Dipende, tipicamente medio</i>	<i>4. come sopra La complessità è molto minore se si utilizzano librerie di AR (vediamo dopo)</i>

2.3 Calcolo della posizione basato su segnale radio

Iniziamo a vedere una tecnica molto semplice e man mano le raffiniamo. Prima di vedere le tecniche dobbiamo sapere che ci sono varie tipologie di

segnale radio:

- WiFi: è importante perchè lo abbiamo sui dispositivi mobili e noi non vogliamo HW aggiuntivo. Ed è già disponibile nell'ambiente.
- Bluetooth: è disponibile nei dispositivi mobili e tipicamente non è nell'ambiente, quindi dovremmo pensare di dotare gli ambienti di questa comunicazione radio, potremmo usare un "beacon" bluetooth che è fatto da una circuiteria e una batteria che trasmette un segnale.
- Near Field Communication (NFC): funziona a distanze piccolissime, di fatto vuol dire che se so dov'è l'antenna sono lì
- Ultra Wide Band: ne parleremo poco perchè è poco diffuso

Un aspetto comune (tra wifi e bluetooth) che ci serve è il concetto di RSSI, nelle comunicazioni wireless è un valore calcolato dall'antenna che riceve un segnale, che indica la potenza del segnale ricevuto dal dispositivo mobile.

C'è una premessa molto tecnologica, applicativa, alcune delle tecniche che andiamo a vedere adesso richiederebbero di usare delle API del sistema operativo che però non sono disponibili. Ad esempio alcune tecniche chiedono di leggere l'RSSI ricevuta da alcune antenne, su iOS non si può fare, o meglio si può fare usando delle API in fase di sviluppo ma non si può pubblicare l'applicazione con quelle API attive.

Nel seguito vediamo alcune tecniche di calcolo della posizione. Molte di queste si possono adottare con più di un segnale radio. La tabella al termine di questa sezione riassume le combinazioni più comuni.

2.3.1 Posizionamento per prossimità

È la prima tecnica che vediamo ed è banale. È simile a quello visto con i marcatori visivi, se vedo quel marcatore so che sono in una zona vicino a quel marcatore. Analogamente se il device vede l'antenna (che ha un certo identificativo) significa che so che circa sono nei paraggi dell'antenna. Con il WiFi posso assumere che mi trovo entro 100 metri dall'antenna.

Detto così sembra una tecnica inutile ma non è così:

- È semplice, devo guardare solo un'antenna
- Potrei non usarla da sola: potrei usarla in una soluzione ibrida in cui utilizzo il calcolo della posizione con WiFi con qualcos'altro

- Non è detto che debba essere utilizzato solo con il WiFi, se utilizzato con il bluetooth mi da una precisione migliore, perchè il segnale bluetooth si riceve entro 5/10 metri. In alcuni musei l'audio guida fa partire la descrizione di una stanza nel momento in cui ci arrivi, questo veniva fatto con dei "beacon" bluetooth nella stanza.

Questi beacon bluetooth trasmettono via radio il loro ID, venivano configurati impostando la potenza del segnale e gli impostavamo l'identificatore di quello specifico beacon. Il segnale riceveva quell'identificatore e dato che l'app aveva l'identificatore hardcodato dentro riusciva a capire di che beacon riceveva il segnale. Il segnale andava calibrato perchè dovevano consumare poco per scaricare meno la batteria, inoltre se impostiamo un segnale debole lo percepiamo da più vicino.

Come facciamo a conoscere la posizione dell'antenna? se installiamo un beacon bluetooth sappiamo dove lo abbiamo messo e mettiamo quest'informazione nell'app. Anche con il WiFi tendenzialmente sappiamo dove abbiamo messo l'antenna.

Il problema è che la precisione è bassa, ma perchè?

- Si considera solo un'antenna, sull'outdoor ci sono una serie di tecniche basate sulla multi-laterazione
- Si considera solo il fatto di percepire il segnale di quell'antenna, non si prova a stimare la distanza dall'antenna stessa. Sappiamo che siamo nel raggio percepito dall'antenna ma non stiamo calcolando la distanza.

Le tecniche che vediamo nel seguito provano a risolvere entrambi i problemi, ovviamente si potrebbe anche pensare di risolverne solo uno.

2.3.2 Multilaterazione

Se riuscissimo a stimare la distanza tra un device mobile tra una e più antenne potremmo usare tecniche di multilaterazione per stimare la distanza dall'antenna (la multilaterazione geometricamente è un'intersezione tra le aree, nel nostro caso delle corone circolari, perchè abbiamo un raggio entro cui il device percepisce il segnale radio). Ma come faccio a stimare la distanza da un antenna? usando l'RSSI, esiste un'equazione che mette in relazione la potenza del segnale ricevuto con tutta una serie di altre cose. Inoltre mi aspetto che la potenza del segnale decresca con la distanza.

La potenza del segnale ricevuto la conosciamo, la potenza del segnale inviato lo sappiamo, le caratteristiche delle antenne del device la conosciamo (più o meno), la distanza non la sappiamo. Fino a qui andrebbe tutto bene

perchè vorrebbe dire avere un'equazione ad un'incognita, il problema è che c'è un fattore di correzione di come viene ricevuto il segnale, si chiama "path lost factor", che dipende da una miriade di caratteristiche ambientali difficilissime da prevedere. Può avere valori diversi, in ambienti aperti vale 2 mentre in ambienti chiusi varia tra 4 e 6. Riusciamo quindi solamente a stimare la distanza, c'è in realtà il problema che in alcuni casi non varia tra 4 e 6, ma può anche essere minore di 2 o maggiore di 6 in indoor. Inoltre ci sono una serie di fattori dinamici che rendono impossibile stimare questo valore (quante persone ci sono nella stanza?). Un altro fattore che lo influenza è l'umidità dell'aria.

Questo vale anche per il bluetooth, è una misura molto variabile. In pratica, nella maggior parte degli ambienti indoor il calcolo della distanza basato su RSSI è talmente impreciso da rendere difficile l'adozione di questa tecnica.

2.3.3 WiFi Round Trip Time

Una soluzione esisterebbe grazie ad un HW specifico, Google aveva proposto una modifica al protocollo WiFi per permettere il calcolo del round trip time. Questo era molto meno influenzato dalle interferenze. Usando delle antenne che supportano questo protocollo diventava possibile stimare con grande precisione la distanza tra un device ed un antenna. Così bastava avere tre antenne riuscivamo a stimare la posizione precisamente. C'è una demo ma poi negli anni questa cosa si è persa.

2.3.4 Fingerprinting

L'intuizione è che se ho un'antenna e io mi sposto il valore di RSSI non cambia in maniera lineare e predittiva in base alla mia posizione. Però più o meno, se io vedo una certa potenza del segnale in un punto, quando torno in quel punto vedrò la stessa potenza del segnale. Quindi se metto insieme varie potenze del segnale di varie antenne che percepisco in un punto queste saranno abbastanza caratteristiche e sarà molto difficile che ci saranno altri punti in cui percepisco la stessa potenza da quelle antenne.

Quindi supponiamo di conoscere una mappa e sapere di ogni punto di quella mappa il valore di potenza del segnale che percepisco dalle antenne, una volta che percepisco la potenza delle antenne capisco dove sono sulla mappa.

Quindi si divide in due fasi:

- Creazione della mappa, setup

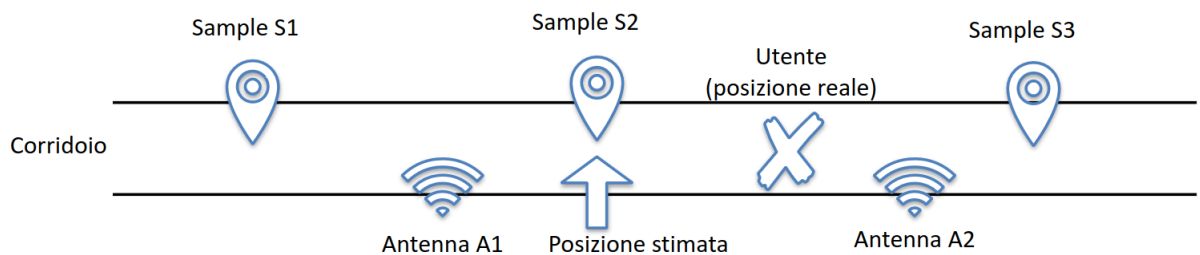
- Uso

Si chiama così perchè ho l'idea che in ogni punto so un'impronta digitale della potenza del segnale. La coppia potenza del segnale - id dell'antenna è chiamato fingerprint: $[id, RSSI]$. Quando creo la mappa a questo insieme di coppie ho associato una posizione: $\langle posizione, [id, RSSI] \rangle$.

Radio map

Il primo obiettivo è creare la radio map, rientra nella fase di setup, ma chi la crea? non l'utente finale. Il modo più semplice è avere un operatore che ha in mano una mappa, indica dove si trova, in quel punto il dispositivo prende la potenza del segnale da varie antenne e si salva le coppie id antenna e potenza del segnale e le associa a quel dato punto. In realtà l'operatore deve fare un giro su se stesso perchè il suo corpo potrebbe impattare la potenza del segnale.

Vediamo un esempio, siamo in un corridoio e l'operatore ha preso tre posizioni:



- $\langle S1, [\langle A1, 0,5 \rangle, \langle A2, 0,01 \rangle] \rangle$
- $\langle S2, [\langle A1, 0,6 \rangle, \langle A2, 0,4 \rangle] \rangle$
- $\langle S3, [\langle A1, 0,1 \rangle, \langle A2, 0,8 \rangle] \rangle$

Abbiamo creato la radio map e ora dobbiamo calcolare la posizione, la tecnica più semplice è calcolare il sample che assomiglia di più a quello che vedo io adesso, calcolo la distanza facendo la somma delle distanze quadratiche, questo perchè penalizzo molto gli errori grandi. Per semplicità dei calcoli facciamo una somma delle distanze:

$$S1 : |0,3 - 0,5| + |0,7 - 0,01| = 0,2 + 0,699 = 0,899$$

$$S2 : |0,3 - 0,4| + |0,7 - 0,6| = 0,1 + 0,1 = 0,2$$

$$S3 : |0,3 - 0,1| + |0,7 - 0,8| = 0,2 + 0,1 = 0,3$$

Quindi l'utente percepisce un fingerprint e calcola la distanza con tutti gli altri finger print della radio map e vedo quello con valore minore, significa che sono lì vicino. Nel nostro esempio sarà il sample S2 ad essere scelto.

Dobbiamo capire se becchiamo quello giusto, ma se anche becchiamo quello giusto quanto è precisa questa tecnica? stiamo usando le posizioni nella radio map come se discretizzassero lo spazio, cioè possiamo essere nei punti ma non potrò mai stare in mezzo.

Quindi una soluzione più avanzata è fare la media pesata, dando un peso che è inversamente proporzionale della similarità. In bidimensionale calcoliamo un baricentro tra tutti i sample dove però peso il calcolo della distanza in base al baricentro.

$$\frac{0,2}{(0,2 + 0,3)}$$

Rimane un calcolo molto soggetto a rumore (per eventi ambientali come una porta che si apre), nonostante questo mettendo insieme tante antenne riesco a mediare bene. Per tanto tempo questa è stata considerata la tecnica più solida. In realtà ad oggi si possono addestrare modelli di ML che provano a capire dove sta l'utente. Una volta che abbiamo la radio map addestriamo il modello su questa e lui stima la posizione dell'utente.

Il terzo problema è che la fase di setup è molto onerosa. Proviamo a fare un esempio di un centro commerciale di circa 100k metri quadri e vogliamo fare un fingerprint ogni metro, se ci mettessimo 10s per ogni fingerprint ci vorrebbero circa 34 giorni di lavori full-time.

Sono state proposte delle tecniche per ovviare a questo problema:

- Interpolazione: l'operatore non deve dire in ogni punto dove si trova, ma indica la posizione iniziale poi cammina in modo costante e l'applicazione prende delle misurazioni in mezzo ad intervalli regolari
- Robot: un sistema automatizzato che si sposta nell'ambiente e fa le letture. Usiamo dell'HW apposito che ci aiuta a creare la mappa. Il vantaggio è che va in giro da solo e può avere dell'HW apposta per calcolare la posizione.
- Crowdsourcing: si fa in modo che sia l'applicazione degli utenti finali a comunicare i fingerprint

2.3.5 Recap tecniche basate su segnale radio

<i>Tecnica</i>	<i>Segnale</i>	<i>Errore ODG</i>	<i>Complessità</i>	<i>Costo (setup, hardware)</i>	<i>Note</i>
<i>Prossimità</i>	<i>WiFi</i>	<i>100m</i>	<i>Molto bassa</i>	<i>Molto basso</i>	<i>Errore troppo elevato Implementabile nei sistemi attuali?</i>
	<i>BT</i>	<i>10m</i>	<i>Molto bassa</i>	<i>Medio</i>	<i>Richiede acquisto e manutenzione beacon</i>
	<i>NFC</i>	<i>cm</i>	<i>Molto bassa</i>	<i>Medio</i>	<i>Richiede azione esplicita Calcolo sporadico della posizione API non consentono accesso a RSSI</i>
<i>Multilaterazione</i>	<i>WiFi</i>	<i>10m</i>	<i>Bassa</i>	<i>Molto basso</i>	
	<i>BT</i>	<i>10m</i>	<i>Bassa</i>	<i>Medio</i>	<i>Errore non significativamente migliore a tecnica basata su prossimità</i>
	<i>WiFi RTT</i>	<i>m</i>	<i>Bassa</i>	<i>Alto (se si cambiano gli access point)</i>	<i>Implementabile su iOS?</i>
<i>Fingerprinting</i>	<i>WiFi</i>	<i>m</i>	<i>Alta</i>	<i>Alto</i>	<i>API non consentono accesso a RSSI</i>
	<i>BT</i>	<i>m</i>	<i>Alta</i>	<i>Molto alto</i>	<i>Richiede acquisto e manutenzione beacon</i>

2.4 Calcolo dello spostamento

Abbiamo già visto come le tecniche inerziali permettono di calcolare uno spostamento, a patto che l'utente tenga il telefono in mano. In letteratura sono state proposte delle soluzioni alternative di calcolo dello spostamento basate sul conteggio dei passi. L'idea è che se riesco a stimare com'è orientato l'utente, faccio un'assunzione su quanto sia lungo un passo (o provo a calcolarlo) e seguo i passi che fa l'utente allora magari riesco a stimare come si sposta. Rimane soggetto a errori, però in realtà quando sono fermo non continua a contare passi, a differenza di come faceva l'accelerometro, perchè ogni volta che conta un passo ripristina i valori che riceve. Ci sono due classi iOS (CMPedometer) e Android (StepCounter) che sono sensori virtuali che ci aiutano a contare i passi.

Esistono delle tecniche per stimare la lunghezza del passo, non sono incluse in queste librerie. Un modo è stimare la lunghezza del passo in base ai sistemi inerziali, un altro modo è usare delle tecniche ibride.

2.5 Tecniche ibride

Sono rari i casi in cui una sola delle tecniche vista fino ora sia sufficiente da sola. Spesso si usa una combinazione di due o più tecniche per trarre i

vantaggi di ciascuna minimizzandone i limiti.

Una domanda che fa spesso è di ragionare su queste tecniche di calcolo, e la risposta tipica è di usare una tecnica ibrida che usa tutto. Una tecnica molto complicata ha costi molto alti, quindi bisogna bilanciare benefici e costi anche nella scelta delle tecniche ibride.

Vediamo alcune combinazioni particolarmente sensate.

2.5.1 Segnale radio + computer vision

Usiamo una tecnica molto semplice basata sul segnale radio, la proximity con il segnale WiFi che mi fa capire in che stanza sono, poi posso guardare il cartello (un marcatore implicito) e capire che sono in quella stanza in quella posizione, usando tecniche di computer vision.

2.5.2 Marcatori visivi + calcolo dello spostamento

Uno dei limiti principali dei marcatori visivi era che l'utente doveva fare una azione esplicita per vedere il marcatore, poi tra due marcatori non sappiamo la posizione dell'utente.

Usando marcatori visivi so esattamente dove mi trovo, poi con tecniche viso-inerziali tengo traccia di dove si sposta l'utente. Dopo un po' la precisione degraderà e l'utente dovrà inquadrare un nuovo marcatore.

Si usano le librerie di AR per usare il calcolo dello spostamento.

Queste tecniche sono molto promettenti ma non sono mai state presentate in letteratura. L'intuizione è che uno dei limiti del fingerprinting è la creazione della radio map, che tra l'altro andrebbe aggiornata nel tempo. Una soluzione potrebbe essere di far fare la radio map agli utenti finali. Che se ci pensiamo è un po' quello che avviene nell'outdoor positioning (funziona perchè sentiamo gli access point wifi e abbiamo la posizione tramite il GPS). Si può fare anche in indoor, ma non abbiamo la posizione del GPS, una soluzione potrebbe essere quella di utilizzare più tecniche combinate.

2.6 Tecniche di aggregazione di dati soggetti a rumore

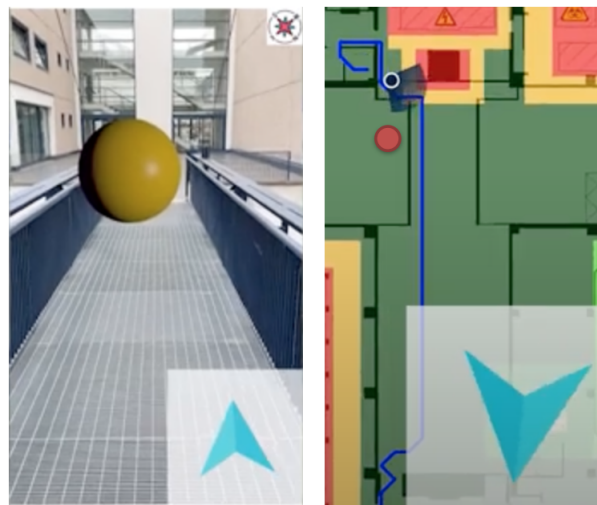
Ci concentriamo su come fare ad aggregare i dati provenienti da varie tecniche. Vorrei avere tante fonti di un dato, sapendo che sono tutte rumorose e vorrei riuscire ad aggregarle per avere una stima robusta di quale sia la mia posizione corrente. Questo problema è un problema da porci in modo più generale, tutte le volte che ho un dato che voglio misurare e ho tante

sorgenti che mi danno questo dato come faccio a stimare questo dato per capire quanto è preciso.

Per esempio questo è un argomento che gli ingeneri studiano tantissimo. Immaginiamo di avere un aereo con tanti sensori che leggono valori, questi valori possono essere sporchi, riesco a fare una stima robusta di qual'è lo stato dell'aereo?

Andremo a vedere un nuovo strumento che potremo usare per rimappare un problema su una soluzione simile a quella per l'indoor positioning.

C'è un'altra motivazione per cui quello che andremo a fare è molto rilevante. Partiamo da un esempio, l'esempio della navigazione nell'impianto industriale, passando sopra un ponte l'applicazione ha calcolato la posizione giù dal ponte, l'effetto era che suggerisse di buttarsi giù dal ponte. Come si fa a risolvere un problema del genere? fino ad ora abbiamo visto tecniche che ci dicono dove siamo al tempo corrente, non abbiamo mai preso in considerazione l'aspetto temporale. Inoltre non abbiamo mai preso in considerazione la conoscenza della mappa dell'ambiente.



2.6.1 Modello Bayesiano

Iniziamo a capire quali sono due comuni strumenti matematici che ci possono aiutare a modellare il problema, il primo strumento è il modello Bayesiano. L'idea di un modello Bayesiano è di avere delle osservazioni nel tempo (es: da uno o più tecniche di posizionamento), $z_1...z_n$. Quello che vogliamo calcolare è il mio belief $B(x_t)$, cioè quanto io creda che l'utente sia in un certo punto x dello spazio ad un certo tempo t . In questo modello andiamo a dire che la

probabilità del nostro belief è uguale alla probabilità di stare in quel punto condizionata a tutte le letture che abbiamo fatto:

$$B(x_t) = P(x_t | z_1 \dots z_n)$$

Questo P è una distribuzione di probabilità, quindi vuol dire che la somma per tutte le x è 1, siccome siamo sul continuo devo fare l'integrale. Il problema è come andare a calcolare questo $B(x_t)$ per ogni posizione.

2.6.2 Modello Markoviano

Notiamo che questo modello è molto generico e molto espressivo, è anche troppo espressivo. Secondo questo modello per calcolare il nuovo spostamento dovremmo usare tutti i sample che avevo preso nel tempo, se ho il sistema acceso da tanto devo usare tanti dati, l'idea potrebbe essere usare l'ultima osservazione. In realtà ci sono dei problemi in cui ho veramente bisogno di tenere traccia di tutta la storia degli spostamenti.

Quindi possiamo usare un modello più specifico, che è il modello Markoviano, il quale mi dice che la probabilità di stare in un certo punto non dipende da tutta la storia delle osservazioni che ho fatto in precedenza, ma dipende dal mio belief su tutto x al tempo $t-1$, ovvero $B(x_{t-1})$ e sull'ultima osservazione che ho fatto z_t . Non serve guardare tutta la storia ma basta che guardo cos'era la mia idea della posizione a $t-1$ e poi guardo la mia ultima osservazione. Se so che era in un'area probabilmente alla prossima osservazione sarà lì se è stato fermo, oppure se si è spostato allargando l'area. Teniamo traccia dello stato precedente e dell'osservazione corrente, la cosa importante è che non mi interessa COME sono arrivato allo stato precedente, ma mi interessa solo QUALE sia quello stato e quale sia l'ultima osservazione.

Modello di transizione di stato

Dobbiamo provare a dire come calcolare il nostro $B(x_t)$ secondo il modello Markoviano. Dividiamo il problema in due parti:

- Calcolare la probabilità che mi trovi in un qualunque punto dello spazio (qualunque posizione del sistema), senza sapere qual'è stata l'ultima osservazione. Consideriamo solo lo stato precedente senza aver fatto ancora l'osservazione.
- Qual'è la probabilità di osservare una misurazione z_t se ci troviamo nel punto x_t ?

Intuizione: facciamo finta di voler stimare la posizione di un treno che va da Milano a Torino (monodimensionale), al tempo $t - 1$ abbiamo un belief che è una gaussiana, vuol dire che è probabile che sia al centro della gaussiana. Ma dopo 10 secondo quanto sarà cambiata questa probabilità? non troppo, perchè noi abbiamo un intuizione sapendo che il treno ha una velocità limitata. Quindi quello che stiamo dicendo al primo punto è che potremmo provare a stimare un $B^-(x_t)$ (fatto prima dell'osservazione) che dice dove sarà il treno dopo poco. Dobbiamo usare un modello di transizione di stato.

L'idea quindi è modellare il modello di transizione di stato che ci dice la probabilità di essere in x al tempo t è condizionata dalla probabilità di essere in x al tempo $t - 1$. È la probabilità di essere arrivati in x_t partendo da $t - 1$. Possiamo chiamarla anche probabilità dello spostamento.

Quindi per calcolare il belief prima di aver preso l'osservazione devo partire dalla mia posizione precedente e dove sono, quindi tramite il modello di transizione di stato, questo modello lo calcoliamo in base al caso. Di fatto vado a fare per ogni possibile x , prendo la probabilità di trovarsi in x a t moltiplicato la probabilità di stare in x a $t-1$, e questo lo faccio per tutte le x . Dato che siamo nel continuo diventa un integrale:

$$B^-(x_t) = \int B(x_{t-1}) \cdot P(x_t|x_{t-1})dx_{t-1}$$

B^- a questo punto non è più una distribuzione di probabilità.



Vediamo un esempio nello spazio monodimensionale, voglio calcolare $B^-(x_t)$. Consideriamo uno spazio discreto. Ho tre posizioni al tempo $t - 1$: x^a, x^b, x^c con $B(x^a) = 1/2, B(x^b) = 1/6, B(x^c) = 2/6$. Dobbiamo calcolare ora la probabilità di spostamento per essere arrivati a x_t per ogni osservazione che abbiamo:

$$\begin{aligned} P(x_t|x^a) &= 0 \\ P(x_t|x^b) &= 0,1 \\ P(x_t|x^c) &= 0,5 \end{aligned}$$

A questo punto possiamo calcolare il modello di transizione di stato moltiplicando i belief per la probabilità di spostamento:

$$B^-(x_t) = 1/2 * 0 + 1/6 * 0,1 + 2/6 * 0,5$$

Modello percettivo

Definiamo un modello percettivo che mi dice qual'è la probabilità di fare un osservazione z_t condizionata dal fatto che sono in x_t . Quindi usiamo il modello percettivo per trovare per ogni punto in cui abbiamo calcolato in B- qual'è la probabilità che mi trovo in x_t : $P(z_t|x_t)$. Attenzione è facile confondersi. Il modello percettivo NON stima la probabilità di essere in x_t , ma la probabilità di osservare z_t se siamo in x_t .

Quindi è la moltiplicazione tra la probabilità che ho di trovarmi in un certo punto senza aver fatto l'osservazione e la probabilità di trovarmi nello stesso punto condizionata dal fatto che ho effettivamente fatto la nuova osservazione. Quindi prima calcoliamo un valore $B^+(x_t)$ che rappresenta la predizione di belief considerando il modello di transizione di state e il modello percettivo:

$$B^+(x_t) = p(z_t|x_t)B^-(x_t)$$

Poi, dato che non è normalizzato (potrei ottenere valori piccolissimi), calcolo $B(x_t)$ normalizzando i valori di $B^+(x_t)$:

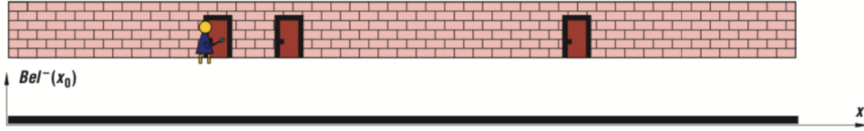
$$n = \int B^+(x_t)dx_t$$
$$B(x_t) = \frac{B^+(x_t)}{n}$$

2.6.3 Esempio

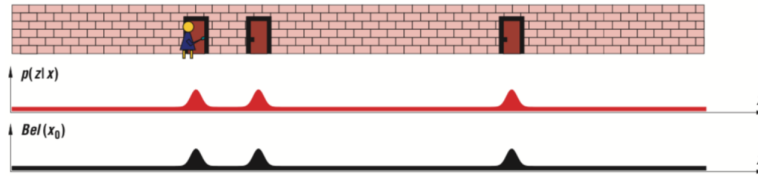
Vediamo un esempio, l'idea è la seguente:

- Siamo in uno spazio mono-dimensionale
- C'è un sensore che ti avvisa quando sei vicino ad una porta, ma non sai a quale porta
- In qualche modo il sistema sa anche come si sta spostando l'utente
- Nelle immagini la funzione B è chiamata "Bel" (sta sempre per Belief)

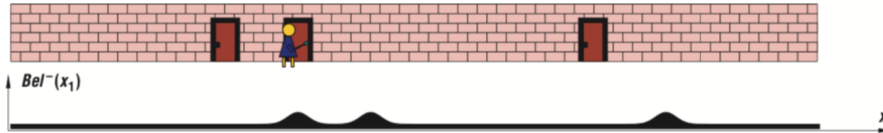
Questa è la situazione iniziale:



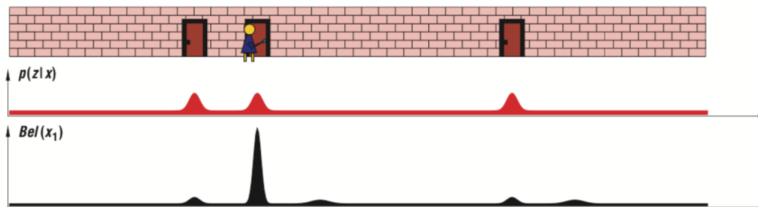
Alla prima osservazione sappiamo che abbiamo una probabilità più alta di essere vicino ad una porta, il sensore può sbagliare, dunque la probabilità lontano da una porta è bassa, ma non zero. Questo è catturato da $p(z|x)$. In questo caso $B(x_0)$ coincide con $p(z|x)$ perchè la distribuzione precedente $B^-(x_0)$ è uniforme.



Il sistema percepisce che l'utente si è spostato verso destra. (i picchi rispetto all'osservazione precedente sono meno marcati, perchè anche nell'osservazione dello spostamento ci possono essere degli errori):



Quando viene fatta la seconda osservazione, $p(z|x)$ non cambia rispetto a prima (l'osservazione è identica, l'utente è vicino ad una porta). Ora però $B(x_1)$ viene calcolato partendo da una distribuzione precedente non uniforme, otteniamo un altro picco alto dove un picco su $P(z|x)$ coincide con un picco di $B^-(x_0)$.



2.7 Calcolo della posizione adottando il modello Markoviano

La prima soluzione che ci viene in mente è discretizzare lo stato. Ad esempio se siamo in una stanza la dividiamo in quadratoni, quindi la prima cosa da fare è calcolare il modello di transizione di stato, dobbiamo calcolare la funzione che trova la probabilità di spostarsi nella stanza. La funzione di probabilità possiamo inventarcela noi, spostarsi di 1m/s è probabile poi decade. Quindi posso calcolare la probabilità di essere andato in un quadrato. Nel caso peggiore la complessità computazionale sarà n^2 , perchè devo calcolare la probabilità e tutte le osservazioni passate. Posso anche creare un modello percettivo, se conosco le celle che stanno a 100 metri hanno probabilità 1 di essere viste, 0 altrimenti. É banale ma quantomeno è un'implementazione del modello.

Questo è il modo più tipico in cui un informatico potrebbe affrontare il problema. Quello che vogliamo vedere ora è un altro approccio che in un certo senso discretizza, ma non nel senso che ci viene naturale. Andiamo in un certo senso a discretizzare la probabilità di essere in un posto. É un'idea che sarà difficile da digerire, ma quando l'abbiamo capita e pensiamo all'implementazione ci verrà molto facile. Discretizzando le celle abbiamo due problemi:

- Quanto fare grande la cella? Tenzialmente vorremmo fare la cella più piccola possibile, così abbiamo una precisione maggiore per capire dove sta l'utente nella cella.
- Scala male in base al numero di celle, più sono le celle più il n^2 si fa sentire.

In questo sistema continuiamo a calcolare la probabilità con lo stesso costo computazionale anche nei punti in cui l'utente è altamente improbabile che ci sia, mentre quello che vediamo ora si focalizza su dove è più probabile che sia l'utente.

Tutto aumenta ancora di complessità se abbiamo anche la dimensione verticale (se l'utente può spostarsi anche in verticale), inoltre se dovessimo discretizzare anche l'orientamento dell'utente? aumentiamo ancora le dimensioni, quanti spicchi dell'angolo giro consideriamo? Stiamo andando ad aumentare esponenzialmente il numero di celle. Questa tecnica non scala bene all'aumentare dello spazio che vogliamo coprire e all'aumentare delle dimensioni. Quindi va bene per problemi a basse dimensionalità.

Ci sono 3 approcci principali per implementare un modello Markoviano:

- Filtri di Kalman. Ci interessano meno perchè hanno bisogno di assunzioni che stridono con l'indoor positioning
- Sistemi discreti che rappresentano lo spazio come una griglia o un grafo
- Particle filter

2.7.1 Particle Filter

Sono trovati in letteratura anche con nome di "Sequential Monte Carlo", quando applicato al calcolo della posizione "Monte Carlo Localization". È molto studiato per la robotica, quindi la troviamo spesso descritta in questo concetto.

L'idea è quella di avere un numero di ipotesi discrete, cioè al posto di discretizzare rispetto allo spazio, noi proviamo a dire: "supponiamo che lo stato sia questo" e poi ripetiamo per tanti casi possibili. Cioè creiamo un insieme di ipotesi ognuna pesata rispetto a quanto crediamo che quell'ipotesi sia plausibile. Ogni coppia (ipotesi e probabilità) la chiamiamo particella. La somma dei pesi è uguale a 1, quindi abbiamo una distribuzione di probabilità. Solo che le ipotesi sono discrete, non continue.

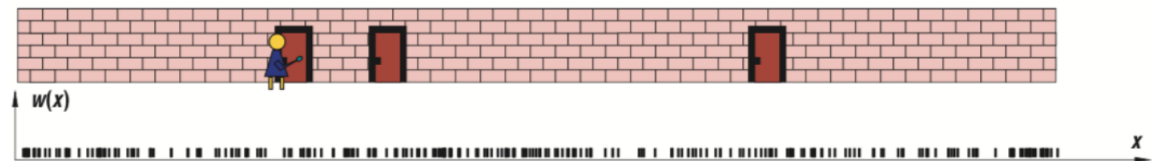
Poi lavoriamo in maniera iterativa. Ad ogni iterazione facciamo due cose:

- Teniamo le ipotesi più probabili
- Propaghiamo le ipotesi al passo successivo

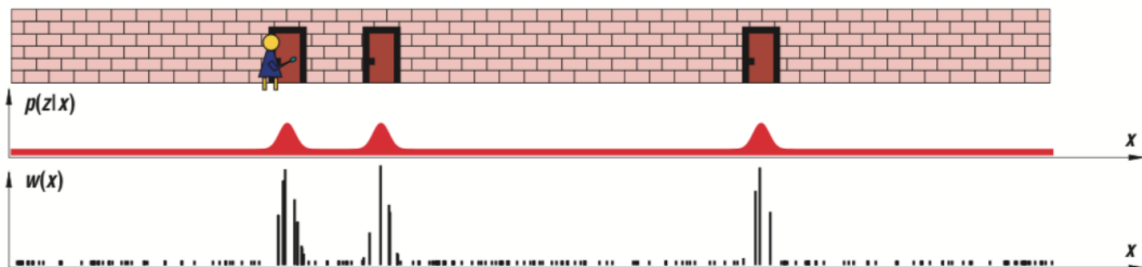
Vediamo l'esempio di un aereo che sa la distanza dal suolo e si muove in monodirezionale. All'inizio non sappiamo niente, quindi generiamo una serie di ipotesi casuali, poi misuriamo l'altezza dal suolo e andiamo a cambiare il peso di queste particelle rispetto alla probabilità che da quella particella si faccia quella osservazione, è proprio il modello percettivo, qual'è la probabilità di fare quell'osservazione condizionato dal fatto che sono in quel punto? Non abbiamo identificato il punto in cui siamo, perchè abbiamo tante particelle. Adesso dobbiamo fare un passaggio detto resampling, ricreiamo lo stesso numero di particelle, partendo da quelle date si sceglie una particella di quelle precedenti e si applica lo spostamento, ma di quanto lo spostamento? non lo sappiamo, randomicamente scegliamo un punto nell'intervallo che riteniamo probabile. Quando scelgo la particella di partenza la scelgo con probabilità proporzionale al peso, in pratica devo generare una nuova particella partendo da una vecchia, scelgo da quale partire in modo randomico ma con un peso dato dalla particella del turno precedente. Le rigenera tutte dello stesso peso. Poi itero, calcolo come prima qual'è il peso di quelle particelle. Quindi stiamo evitando di usare risorse computazionali per capire dove

sta l'utente in punti in cui è molto improbabile che sia lì. A questo punto prendiamo ognuna di queste particelle e la spostiamo, di un valore esatto scelto a caso secondo una certa distribuzione, questo ci ricorda il modello di transizione di stato. Arriviamo ad un punto in cui siamo abbastanza sicuri che sia in quel punto. Quando si trova sul mare la mia conoscenza sulla sua posizione si va ad approssimare.

All'inizio non sappiamo dov'è l'utente quindi mettiamo le particelle con pesi uguali secondo una distribuzione uniforme:



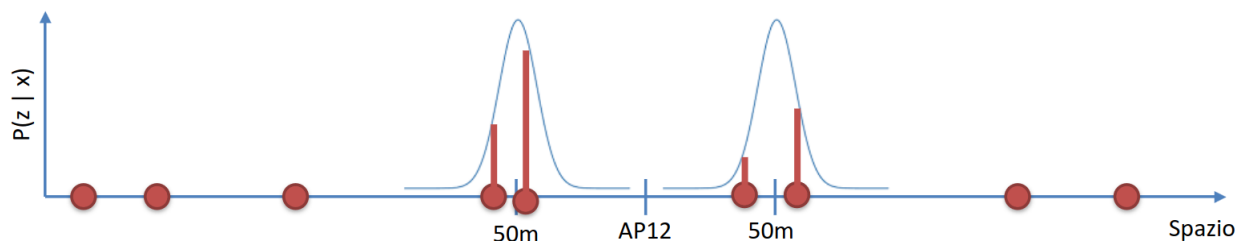
Poi quando arriva la prima osservazione ad ogni cella diamo il peso dando proporzionale al modello percettivo, che di nuovo non potremmo non poterlo definire nel continuo, ma possiamo farlo nel discreto, ovvero calcoliamo il modello percettivo per ogni particella:



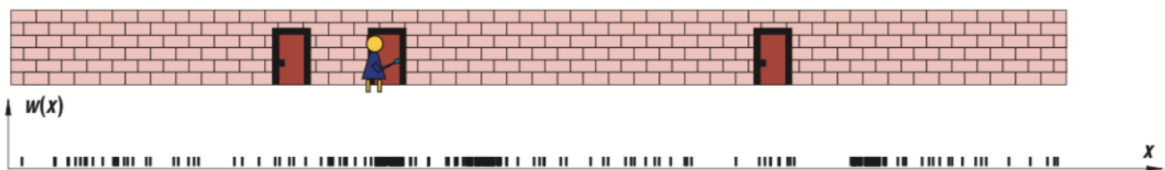
Vediamo un ulteriore esempio di applicazione del modello percettivo. In uno spazio mono-dimensionale abbiamo 9 particelle sparse nello spazio. Facciamo questa osservazione:

- Percepisco l'antenna WiFi con ID12
- Data la potenza del segnale stimo di essere all'incirca a 50 metri di distanza dall'antenna
- Modello questa incertezza con una gaussiana che mi dice la probabilità di essere in un intorno dei 50mt

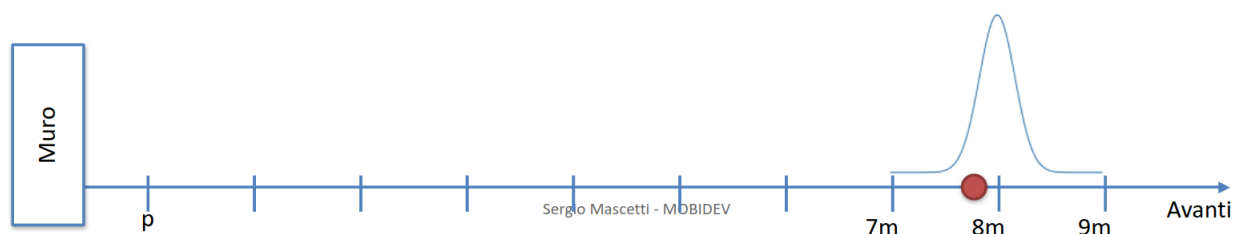
Assegno alle particelle un peso uguale a quello della gaussiana, cioè un valore molto basso, praticamente nullo alle particelle che non si trovano vicino ai 50mt di distanza dall'AP:



Tornando all'esempio precedente, dopo aver applicato il modello percettivo faccio il passaggio di resampling, scegliamo una particella e la scegliamo secondo la probabilità della particella, poi genero la particella nuova e applico uno spostamento, secondo il modello di transizione, ma anche in questo caso calcolo il modello solo quel sample lì. Tipicamente ho delle osservazioni di spostamento e dei dati relativi all'ambiente. La singola particella è lo stato singolo e l'incertezza sullo stato completo è rappresentato dalla distribuzione delle particelle nello spazio.

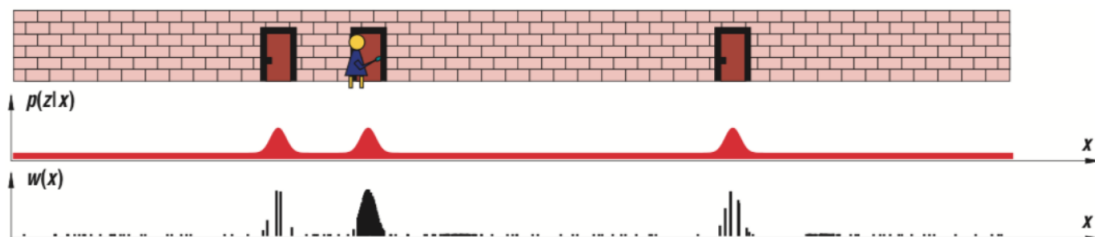


Ma data la particella p di partenza, come faccio a definire lo spostamento? Tipicamente abbiamo delle osservazioni di spostamento e dei dati relativi all'ambiente. Vediamo un esempio del calcolo dello spostamento. Siamo in uno spazio mono-dimensionale. Abbiamo contato 10 passi dell'utente tutti nella stessa direzione (non sappiamo se avanti o indietro). Partiamo da p . L'utente non può essere andati indietro, perchè c'è un muro. Può essere andato solo avanti. Sappiamo che un passo è all'incirca lungo 80cm. Modelliamo la probabilità di arrivare in un dato punto con una gaussiana. Scegliamo un punto di arrivo casuale all'interno della gaussiana seguendo la probabilità della gaussiana stessa. Il puntino rosso in figura rappresenta la nuova posizione di p :



Tornando all'esempio delle porte stiamo di fatto riuscendo a tenere traccia

della storia usando una serie limitata di ipotesi, la cosa bella è che dopo un po' di iterazioni avremo molte più particella attorno alla posizione corretta quindi useremo costo computazionale per calcolare posizioni in cui è probabile che sia l'utente scartando posizioni in cui è improbabile che sia:



2.7.2 Quanta incertezza?

Una domanda è quanta incertezza devo considerare nelle misurazioni? Qui abbiamo una range di approcci, vediamo gli estremi:

- Fare finta che i dati che ci arrivino siano molto precisi, quindi modelliamo poca incertezza. Quando vedo che l'utente è davanti alla porta metto a zero i pesi delle altre particelle. Questo mi permette di convergere molto velocemente alla soluzione corretta, però rischio di non arrivare alla soluzione corretta. Se i dati ogni tanto i dati si sbagliano rischio di mettere zero particelle intorno alla soluzione.
- Modello tantissima incertezza rischiamo di avere un sistema in cui non convergiamo mai alla soluzione o abbiamo tantissime iterazioni prima di convergere.

In realtà c'è sempre il problema che i pesi delle particelle vadano a zero, quindi quando si fa il resampling molte tecniche fanno in modo di generare una piccola percentuale di particelle nello spazio con distribuzione uniforme.

La complessità computazionale è quantomeno lineare con il numero delle particelle, quindi più particelle usiamo migliore è l'approssimazione che potremmo avere. Potrebbe succedere che ci siano zero particelle attorno allo stato vero. Per risolvere questo problema come detto prima facciamo resampling casuale e metto tante particelle, in modo che sia improbabile che neanche una stimi lo stato reale.

È ovvio che come visto per la discretizzazione tramite celle dobbiamo aumentare il numero delle particelle man mano che crescono le dimensioni del sistema. Però tipicamente per il calcolo della posizione il problema della dimensionalità si riesce a gestire abbastanza bene, perchè solitamente abbiamo

4 dimensione. Rispetto alla dimensione dell'ambiente cosa impatta? quello che potremmo pagare è che aumentando lo spazio potrebbe metterci un po' di più a convergere.

Ma quindi come faccio a dire dove si trova l'utente? si trova il baricentro delle particelle come possibile punto dell'utente e l'incertezza è calcolata con il raggio che ci contiene, per esempio, il 98% delle particelle.

Chapter 3

Human Activity Recognition

La human activity recognition è un insieme di tecniche finalizzate a capire qual'è l'attività che sta svolgendo l'utente. Le attività possono essere le più disparate, alcuni lavori in letteratura le distinguono in attività:

- Di basso livello: specifici movimenti (correre)
- Di alto livello: attività quotidiane (mangiare)

Quali sono le applicazioni:

- Well-being: app che riconoscono le attività del corpo e le usano per supportare l'attività fisica. O ad esempio app che contano le ripetizioni di un esercizio solo quando è svolto in maniera corretta.
- Health-Care: ad esempio riuscire a riconoscere il decadimento cognitivo, si prova a capire in un ambiente smart home (con vari sensori) si prova a capire cosa fa il paziente e se quelle attività possono prevedere un decadimento cognitivo
- Applicazioni context-aware: l'attività dell'utente è una delle informazioni che possono aumentare il contesto

Il contesto è:

- La posizione
- L'ora
- Informazioni del traffico
- ecc...
- L'attività svolta dall'utente

Facciamo un esempio di applicazione context-aware basata sul riconoscimento dell'attività. Ad esempio google maps memorizza il punto in cui abbiamo posteggiato. Tramite delle API del sistema operativo capisce qual'è l'attività, capisce che eravamo a bordo di un veicolo e poi stavamo camminando (tramite sensori inerziali). Questo è quello che faremo in questi giorni, capire come capire come si comporta l'utente utilizzando sensori inerziali e la camera.

Possiamo vedere alcuni esempi di attività:

Group	Activities
Ambulation	Walking, running, sitting, standing still, lying, climbing stairs, descending stairs, riding escalator, and riding elevator.
Transportation	Riding a bus, cycling, and driving.
Phone usage	Text messaging, making a call.
Daily activities	Eating, drinking, working at the PC, watching TV, reading, brushing teeth, stretching, scrubbing, and vacuuming.
Exercise/fitness	Rowing, lifting weights, spinning, Nordic walking, and doing push ups.
Military	Crawling, kneeling, situation assessment, and opening a door.
Upper body	Chewing, speaking, swallowing, sighing, and moving the head.

Esistono già delle API fornite dal sistema operativo per riconoscere delle attività dell'utente.

Nell'esempio vediamo che quando chiediamo al sistema operativo le attività ci restituisce un array di attività con la confidenza per ogni attività, la confidenza è un valore compreso tra 0 e 1 che dice quanto il sistema sia convinto che sia quella l'attività:

```
protected void onHandleIntent(Intent intent) {  
    ActivityRecognitionResult result = ActivityRecognitionResult.extractResult(intent);  
  
    ArrayList<DetectedActivity> detectedActivities = (ArrayList) result.getProbableActivities();  
  
    for (DetectedActivity activity : detectedActivities) {  
        Log.e(TAG, "Detected activity: " + activity.getType() + ", " + activity.getConfidence());  
        broadcastActivity(activity);  
    }  
}
```

Quali sono i limiti delle soluzioni esistenti?

- Limitato set di attività riconosciute
- Modello di riconoscimento non personalizzato sull'utente finale. Sono soluzioni generiche, sono state fatte per un generico utente e cercano di

capire qual'è l'attività dell'utente dopo aver imparato sulla base di un insieme di persone. Da un lato va bene perchè non abbiamo bisogno di ri-personalizzarle per ogni utente ma se abbiamo un utente che ha caratteristiche diverse non va bene.

- Nonostante sia già usato è un ambito di ricerca ancora attivo

3.1 Panoramica delle tecniche di riconoscimento delle attività

Affronteremo il problema con un approccio top-down, prima vedremo la soluzione ai problemi e poi vedremo i passaggi che portano alle soluzioni.

In queste lezioni non parleremo di tecniche di machine learning, ma le useremo come delle black box. Noi vedremo qual'è il processo che ci porta ad utilizzare tecniche di machine learning per risolvere il problema dell'human activity recognition.

Vedremo che abbiamo un flusso di dati da sensori inerziali e camera, ci serve una tecnica per capire che attività sta facendo l'utente usando quei dati. Quale tecnica potremmo usare?

3.1.1 Tecniche analitiche

Prima del machine learning uno dei modi per risolvere il problema era fare delle soluzioni basate su regole (tecniche analitiche). Era il modo in cui i programmatori avevano sempre risolto i problemi. Solitamente una cosa del genere non è fattibile senza il machine learning. Ci possono essere persone diverse che svolgono attività in modo diverso, la stessa persona può svolgere la stessa attività in modo diverso e possono esserci attività diverse che hanno pattern registrati dai sistemi inerziali molto simili, i dati inerziali sono soggetti a rumore. Devo quindi estrarre delle feature, il vantaggio delle reti neurali deep è che sono in grado di estrarre automaticamente le feature (alle tecniche non deep dobbiamo dare noi le feature e loro ce le pesano).

3.1.2 Uso del Machine Learning

É la tecnica sulla quale si basa la quasi totalità delle tecniche di human activity recognition. Il ML è un modo automatico per trovare queste regole usando la statistica. Vediamo alcuni concetti base.

Le tecniche di ML risolvono varie classi di problemi:

- Detection: trovare un oggetto in un immagine (o un evento nel tempo dati dei dati)
- Regressione: trovare correlazioni tra due o più variabili
- Classificazione: classificare cose, ad esempio classificare una mail spam

L'activity recognition è solitamente un problema di classificazione in cui ho un insieme possibili di classi (le attività) e sulla base dei dati ricevuto devo sceglierne una.

Le tecniche di ML possono essere:

- Non supervisionate: vogliamo fare qualcosa ma non abbiamo la ground truth, ad esempio il problema del clustering (voglio suddividere dei punti nello spazio in gruppi)
- Supervisionate: necessitano di dati etichettati per addestrare il modello
- Semi supervisionate

Le tecniche di activity recognition sono solitamente supervisionate.

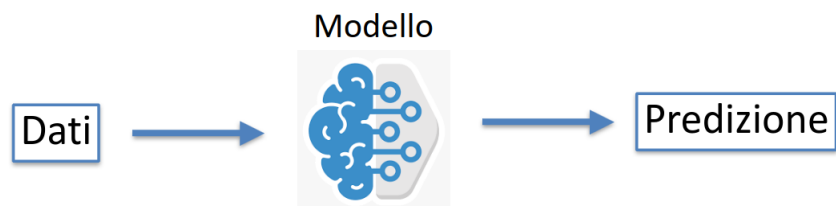
Addestramento

Abbiamo dei dati e un etichetta che è la conoscenza di ground truth che dice la classe corretta cui il dato appartiene, attenzione che la ground truth può essere piena di errori, le tecniche di ML dovrebbero essere resistenti ad etichette sbagliate. È possibile fare apprendere il modello fornendogli queste informazioni.



Classificazione

Vuol dire che in fase di classificazione diamo al modello i dati e lui ci dà la predizione, vuol dire che ci dà la confidenza di appartenenza ad ogni classe.



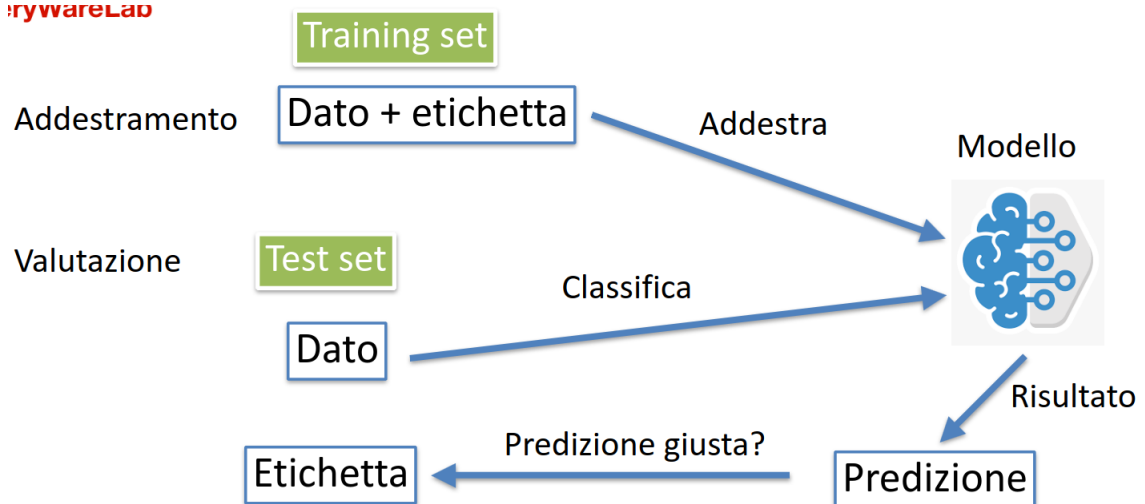
Valutazione delle qualità del modello

Un problema che abbiamo è capire quanto è valido il nostro modello. L'errore comune è fare overfitting sui dati di test, se facciamo la valutazione del modello sui dati di test otterremmo sicuramente degli ottimi risultati, ma sarebbero falsati. La cosa da fare è dividere il set di dati in due:

- Set di training
- Set di test (con le etichette, per capire se la risposta è giusta o sbagliata)
- In alcuni casi potremmo avere un set di validazione per capire quando finire il processo

Noi vorremmo che questi set fossero il più grande possibili. In realtà esistono tecniche di cross validation che permettono di limitare la scelta dei dati.

trywareLab



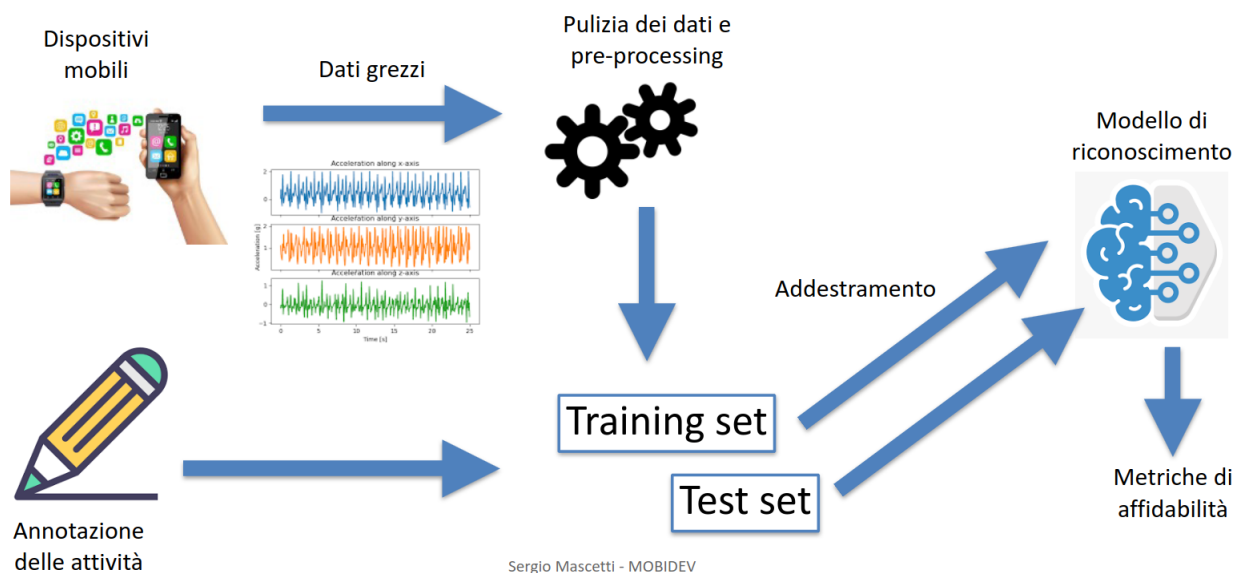
L'input del modello

Il sistema di riconoscimento prende dei dati in input, ma le tecniche di ML non possono prendere direttamente lo stream di dati, ma prendono in input

un insieme di features. È quindi necessario fare una serie di passaggi di pre-processing per pulire questi dati.

Formalizziamo meglio il problema di classificazione. Abbiamo un vettore X n -dimensionale che rappresenta il dato che vogliamo rappresentare (il feature vector) ed abbiamo un insieme y di classi, esiste una funzione teorica $f(x)$ (di cui non esiste un'implementazione) che dato un elemento di x qual'è la classe corretta a cui appartiene. Noi la funzione non ce l'abbiamo e vogliamo trovare una funzione $h(x)$ che approssimi $f(x)$ al meglio possibile, in questo caso h è un classificatore.

Noi abbiamo i dispositivi mobili con dei dati (inerziali o immagini), delle tecniche di pulizia dei dati e un'annotazione delle attività (fatta a mano):



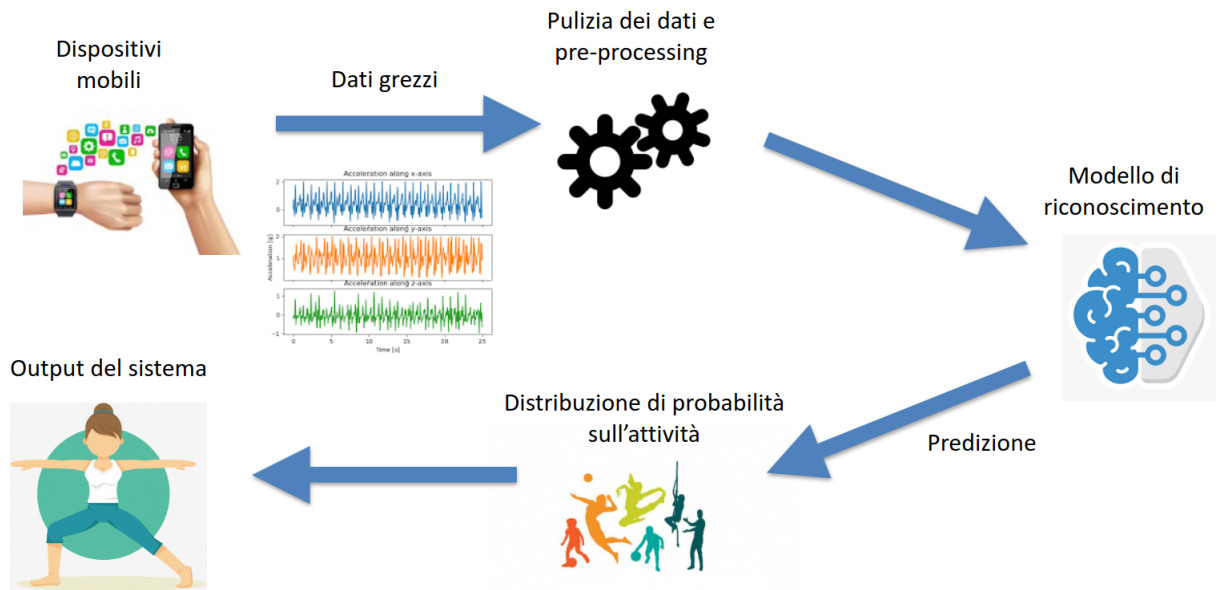
Tuning dei parametri

In tutte queste fasi compaiono un certo numero di parametri, anche detti iper-parametri, ovvero tutta una serie di variabili per cui dobbiamo trovare un valore per cui l'addestramento funzioni meglio. Per cui tutto l'operazione di addestramento e misura delle performance viene ripetuta più volte per cercare di ottimizzare le metriche che tiriamo fuori.

Fase di predizione

Abbiamo il dispositivo mobile che raccoglie dati grezzi, utilizziamo la tecnica di pre-processing utilizzata prima (fissando i parametri) tiriamo fuori delle feature, le do al modello il quale ci dà una distribuzione di probabilità e

sulla base dell'applicazione che stiamo facendo tiriamo fuori l'attività. Dalla distribuzione di probabilità non è scontato l'output del sistema, a volte scelgo quello con confidenza più alta a volte devo fare un ragionamento diverso (in un sistema medico la soglia di confidenza deve essere molto alta per essere sicuri):



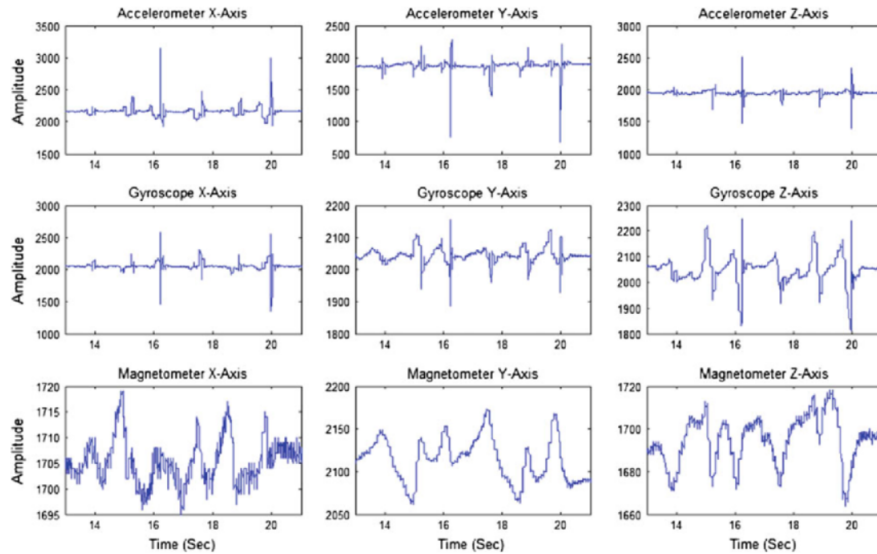
3.2 Acquisizione e pre-processing

Come vengono acquisiti i dati inerziali? il modo scontato è quello di acquisirli tramite dispositivi mobili i quali sono dotati di sensori inerziali e tipicamente hanno accelerometro, giroscopio e magnetometro. Ognuno di questi sensori produce dati sui tre assi.

In realtà spesso per ricerca si usano delle board (IMU) che contengono una piccola circuiteria e una batteria.

Accedere ai dati inerziali sui dispositivi mobili è banale, si registra un listener, c'è un observer che ci notifica quando sono disponibili i dati.

I dati grezzi sono fortemente soggetti a rumore. Inoltre nei dispositivi mobili abbiamo 9 grafici per i dati (1 per ogni asse di ogni sensore):



Con quale frequenza prendiamo i dati dai sensori, è un problema di discretizzazione in informatica. Quando dobbiamo campionare il segnale audio prendo ogni secondo tanti valori e memorizzo il valore di ogni campione. Per non esagerare con il numero di campioni si trova il limite secondo il cui non si percepisce un reale vantaggio nell'avere altri campioni. Esistono risultati in letteratura che oltre i 20Hz non si introducono significativi miglioramenti nel riconoscimento.

3.2.1 Allineamento temporale

Se prendiamo i dati succede che le letture dei dati dei vari sensori non avvengono allo stesso istante. Però abbiamo lo stesso clock, per come utilizzeremo i dati (mettendoli in blocchi) il fatto che non arrivino allo stesso istante non è un problema.

Quando abbiamo più device il cui clock non è allineato è più complicato, vedremo una possibile soluzione a questo problema.

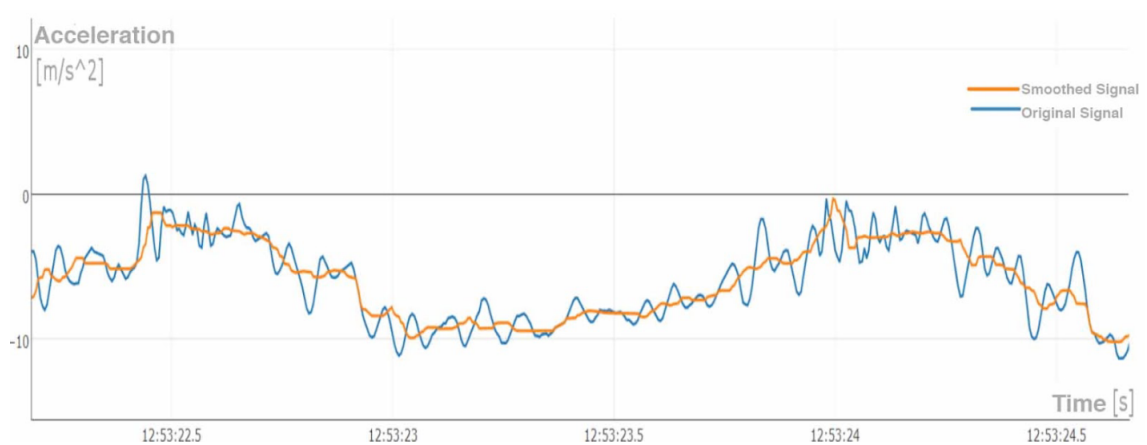
3.2.2 Pre-processing

Come detto l'obiettivo della fase di pre-processing è quello, dato uno stream di dati inerziali in input, produrre un feature vector da usare con le tecniche di ML. Ci sono 4 passaggi principali:

- Data cleaning
- Allineamento temporale
- Segmentazione
- Estrazione delle features

Data cleaning

Sono tecniche per smussare il segnale e rimuovere del rumore. Conviene applicare dei filtri, quello più famoso è il filtro mediano in cui scelgo 5 dati in cui ho quello corrente e ne faccio la mediana (mette in ordine i dati e prende quello in mezzo), perchè la mediana e non la media? perchè utilizzando la mediana non diamo tanto peso agli outlier.



Quale filtro usare? se usarlo? fa parte della calibrazione del sistema, dobbiamo usare vari filtri con vari parametri per capire con quale funziona meglio.

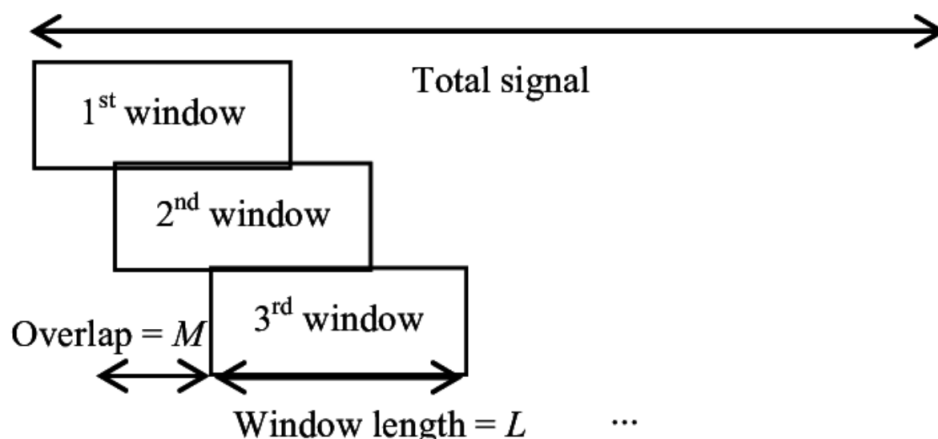
Allineamento temporale

I dati se arrivano da device diversi hanno clock diversi. Esistono delle tecniche di allineamento del clock, ma non tutti i dispositivi sono programmabili. La tecnica empirica che viene usata è quella del CIACK (nel cinema serviva per allineare audio e video), attacchiamo tutte le board ad un oggetto e facciamo fare un movimento inconfondibile all'oggetto per poter avere un chiaro segnale su cui allineare le osservazioni.

Segmentazione

È forse la cosa che ha un impatto maggiore sulle performance. Dobbiamo dare alle tecniche di ML un insieme di feature, non le calcoliamo istante per istante ma aggregiamo i dati all'interno di finestre temporali, aggregiamo queste feature e queste le diamo a modelli di ML.

L'idea di base è avere delle finestre temporali contigue (dal secondo 0 al 2, 2-4, 4-6 ...), il problema è che se abbiamo un'attività che si accavalla tra due finestre rischiamo di avere il movimento che vogliamo riconoscere in due finestre diverse. Per risolvere questa cosa si usano le SLIDING WINDOWS, l'idea è di fare partire una finestra quando la precedente è ancora in corso. Mettiamo un certo livello di overlap per avere tutti i dati che mi servono in un'unica finestra. Solitamente una scelta comune è avere un overlap del 50%.



Quindi mi interessano due parametri:

- La percentuale di overlap
- La dimensione delle finestre

Questi parametri vanno scelti caso per caso, perchè in base all'attività che vogliamo riconoscere vogliamo cambiare la lunghezza delle finestre. Tipicamente delle finestre più lunghe ci aiutano ad essere sicuri che l'attività sia avvenuta in quella finestra, ma hanno dei problemi perchè l'attività potrebbe cambiare nella stessa finestra.

Facciamo due esempi. Abbiamo un app per tenere traccia di quanto tempo viene dedicato alle attività domestiche, abbiamo delle attività lunghe, le attività potrebbero non essere facili da riconoscere perchè ci aspettiamo che non cambino molto rapidamente quindi probabilmente non abbiamo necessità di finestre brevi ma potremmo avere bisogno di finestre un po' lunghe

per catturare movimenti caratteristici che non avvengono sempre (mestolo). Finestre di 5 o 10 minuti potrebbero anche andare bene ed un overlap del 50% andrebbe bene.

Facciamo finta che ci sia un trend di fare un salto ogni tanto, vogliamo fare un app che conta per quanto tempo l'utente cammina e quanti salti fa. Questo caso è più problematico perchè sono due attività di lunghezza diversa. Per camminare ci andrebbero bene anche finestre lunghe, mentre il salto è breve e ci servono finestre brevi. Inoltre siccome l'app deve essere responsiva riconoscendo quello che sta facendo l'utente quindi finestre troppo lunghe non andrebbero bene. Dobbiamo quindi ridurre la lunghezza delle finestre andando ad avere un'idea della durata dell'evento più breve che dobbiamo riconoscere. Però per il problema che dicevamo prima dovremmo avere un overlap più marcato in modo da avere una finestra che contenga veramente l'azione.

Perchè non proviamo a creare delle finestre in modo dinamico? idealmente ci piacerebbe che una finestra inizi quando inizia l'attività e finisce quando finisce l'attività, ma per fare questo dovremmo sapere a priori quando inizia l'attività. Esistono delle tecniche di ML in cui alleniamo il modello a riconoscere i limiti delle attività (ovvero tipicamente quando inizia e finisce una particolare attività), sono casi veramente rari.

Estrazione delle features

Devono avere due caratteristiche principali:

- Quantità fissa: tra una finestra e l'altra devo avere lo stesso numero di features
- Devono essere informative di quanto è avvenuto per permettere di discriminare l'attività

O ci sono casi molto particolari in cui possiamo inventarci qualche feature nuova, solitamente si prende un insieme di feature definite in letteratura e almeno alcune di queste catturano la nostra attività. Le feature vengono divise in tre gruppi principali:

- Time-domain: media, deviazione standard, varianza per ogni asse
- Frequency-domain: trasformata di fourier
- Altre

Abbiamo alcune features specifiche per un asse ed altre che confrontano assi tra di loro.

Quindi per ogni dispositivo si calcolano le feature per ogni finestra di ogni sensore inerziale, concatenano queste feature per ogni asse di ogni sensore di ogni device e vado così a formare il FEATURE VECTOR. Per ogni sensore possiamo usare qualche decina di features.

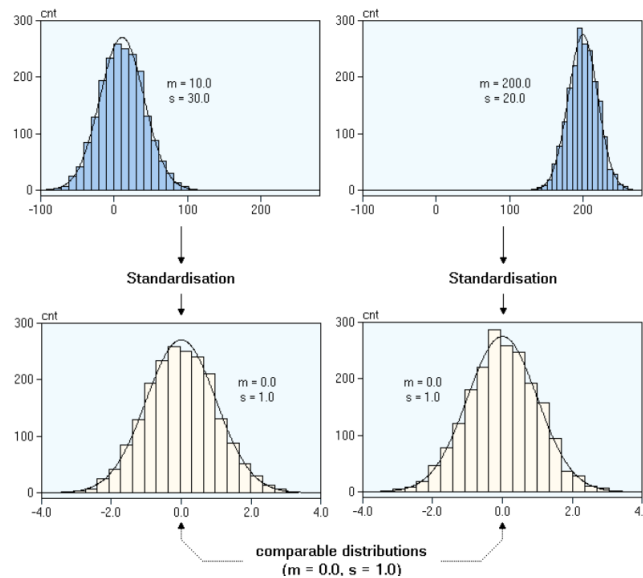
A volte le feature hanno valori su range diversi (valori molto diversi tra di loro), quindi si applicano tecniche di normalization, per ogni feature x di un feature vector, viene applicata la seguente formula.

$$norm(x) = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Dove $x_{min}x_{max}$ sono il valore minimo e massimo assunti dalle feature nel training set. Di fatto stiamo scalando tutti i valori tra 0 e 1.

Poi potrebbero essere diverse in termini di distribuzione, quindi facciamo tecniche di standardization che ci centra tutte le feature intorno allo 0 (valore meno la media fratto la deviazione standard):

$$scal(x_i) = \frac{x_i - m_i}{s_i}$$



Non è detto che in ogni caso vadano fatte queste operazioni, bisogna valutare empiricamente in fase di tuning.

Feature selection

C'è un'altra fase che si può applicare dopo la fase di addestramento, la maggior parte dei modelli di ML mi permettono di capire quali sono le feature che sono più importanti per il mio modello. Quindi vuol dire che ci saranno alcune che saranno poco o per niente impattanti. Ad esempio se voglio capire se sto facendo un salto i valori del magnetometro non sono impattanti. Quello che possiamo fare quindi è toglierle usando solo quelle più importanti perché come minimo ci riduce i calcoli da fare.

3.3 Acquisizione e annotazione dei dati

In molti campi in cui si applica il ML esistono spesso dei dataset, ci sono campi in cui questo non vale, ad esempio per i dataset di sensori inerziali. La prima cosa è pianificare i dati che vogliamo raccogliere.

3.3.1 Quali attività?

Innanzitutto dobbiamo scegliere quale attività, molte volte l'attività è data dal contesto applicativo (l'utente sta guidando) a volte abbiamo più margine di manovra (ad esempio l'app per il fitness, magari cerchiamo di riconoscere alcune attività sono più facilmente distinguibili). Anche quanto è difficile raccogliere questi dati, ci sono dei casi in cui non possiamo raccogliere dei dati (riconoscere quando l'utente cade nel dirupo).

3.3.2 Quali dispositivi?

Dobbiamo determinare quali dispositivi usare, ci sono dei casi in cui abbiamo margini di manovra e casi in cui non ne abbiamo. Ad esempio il produttore di uno smartwatch ci chiede di sviluppare app di fitness quindi dovremo utilizzare lo smartwatch. Per un app di fitness generica potremmo scegliere di usare lo smartwatch, lo smartphone, sensori indossabili, banace-board ...

3.3.3 Quante persone coinvolgere?

Un altro aspetto importante è capire quante persone coinvolgere (dipende se vogliamo fare un modello generalista o specifico per un utente), tipicamente raccogliere i dati da almeno una decina di persone è importante perché solitamente le persone svolgono le attività in maniera diversa.

3.3.4 Dove svolgere le acquisizioni?

C'è un'altra questione, dove vogliamo svolgere l'acquisizione, diciamo due estremi:

- Laboratorio: organizziamo una serie di attività che la persona deve svolgere e le facciamo fare in un ambiente controllato
- Nel vero ambiente dove si svolgerà l'attività

3.3.5 Quali azioni?

L'altra cosa importante è quali azioni si può chiedere all'utente di svolgere le sue normali attività oppure chiedere all'utente di fare delle azioni in un ordine prefissato. Più stiamo in un contesto reale di utilizzo più siamo certi che i dati non siano difformi, ma potrebbe essere difficile prelevarli.

3.3.6 Come annotare i dati

- In tempo reale:
 - Li annota l'utente stesso: in generale il problema è convincere a farlo fare all'utente. Una tecnica è fare annotazione implicita in cui è avere delle situazioni in cui non è necessario che l'utente faccia un'annotazione ma che ci dicono che sta avvenendo un'azione (ad esempio l'utilizzo di android auto quando si guida)
 - Li annota un incaricato
- Ex-post: farlo dopo, vuol dire che solitamente abbiamo una registrazione video di quello che succede. Sono più accurate ma sono time consuming, inoltre non sono adeguate in un sistema in the wild.

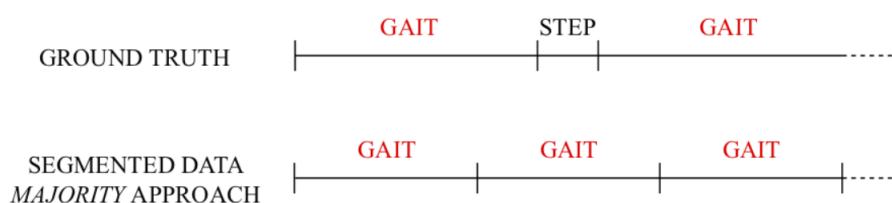
3.3.7 Quantità di dati da raccogliere

I dataset spesso sono sbilanciati, cioè spesso ci sono delle attività che gli utenti svolgono di più e altri di meno. Certe attività sono più lunghe (più finestre in cui si svolge l'attività), di alcune attività potremmo non avere degli esempi.

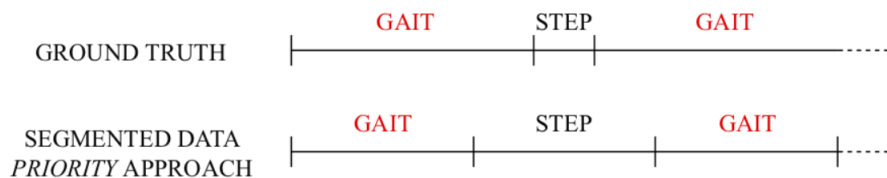
3.3.8 Annotazione delle finestre

Annotiamo la finestra con l'attività che viene svolta per la maggior parte del tempo all'interno di quella finestra. Questo approccio va bene in molti casi, ma presenta problemi quando ho attività di lunghezza diversa. Nell'esempio abbiamo un utente che cammina, poi salve uno scalino e poi continua a camminare. Rischiamo di non annotare quasi mai una finestra come "scalino" e dunque il nostro sistema non imparerà a riconoscere queste attività.

Lui impara che ogni tanto l'andatura ha un comportamento strano come quando c'è lo scalino.



Per risolvere questo problema si sono inventati l'approccio priority, l'annotatore deve annotare le finestre che contengono un evento particolare con quell'evento. Hanno fatto questo creando una gerarchia di attività, in modo da annotare quella che ritengono più interessante.



Nel ML dobbiamo capire qual'è la semantica del nostro dato. Questa è una cosa che spesso viene trascurata. Dobbiamo adattare quello che viene fatto nella pipeline rispetto ai tipi di dati che abbiamo.

3.3.9 Cosa abbiamo ottenuto

Abbiamo definito una tecnica per ripulire i dati, separarli in blocchi ed estrarre il feature vector. Abbiamo creato una ground truth per sapere quale sia la vera attività svolta da un individuo. Abbiamo trovato un modo per annotare i feature vector con la relativa etichetta. Abbiamo finalmente ottenuto le coppie $\langle \text{etichetta}, \text{feature-vector} \rangle$ che ci permette di addestrare il modello.

3.4 Caratteristiche ed uso degli algoritmi di classificazione

Vediamo quali sono le caratteristiche degli algoritmi di classificazione. Non entriamo nel dettaglio ma vediamo come possiamo utilizzarli:

- K-Nearest Neighbor KNN
- Support Vector Machine SVM
- Decision Tree
- Random Forest
- Reti neurali
- Reti neurali deep

Di base ad oggi è molto comune l'utilizzo di reti neurali deep, le altre si utilizzano quando ci sono casi speciali.

3.4.1 Apprendimento e generalizzazione

Una delle caratteristiche di questi algoritmi è la capacità di generalizzare. Questa è una proprietà tipica di tutte le tecniche di ML, ci sono alcuni algoritmi più bravi a farlo ed altri meno. È ovvio che per fare questo processo di generalizzazione bisogna avere delle accortezze, avere un set di dato abbastanza ampio per generalizzare bene.

Un aspetto importante per progettare una tecnica di HAR è avere due modelli:

- Progettare una tecnica che impari su un insieme di persone e poi utilizzata su altre persone che non ha mai visto riesce a riconoscere la stessa attività. Tecnica generalizzata, il pro è che l'addestramento viene svolto una volta sola. Se il modo in cui una certa attività varia molto tra individui è molto difficile per l'algoritmo riconoscere il caso.
- Modelli personalizzati: il modello è allenato usando solo i dati dell'individuo che usa il sistema. Il pro è che ci aspettiamo dati più uniformi e dunque più semplici da classificare. Il contro è che richiede una fase di addestramento specifica per ogni utente, in molti casi è necessario che il modello apprenda con pochi (pochissimi) esempi.

Vediamo un esempio di modello personalizzato. Il contesto è un app per persone con disabilità motorie per controllare un videogioco tramite espressioni facciali, le persone con disabilità motorie potrebbero avere limitata mobilità anche nelle espressioni facciali. Il problema è che ogni utente deve essere in grado di definire le proprie espressioni, in un gioco si usano pochi comandi, dunque il numero delle classi tra cui distinguere è molto limitato. Abbiamo adottato una soluzione personalizzata, l'utente definisce le proprie espressioni per ognuna registra qualche secondo di video. Il modello viene addestrato direttamente sul dispositivo mobile dell'utente e poi è in grado di riconoscere le espressioni.

C'è una possibilità di avere modelli ibridi in cui una prima fase dell'addestramento viene svolta come per i modelli generalizzata e poi viene chiesto all'utente di svolgere delle attività per personalizzare il modello in base alle sue caratteristiche (ovviamente bisogna chiedere all'utente di annotare le sue azioni).

C'è un'altra questione che è rognosa e viene poco enfatizzata ed è l'azione altro. Tu insegni alla rete a riconoscere quattro azioni ma quando l'utente non sta facendo niente cosa riconosce il sistema. Una possibilità è creare una classe per ogni azione specifica, sarebbero tante tutte le classi. Quindi spesso si usa una classe altro, ma la sua definizione è un pasticcio, si può fare in due modi:

- Definire esplicitamente la classe altro, ci sono una set di cose che metto qui dentro
- Se la confidenza della classe che hai trovato è molto bassa allora puoi dire che è altro (funziona meno bene)

3.4.2 Addestramento offline vs online

Un'altra cosa da stabilire è quando faccio l'addestramento, solitamente nei modelli generalizzati l'addestramento è complesso perchè richiede tanti dati e richiede tanto tempo quindi si farà offline su server ad alte prestazioni. Viceversa è possibile che alcuni algoritmi siano più semplici da addestrare allora lo possiamo fare anche sul device mobile dell'utente.

3.4.3 Inferenza offline vs online

Solitamente l'inferenza richiede capacità computazionali molto minori rispetto all'addestramento. Quindi anche se abbiamo modelli molto grossi possiamo portarli su dispositivo mobile e l'inferenza viene fatta in qualche frazione di secondo anche su dispositivo mobile. Non è sempre così in realtà, esistono

algoritmi più leggeri che permettono di essere eseguiti in tempo reale anche su dispositivo mobile.

A volte non è necessario fare l'inferenza in tempo reale, ma possiamo svolgerla in un secondo momento. Se abbiamo un app che non è interattiva possiamo anche farlo la sera una volta che abbiamo raccolto i dati.

3.4.4 Dove avviene la computazione

Solitamente l'addestramento avviene su un dispositivo tradizionale (macchine ad alte prestazioni). Può essere anche sul dispositivo mobile se l'algoritmo lo consente.

La classificazione avviene sul dispositivo che acquisisce i dati, ma potrebbero esserci casi in cui avviene su un server:

- Solitamente vengono almeno pre-processati dal dispositivo mobile
- Attenzione ai problemi di privacy nell'invio dei dati

Oppure casi in cui i dati sono acquisiti da un dispositivo indossabile e mandarli ad un dispositivo mobile che li elabora. Il vantaggio di eseguire le operazioni sul dispositivo dell'utente ci permette di sfruttare la potenza computazionale dell'utente invece di comprare HW apposito per fare la computazione.

3.4.5 Deploy su dispositivo mobile

Esistono delle SDK che permettono di utilizzare ML su dispositivo mobile per fare addestramento ed inferenza:

- CoreML
- MLKit
- TensorFlow lite

È possibile prototipare e validare un modello di riconoscimento in una fase offline ad esempio con librerie di Python. Una volta soddisfatti con il modello, lo si converte in un formato apposito per CoreML. Il modello può essere quindi deployato sull'applicazione ed utilizzato per il riconoscimento.

Attenzione che bisogna mantenere le fasi di pre-processing su dispositivo mobile come le abbiamo fatte in python.

3.4.6 Creazione del modello

Esistono molti modelli pre-addestrati. Questa è una cosa che in altri ambiti spesso succede, ad esempio in un problema di computer vision (riconoscere un cartello in un'immagine). Per problemi di HAR è improbabile che riusciamo a trovarlo.

Quello che potrebbe succedere è usare tecniche di transfer learning, per cui partiamo da un modello che è in grado di fare alcune operazioni. Ad esempio vogliamo fare un modello che riconosce i maniglioni dalle porte, ma troviamo una rete che riconosce i gatti, quindi possiamo usare quella rete riaddestrando solo gli ultimi livelli della rete.

3.4.7 Creazione e specializzazione di modelli

L'addestramento di modelli non necessariamente ci richiede di scrivere codice. Esistono dei tool chiamati AutoML, ai quali diamo in pasto i dati e le label e loro automatizzano tutto il processo. La procedura di addestramento della rete è in qualche modo sempre uguale, a un livello iniziale tutto si può fare con dei tool automatizzati. Ad esempio:

- CreateML: apple
- Google Cloud AutoML

3.5 Tuning e valutazione dei modelli

Dobbiamo fare una lunga digressione sulla parte di valutazione dei modelli. La prima cosa che avevamo detto era che avevamo bisogno di dividere i dati in due insiemi in modo da testare le performance del modello su dei dati che il modello non ha mai visto:

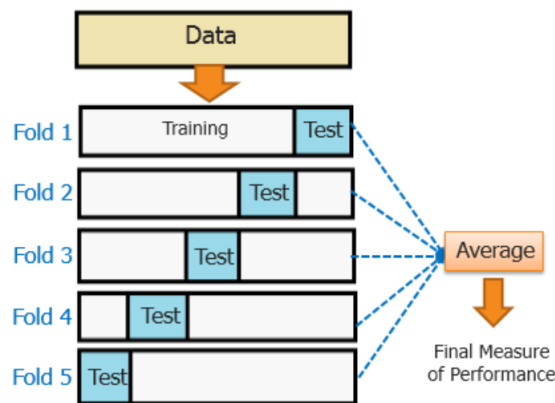
- Training set: si usa per addestrare il classificatore
- Test set: si usa per valutare il classificatore

Accenniamo il fatto che alcuni modelli hanno bisogno di un set di validation. Il problema è che vorremmo avere il set di test molto grande e il set di training molto grande per evitare casi particolari. Inoltre non dovremmo mai usare questi dati per valutare il classificatore. Come divido il training set e il test set? L'altra questione è quali metriche adottare per misurare quanto va bene il mio sistema.

3.5.1 Suddivisione fissa

Un modo semplice per dividere i dati è quello di fare una suddivisione fissa. In moltissimi casi viene usata una divisione 80/20, 80 per il training e 20 per il validation. C'è però un problema, se abbiamo tanti dati questo 20% di dati di test set sarà abbastanza grande quindi più o meno potremmo ri-suddividerlo in una maniera diversa. La verità è che in molti casi se abbiamo pochi dati la scelta di questi pochi dati può essere molto sfortunata o fortunata e portarci a risultati molto diversi.

3.5.2 Cross-validation



L'idea è quella di ripetere il processo di addestramento e di test più volte. Ogni volta scelgo un set di dati di test diverso. Si creano k "fold", in cui uno dei blocchi serve per il test, gli altri per il training. Per ogni fold si ripete l'apprendimento e la valutazione con i dati di test. Poi si aggregano i risultati. Di solito si usa una 5-fold e la 10-fold.

Il vantaggio è che di fatto tutti i dati che ho verranno utilizzati come dati di test, vado quindi a testare su tutti i miei dati. Lo svantaggio è che se il training richiede tanto devo ripeterlo tante volte.

Noi vorremmo usare tante più fold possibili, perchè più fold usiamo più grandi saranno i dati con cui impara, però i tempi aumentano.

C'è un motivo tecnico perchè la 10-fold ha un vantaggio, è più utile se si vuole fare un'analisi statistica dei dati. Supponiamo di avere due tecniche e volerle confrontare. Con la 5-fold spesso non abbiamo una differenza significativa, mentre con la 10 la si ha.

In realtà avendo capacità computazionale illimitata vorremmo fare tante fold quanti dati abbiamo, si chiama leave-one-out cross validation.

Succede molto spesso che abbiamo risultati diversi dalle varie fold, questo ci fa capire che se usassimo una divisione fissa rischieremmo di prendere un granchio.

Per ciascuno dei test tiro fuori delle metriche e ne faccio poi la media.

Leave-one-out validation

Vogliamo avere un modello che generalizzi bene su utenti mai visti. Quindi raccogliamo i dati da 10 utenti e poi abbiamo tutto il nostro set di dati etichettati. Scelgo casualmente il 20% di dati e li uso come test set della prima fold e l'80% di dati come train. Il problema di questa cosa è che potrei avere la stessa persona nel training set e nel test set e potremmo avere un bias. Quindi una pratica molto comune nelle HAR è la leave-one-subject-out cross validation, prende i dati di tutti gli utenti tranne uno e poi testa sui dati di quello tenuto fuori.

Ricerca iper parametri

In generale potremmo usare due approcci:

- Eseguire la cross validation per ogni fold fare il tuning dei parametri
- Nel processo di tuning dei parametri, ad ogni iterazione, eseguiamo la cross validation

È il secondo approccio quello più corretto, perchè vorremmo trovare gli iper parametri che massimizzano il modello, non la singola fold. Come se avessimo un ciclo esterno che fa fare l'addestramento k volte e dentro un altro ciclo che fa il tuning dei parametri.

3.5.3 Metriche di valutazione per i classificatori binari

Iniziamo a pensare al problema in termini di un classificatore binario, in questo caso supponiamo che debba riconoscere quando stai correndo, possiamo definire una classe come positiva e l'altra come negativa, cora è la classe positiva e altro è la negativa. Possiamo distinguere quattro casi:

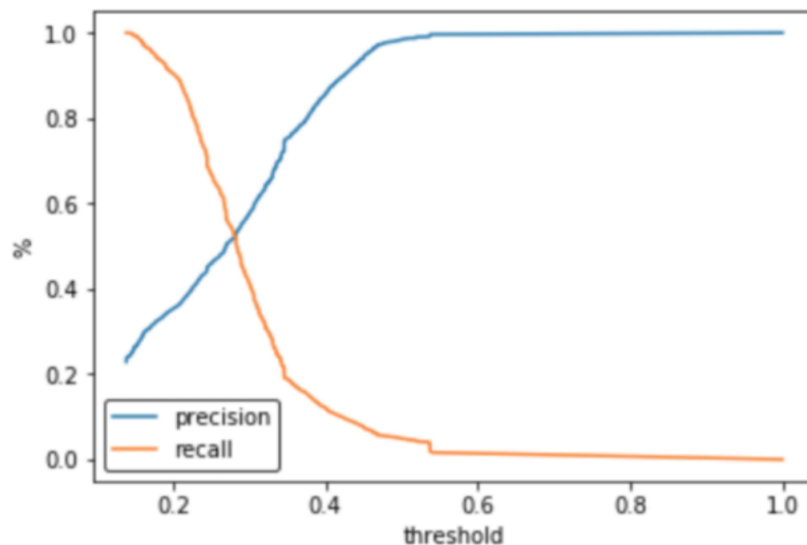
- True positive: se il modello mi dice positivo quando è effettivamente positivo (etichetta ground truth positiva)
- True negative: se il modello dice altre ed effettivamente stava facendo altro
- Falsi positivi: predico positivo ma in realtà era altro

- Falso negativo: stava facendo altro ma in realtà era positivo

Una volta che abbiamo definito cosa sono queste combinazioni, possiamo definire una serie di metriche che sono:

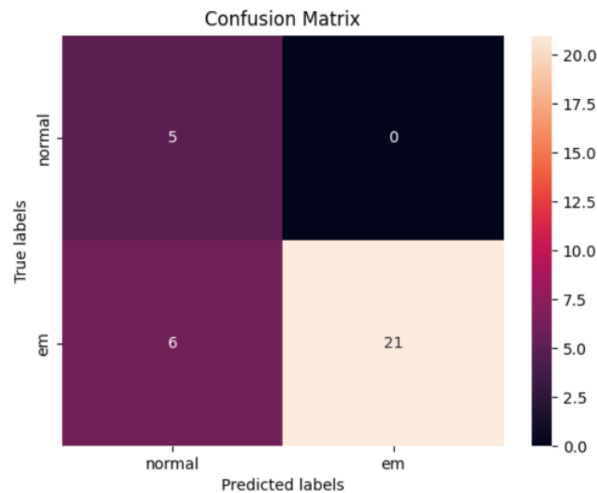
- Precision: True positive diviso tutti quelli positivi. Quando dici positivo quant'è la probabilità che stai indovinando veramente. $TP/(TP + FP)$
- Recall: True positive diviso tutti i positivi. Quanti dei positivi devo predire come tali $TP/(TP + FN)$

Abbiamo detto che solitamente non dice qual'è la classe predetta, ma da una confidenza sulle varie classi. Potremmo stabilire una soglia per definire se fa parte di una classe o meno. Potrei decidere che ci sono altri trade-off tra precision e recall. Al variare di questo valore di soglia tra positivo e negativo mi variano la precisione e recall. Se dico sempre che sono positivi la recall sarà sempre a 1 mentre la precision sarà bassissima. Mentre viceversa se sono super restrittivo e dico che è positivo se abbiamo il 99% di sicurezza avrò la situazione inversa. Ottengo quindi un grafico di questo tipo:



3.5.4 Matrice di confusione

Ho una matrice in cui ho tutte le etichette, vorrei che fossero molto alti i valori sulla diagonale perchè sono i true positive e i true negative.



3.5.5 Altre metriche

Non è detto che ci siano solo precision e recall come metriche. Vanno bene ma vanno usate assieme, quindi ci sono metriche le mettono insieme:

- Accuracy: tutti i positivi / tutti i dati
- F1 score: $2 * (precision * recall) / (precision + recall)$

Quindi se definiamo cos'è true e false positive e negative deriviamo tutte le metriche che vogliamo. La difficoltà di fare questa definizione è quando siamo in un caso multiclasse.

3.5.6 Metriche di valutazione per classificatori multi-classe

Qui bisogna distinguere classe per classe. Se abbiamo un dato di classe A predetto come dato di classe A è un true positive per la classe A e un true negative per le altre classi. Se ho un dato di classe A e lo predico come B, allora per B è un false positive per A è un false negative ed è un true negative per tutte le altre. (notiamo che nelle formule non compaiono mai in true negative)

Usando queste metriche possiamo definire precisione, recall, accuracy per ogni classe. Poi possiamo fare una media per tirare fuori un valore per tutto, per esempio molto spesso (in HAR) non succede quasi mai che le classi sono bilanciate (100 esempi di camminare e 100 esempi di salto mortale) quando calcoliamo le metriche abbiamo la balance accuracy che tiene conto del fatto che le classi sono sbilanciate e tiene più di conto le classi meno rappresentate.

La matrice di confusione qui diventa importantissimo perchè mi fa capire non solo quanto ci becco sulla diagonale, ma anche con cosa mi confondo più spesso, nell'esempio si vede a colpo d'occhio che standing e sitting si confondono spesso.

laying	423	0	0	0	0	0
sitting	0	365	17	0	0	0
standing	0	21	387	0	0	0
walk	0	0	0	351	17	11
walkdown	0	0	0	14	246	18
walkup	0	0	0	14	12	310
	laying	sitting	standing	walk	walkdown	walkup

Random Forest

3.5.7 Validazione dei modelli e semantica dei dati

Cos'è il dato in questo momento? solitamente il dato su cui focalizziamo la nostra attenzione è la dimensione della finestra. Ma spesso non è la metrica adatta, perchè non è detto che gli errori siano tutti uguali e che io voglia misurare le finestre.

Facciamo un esempio, supponiamo di avere un sistema che permette di giocare con un videogioco usando le espressioni facciali. Ci interessa la dimensione della finestra? sì perchè non vorremmo che beccasse tutte le finestre. Ma in realtà non è questo quello che vogliamo misurare. In un contesto di HAR non è sempre detto che quello su cui vogliamo misurare le prestazioni del sistema siano le dimensioni del sistema ma vorremmo misurare le prestazioni del sistema in base a come funziona bene il sistema in base all'interazione dell'utente.

3.5.8 Processo di tuning

Abbiamo visto che le varie fasi di acquisizione dei dati, pre-processing e addestramento sono soggetti a moltissimi parametri (il filtro da usare, anche l'algoritmo è un parametro, le finestre). Per trovare il valore ottimale di questi parametri servirebbe provare tutte le combinazioni, questo però è impossibile in pratica, perchè le combinazioni di parametri sono troppe.

Approccio errato

Un approccio che potrebbe sembrare appropriato a prima vista è il seguente. Fissare il valore di un parametro e fare variare gli altri per trovare il valore migliore, a questo punto faccio variare il parametro fissato per trovare il valore migliore del primo. Il problema è che facendo così non sono più sicuro che i parametri fissati in precedenza siano ancora ottimali.

Un possibile approccio a partire da questo è fissare un parametro e provare tutte le combinazioni degli altri. Il limite di questa cosa è che sto comunque provando tutte le combinazioni. Questo si chiama grid search, si può fare ma bisogna essere molto bravi a ridurre le dimensioni e i parametri su cui faccio la ricerca devono essere parametri con un numero limitato di possibilità.

Se dobbiamo fare la cross validation significa anche che per ogni combinazione di parametri devo rifarla.

Bayesian Optimization

Il sistema guarda come sono andate tutte le esecuzioni precedenti e sulla base di come sono andate calcola il gradiente e cerca di capire dove andare per fare l'esecuzione precedente. Dopo un po' (si può fissare il tempo) lui cerca di avvicinarsi al parametro migliore. Cerca di focalizzare la ricerca laddove sia probabile che ci siano i risultati migliori.

3.6 Il problema dei dati

Ogni volta che si fanno lavori di ML il problema sono sempre i dati. Il problema è particolarmente complicato nello HAR, perchè reperire i dati è difficile in quanto ci sono problemi di privacy ed un sacco di contesti diversi, ci può essere un sacco di differenza in base a chi ha fornito i dati (ci sono casi in cui è proprio impossibile recuperare i dati).

3.6.1 Reperimento

Il primo problema è il reperimento. Vediamo come farlo tramite data augmentation e transfer learning.

Data augmentation

Creiamo una serie di dati sintetici partendo da dati reali, questi dati sintetici devono aiutarci a fare training e valutazione degli algoritmi. Possiamo crearli in mille maniere, che vanno da operazioni semplici di riconoscimento delle

immagini , si possono fare delle trasformazioni delle immagini per poterne creare degli altri sintetici. Attenzione che non in tutti gli ambiti permettono di ribaltare le immagini. Si può fare anche per l'HAR, ma stando attenti a cosa si fa, perchè ad esempio non andrebbe bene fare camminare una persona al contrario invertendo i dati che abbiamo. Si possono ruotare gli assi o variare la componente temporale.

Tecniche collaborative

Un altro approccio è avere tecniche collaborative. IN cui non è un solo ente che si occupa di raccogliere i dati ma sono tanti, molte volte si fa crowdsource tra gli utenti. I problemi principali sono due:

- Scalabilità: capacità di tramettere i dati e creare il modello
- Privacy: le aziende sono molto interessate ai dati, il valore sta nei dati. Dobbiamo quindi creare un modo per fare un modello utile ma senza violare la privacy degli utenti

La tecnica del federated learning è una tecnica per cui ogni client addestra la sua rete e la manda al server, poi il server può unire i modelli dei client. Il vantaggio è che dal modello addestrato al server è molto meno problematico in termini di privacy. È vero che esistono degli attacchi che cercano di ricostruire i dati di partenza a partire dal modello. Si risolve anche in parte il problema della scalabilità perchè il client manda dei modelli che occupano meno spazio rispetto ai dati singoli.

3.6.2 Annotazione dei dati

Come abbiamo già detto annotare i dati è molto costoso. È un processo anche molto soggetto ad errore. Esistono delle soluzioni:

- Semi supervised
- Unsupervised

Un modo semi supervisionato è usare una di questi approcci:

- Self-training
- Co-training
- Active learning

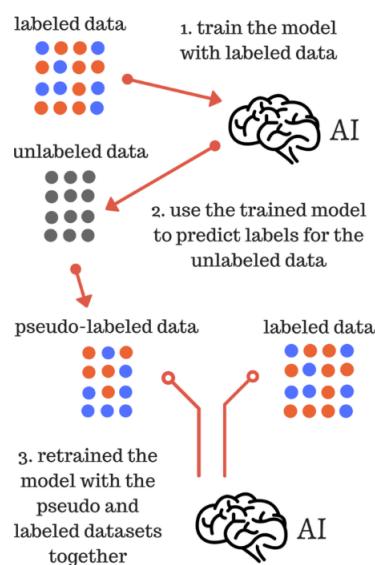
C'è anche una tecnica che non ha bisogno di annotazioni:

- Anomaly detection

Self learning

Uso un piccolo dataset per creare un modello iniziale. Poi uso il modello per predire le etichette di dati non etichettati. Il problema è che se faccio un modello piccolo potrebbe sbagliare. Una cosa che si può fare è usare per addestrare il piccolo modello i dati con confidenza più alta.

L'uso che se ne fa spesso è etichettare prima le azioni più comuni e poi fare etichettare al modello quelle.



Il limite è quello che eventuali errori possono propagarsi e rafforzarsi.

Co-training

È un processo nel quale ho due classificatori che usano due insiemi diversi di feature points. Avendo dei feature points li divido in due, un gruppo per ogni classificatore, poi addestro i classificatori su un gruppo comune di finestre. Poi potrebbe succedere che uno dei classificatori abbia alta confidenza, poi uso quell'etichetta con alta confidenza per etichettare altri dati e usare quei dati per addestrare il classificatore meno preciso.

Comunque l'errore si propaga, quindi sarebbe utile comunque avere un classificatore umano che risolve i casi in cui l'etichetta ha bassa confidenza oppure casi in cui si ha alta confidenza ma per lo stesso dato di due classi diverse.

Active learning

Creiamo un modello con pochi dati e poi il modello funziona e quando utilizziamo il modello in modo interattivo con l'utente e quando siamo in dubbio su qualcosa lo chiediamo all'utente. Quando abbiamo un basso livello di confidenza chiediamo all'utente come vuole classificare quella attività. Dobbiamo trovare il modo di motivare l'utente a fare questa cosa.

Un limite è che di solito chiediamo all'utente quando siamo incerto tra tante attività e lui deve scorrere tra tante attività quindi ci mette di più a trovare l'attività giusta.

Algoritmi incrementali

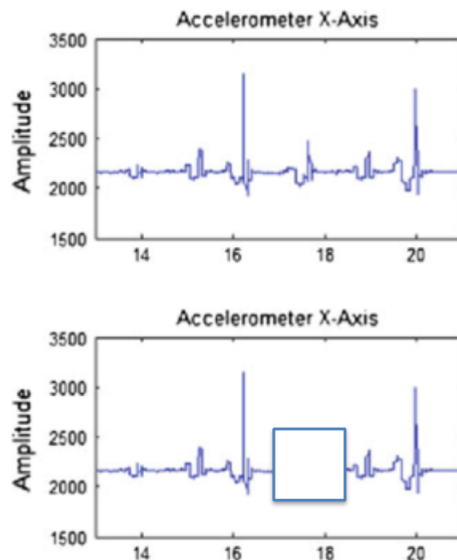
Con tutte queste tecniche stiamo assumendo che usiamo algoritmi di apprendimento incrementali, in cui possiamo ogni tanto aggiungere dei dati di train per fare funzionare meglio il modello. Alcuni algoritmi per loro struttura sono incrementali (come il deep learning), altri modelli non sono di base incrementali ma esistono delle varianti che lo sono.

Anomaly detection

L'idea è che vogliamo identificare dei dati che differiscono sostanzialmente dal normale. Quindi l'algoritmo deve imparare cosa vuol dire un dato normale per poi riconoscere quando arriva un dato anomalo. La cosa interessante è che se noi abbiamo un dataset di dati normali non abbiamo bisogno delle annotazioni, non è vero che è del tutto unsupervised, in realtà un concetto di annotazione ce l'hai, che è il concetto di dato normale. Per verificare come funziona l'algoritmo abbiamo bisogno di un po' di dati annotati come anomali. Però per tutto il train viene fatto con dati normali.

Vediamo una tecnica chiamata "reconstruction by inpainting" che nasce per le immagini ma vediamo lo stesso concetto sui sensori per l'activity recognition. Abbiamo un algoritmo di ML tipicamente deep che viene addestrato a ricostruire il dato normale. Perché è molto furbo l'idea di imparare a ricostruire un dato? perché non mi serve l'annotazione, se devo dire se stai camminando o correndo non c'è scritto nel dato ma devo annotarlo, ma se ho un dato e devo ricostruirlo basta fare finta che parte di quel dato non ci sia. Ho dei dati, faccio finta di non avere i dati in un punto e creo un algoritmo che prova a ricostruire il dato, se sbaglia non è un problema perché conosco il dato che ho nascosto. L'etichetta è il dato stesso. Posso ripetere il train fino a che la tecnica non impara a ricostruire bene il dato.

Una volta che lo abbiamo ricostruito siamo in fase di inferenza, dobbiamo capire se è normale o anomalo. Prendiamo il dato vero e ne cancelliamo un



pezzo per volta. Lui lo ricostruirà come un dato normale (perchè ha imparato su dati normali), se la ricostruzione è simile a quello che abbiamo saremo in un caso di dato normale, altrimenti saremo in un caso di dato anomalo. Quindi abbiamo bisogno anche di una funzione di differenza che mi dice qual'è la differenza tra il dato reale e quello ricostruito.

3.7 Activity recognition con tecniche di visione

Ci sono un po' di pro e di contro tra usare sensori e visione, un limite delle tecniche inerziali è che l'utente deve tenersi il sensore addosso. Ci potrebbero essere dei contesti in cui magari non è sensato assumere che abbia dei sensori addosso ma che magari possiamo assumere che sia inquadrato da una camera, magari perchè è l'utente stesso che si inquadra e in altri che è ripreso da sistemi di sorveglianza. In tutti questi casi possiamo utilizzare il dato della camera per provare a riconoscere i movimenti dell'utente.

Riconoscere il corpo umano (es: ricostruire la posizione) in immagini è un problema sostanzialmente irrisolvibile con tecniche analitiche. C'è un sacco di variabilità:

- Forma: persona magra o robusta, alta o bassa
- Colori: vestiti, carnagione, capelli ...
- Ambiente: luminosità, sfondo ...

L'HAR si incrocia molto con l'interazione naturale dell'utente con la macchina.

3.7.1 Tipi di attività

Possiamo fare delle analisi più complicate quando abbiamo delle immagini rispetto a quello che possiamo fare quando abbiamo i sensori inerziali:

- Pose detection: trovare una posa statica dell'utente
- Azioni
- Gest
- Comportamenti
- Interazioni
- Azioni di gruppo
- Eventi

3.7.2 Quali parti del corpo

Per poter fare activity recognition basata su visione abbiamo bisogno di alcuni strumenti che riconoscono il corpo dell'utente, ci sono tecniche specializzate in queste che possono riconosce il corpo intero o essere specializzati su parti del corpo:

- Viso: riconoscimento del viso serve molto per le espressioni facciali, ad esempio riconoscimento dello stato d'animo della persona tramite l'espressione
- Mani: applicazioni di gesture con le mani
- Torso
- Gambe
- Braccia

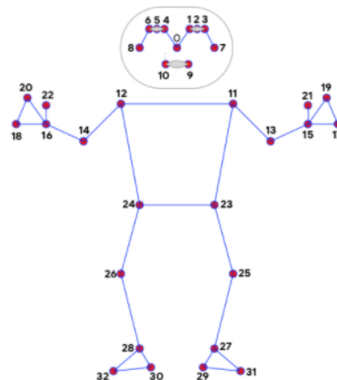
3.7.3 Classificazione vs Detection

Usando tecniche di recognition su visione ci si può porre il problema non solo di classificare l'azione ma anche dove è stata fatta nell'immagine o anche quante persone stanno facendo quell'azione. Una delle applicazioni di queste tecniche è quella della sicurezza per riconoscere un insieme di atteggiamenti pericolosi per richiamare l'attenzione dell'operatore.

3.7.4 Rappresentazione spaziale

La prima cosa che ci serve fare è capire dov'è l'utente. Avere una rappresentazione spaziale di dov'è l'utente nell'immagine. Fino a qualche anno fa il problema era molto complicato, esistevano tecniche che ti davano una silhouette di dove stava l'utente. Ovviamente non si sa che parti del corpo sono.

Un approccio più moderno e comune è quello di creare un kinematic joint model, ovvero creare un intero modello di quello che voglio riconoscere. Quindi identifico dei punti (che sono associati a parti del corpo in letteratura). Questo modello non solo mi dà un'idea più precisa di dov'è posizionato l'utente ma riconosco anche le parti del corpo dell'utente.



Tecniche basate su marcatori

In passato le uniche tecniche possibili erano quelle basate su marcatori fiduciari, l'utente indossa dei marcatori visivi. Il sistema è in grado di conoscere la posizione dell'utente nel tempo grazie ai marcatori e quindi è in grado di riconoscere l'azione. In alcuni casi può ancora essere una buona idea per migliorare le performance riducendo il costo computazionale, ad esempio se abbiamo un atleta e vogliamo dirgli se lo sta facendo bene può essere utile e non troppo scomodo fargli mettere dei marcatori.

Tecniche marker-less

Esistono tecniche basate su deep learning che in tempo reale riescono a ricostruire il kinematic joint model la figura intera dell'utente/mano/volto. La più diffusa si chiama media pipe che è fatta da Google.

3.7.5 Uso delle tecniche

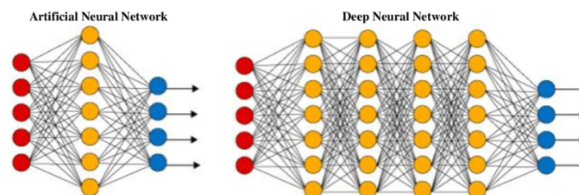
Si possono utilizzare in due modi questi modelli:

- Costruisco una rete in cui un livello è il riconoscimento del kinematic joint model e i livelli successivi mi riconoscono il movimento
- Usare queste reti come se fossero dei feature extractor. Prende i punti riconosciuti da questi modelli e usa questi punti come feature points con un altro modello

Quindi di fatto passiamo invece che avere come dato grezzo l'immagine all'avere come dato i key points.

Un'altra semplificazione usata è quella di non riconoscere esattamente un'azione intesa come un insieme di frame, ma in alcuni casi è possibile riconoscere l'azione attraverso una singola posa dell'utente. Supponiamo che l'utente abbia due azioni per comandare il gioco, per riconoscere un'azione potremmo riconoscere le pose e sulla base di queste pose possiamo capire con delle regole che azioni sono state svolte. Quali sono in questo caso i feature point che usiamo? Sono derivati dai key point che abbiamo preso in considerazione, ad esempio potremmo avere la distanza tra il polso e l'anca, se vogliamo riconoscere l'espressione del volto potremmo calcolare la distanza tra le labbra.

3.8 Deep learning per activity recognition



La differenza principale tra le reti neurali non deep e quelle deep è che nelle deep abbiamo un numero più elevato di layer chiamati hidden. Ognuno di questi neuroni ha associato una serie di parametri che dobbiamo calcolare

in fase di apprendimento, quando il numero dei neuroni aumenta il numero di parametri che dobbiamo imparare è molto maggiore quindi il numero di esempi che mi servono è molto maggiore. Le reti neurali deep sono riuscite a risolvere problemi molto maggiori rispetto a quelle non deep a fronte di avere molti più dati e costi computazionali molto più grandi.

Ci aiutano in tre modi:

- Possiamo affrontare problemi particolarmente complessi
- Non necessitano della fase di feature extraction durante il pre-processing
- Non necessitano della fase di segmentazione. La divisione in finestre non ci serve più.

3.8.1 Il problema dei dati

Con le tecniche deep abbiamo bisogno di molto più dati per fare l'addestramento.

Potremmo trovarci in due situazioni:

- Abbiamo tanti dati e poche annotazioni: potremmo usare alcuni trucchi per provare ad utilizzare comunque i dati che abbiamo
- Pochi dati e poche annotazioni: non possiamo usare reti deep

3.8.2 Estrazione delle feature

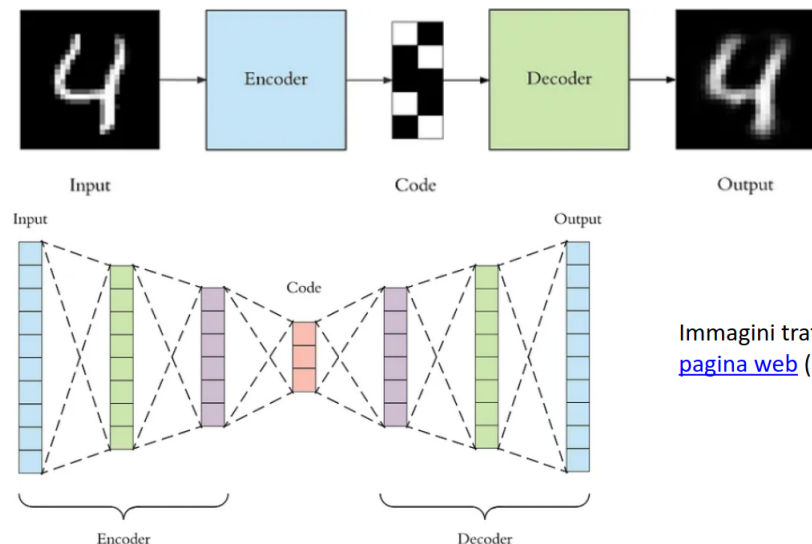
Le feature sono una rappresentazione del dato con meno informazione. Quindi supponiamo di essere nel caso in cui abbiamo tanti dati ma poche annotazioni e vogliamo definire un sistema per estrarre automaticamente le feature. Perché non configuriamo il problema con un problema self-supervised?

Si usano gli autoencoder, l'idea è quella di avere una rete deep con una strozzatura in mezzo. La rete prende in input il dato e ha lo scopo di generare lo stesso dato (non ho bisogno di etichette perché l'etichetta è il dato stesso). La cosa interessante è che potremmo pensare la rete divisa in due:

- Encoder: dalla immagine mi dà una rappresentazione della stessa con meno dati
- Decoder: presa la rappresentazione compressa ricostruisce l'immagine originale

Se riesco a costruire questa rete bene abbiamo la rappresentazione compressa dell'immagine come una rappresentazione dell'immagine. A questo punto possiamo buttare via il decoder e tenere l'encoder, che diventa un feature

extractor perchè ci tira fuori le feature dell'immagine. Così abbiamo trovato un modo più solido per estrarre le feature.



3.8.3 Uso delle reti deep per serie di dati

Un' altro aspetto interessante delle reti deep è il loro uso per le serie di dati. Accenniamo solo a due soluzioni:

- Tecniche con segmentazioni
- Tecniche senza segmentazione

Reti neurali convoluzionali

Sono un tipo di rete solitamente utilizzata per le immagini che ha dimostrato di essere efficace nel risolvere problemi complessi nell'ambito dell'elaborazione delle immagini. Se ci pensiamo un'immagine è un array bidimensionale e anche i dati della nostra finestra possono essere interpretati come array bidimensionale, per ogni istante della finestra abbiamo i dati del sensore. Quindi si possono utilizzare le stesse tecniche per l'elaborazione delle immagini per risolvere il problema dell'activity recognition. Ci sono dei lavori che fanno vedere che queste tecniche sono molto efficaci.

Long short-term memory

La cosa ancora più interessanti sono le reti che si chiamano LSTM. L'idea è che a queste reti dai in pasto il dato man mano che ti arriva, queste reti al

loro interno imparano a tenere memoria di alcune informazioni e imparano che alcune informazioni possono dimenticarle. Imparano da sole che alcune cose possono dimenticarle, quindi di fatto risolvono il problema di creare le finestre, mi fanno contemporaneamente la segmentazione dinamica unita alla

3.8.4 Limiti

É vero che le reti deep risolvono problemi molto complicati, ma è vero solo per alcuni ambiti. Ad esempio per il trattamento delle immagini sono di gran lunga lo strumento migliore. Però per lo human activity recognition c'è un problema, la raccolta di dati (possibilmente annotati) è un problema clamoroso. Inoltre c'è un altro problema, nell'ambito delle immagini possono esserci domini diversi ma non cambia troppo, invece per HAR ci vuole poco a cambiare il dominio, prendiamo i dati dei sensori inerziali e i dati della camera e cambia tutto.

Transfer learning

Quando abbiamo pochi dati e poche etichette si usano tecniche di transfer learning che possono aiutare a risolvere il problema. Utilizziamo una rete già addestrata su un altro ambito specializzandola sull'ambito di nostro interesse. Nell'HAR in linea di principio potrebbe funzionare, ma in termini pratici non si sa, mentre nell'ambito di computer vision va benissimo.

Costo computazionale

Un altro problema delle reti deep è il costo di addestramento di queste reti. Come immaginiamo è impossibile fare fare questo addestramento su dispositivi mobili. A volte anche l'inferenza è impossibile da fare su dispositivo mobile.

3.9 Casi di studio

La parte di HAR necessita di una fase di casi di studio per vedere come hanno messo in pratica quello che ha spiegato.

3.9.1 SmartWheels

Lo scopo di questo lavoro era quello di rendere possibile alcuni servizi di navigazione con persone con disabilità motorie che tenessero conto nel percorso di ostacoli ambientali, non solo ostacoli ma anche urban features (una rampa

non è un ostacolo). Una mappa di questi urban features non esiste, anche se ci sono dei progetti che cercano di recuperare una mappa di queste features. L'idea quindi era di provare a vedere se riusciamo a fare un crowdsourcing da degli utenti con disabilità che provano a raccogliere i dati e memorizzare che in una posizione c'è una urban feature. Per calcolare dove sono questi elementi urbani andiamo a riconoscere il movimento delle persone, se sta salendo uno scalino sappiamo che c'è uno scalino. Questo si potrebbe ottenere anche tramite computer vision, in cui di fatto usano le immagini di Google street view alcuni ostacoli ambientali. Loro hanno provato a montare sensori inerziali sopra le carrozzine. Quindi quello che fanno vedere nell'articolo è se riescono a riconoscere l'attività dell'utente, poi sanno che ci vuole ancora un po' di reasoning per riconoscere veramente le urban features.

Hanno individuato i tipi di utenti e che tipo di spostamento potevano fare con la sedia a rotelle:

- Si spingevano da soli
- Spinti da un'altra persona
- Propulsione elettrica

Da questo hanno identificato una serie di urban feature che gli interessava riconoscere:

Poi hanno applicato la pipeline di riconoscimento:

- Preso i dati dai sensori
- Ripulite e aggregati
- Segmentati
- Classificazione delle feature

Come hanno acquisito il dataset? perchè ovviamente non esiste un dataset prefatto di dati da sensori inerziali presi da persone in sedia a rotelle. Alcuni dati presi in strada e altri in area di allenamento.

Avevano 3 board con accelerometro, giroscopio e magnetometro. Poi c'era uno smarthwatch al polso e uno smartphone alla gamba. Avevano un'applicazione che salvava i dati e poi li processavano grazie ad un video che l'operatore registrava mentre l'utente si spostava.

Da questo hanno ottenuto questo dataset molto sbilanciato, perchè abbiamo tanti secondi di movimento dritto:

C'è stata anche una fase di allineamento temporale:

Quello che abbiamo detto per l'etichettatura priority nasce proprio da questo studio.

3.9.2 Estrazione delle feature

Basandosi su quello che c'è in letteratura hanno preso 46 feature per ogni asse del sensore inerziale $((46*9)*5)$. Poi hanno investigato due classificatori principali:

- Random Forest
- Hirerachical Random Forest
- (Ad oggi non si potrebbe evitare di provare tecniche di deep learning)

3.9.3 Tuning dei parametri

Il testo lo hanno fatto facendo la leave one subject out cross validation. Come il sistema addestrato su 16 utenti fosse in grado di predire sul 17-esimo.

3.9.4 Risultati

Vediamo che nel grafico majority/priority quando la finestra è molto breve anche l'approccio majority funziona, ma aumentando la dimensione della finestra il divario aumenta:

3.10 Play4Physio

È un lavoro ancora in attività e riguarda il riconoscimento di movimenti fatti con il corpo. L'esigenza alla base è che i bambini emofilici devono fare esercizi di movimento, che sono spesso esercizi di fisioterapia molto noiosi. Si è provato a fare tecniche di gamification, hanno sviluppato in un progetto precedente dei videogiochi che si controllassero giocando. Il problema di questa cosa è che giochi del genere vanno sviluppati. Loro hanno sviluppato un sistema che permette di controllare dei giochi esistenti con i movimenti del corpo.

Una questione molto importante di questo sistema è che questi bambini potrebbero non riuscire a muovere bene le articolazioni, quindi era importante capire quale fosse il movimento corretto. Serviva quindi un sistema personalizzato. Quindi durante la visita il fisioterapista fa registrare i movimenti al bambino, il sistema impara a riconoscere i movimenti specifici e poi permette di giocare al gioco.

3.10.1 Riconoscimento degli esercizi

Come facciamo a riconoscere i movimenti del corpo? usiamo la libreria media pipe ed estraiamo i keypoint.

Usiamo tecniche di pre-processing per ripulire i dati ed estrarre delle feature (distanza dal baricentro). Poi usiamo delle tecniche di ML per riconoscere da ogni frame una posa statica, hanno dovuto scegliere tecniche di ML che fossero in grado di apprendere sul device dell'utente (KNN).

Una volta riconosciute le pose abbiamo un algoritmo a regole che ci dice cosa sta facendo l'utente.

Vantaggi:

- Usa una libreria esistente per riconoscere i punti del corpo

Potremmo risolvere il problema usando le finestre e cercare di capire cosa sta facendo l'utente in quella finestra, la soluzione su pose risolve il problema di dimensione delle finestre.

Chapter 4

iOS

É importante tenere in mente quali sono gli obiettivi di chi crea un nuovo linguaggio di programmazione:

- Evitare/ridurre gli errori più comuni del programmatore
- Aumentare la compattezza del codice
- Ridurre i tempi di apprendimento
- Migliorare la leggibilità del codice
- Migliorare le prestazioni

Ma tutte queste cose insieme non si possono mai avere.

Swift è un linguaggio di programmazione molto moderno, la prima versione è del 2014.

É un linguaggio multi piattaforma (Apple), compilato con due paradigmi di programmazione:

- Object Oriented
- Funzionale

Si possono creare applicazioni utilizzando combinazioni di Swift, Objective-C, C e C++.

4.1 Programmazione procedurale in Swift

Variabili con `var` e costanti con il `let`:

```
1 let a: Int = 5
2 var c: Int = 3
```

4.1.1 Assegnazione del tipo ad una variabile

É un linguaggio di programmazione fortemente tipizzato. Swift ha un sistema di riconoscimento automatico dei tipi:

```
1 var a = 5 // this is Int
2 var b: Int = 3
3 var c=a+b //this is Int
4 var d = "hello!" // this is a String
5 c=d //compile-time ERROR
```

4.1.2 Tipi

I tipi che possiamo utilizzare sono diversi:

- Interi
- Double
- String
- Bool

Esistono delle funzioni esplicite di conversione tra i tipi.

Swift non implementa nessuna conversione implicita dei tipi, tutti vanno convertiti in modo esplicito:

```
1  var a: Int = 10
2  var c: Double = a //Cannot
3  var b: Int = 3
4  var c: Double = a/b //Cannot
```

A differenza di come succede in Java, un codice del genere non genera errori ma potrebbe creare dei problemi:

```
1  public class MyClass {
2      public static void main(String args[]) {
3          int a = 10;
4          int b = 3;
5          double c=(a/b)*b;
6          System.out.println("c: " + c);
7      }
8  }
```

4.1.3 Tuple

Una variabile può essere associata a una tupla di valori ad esempio let error = (404, "Not Found").

Questo ci torna comodo quando abbiamo una serie di valori da usare in maniera estemporanea.

```
1  let httpError = (404, "Not Found")
2  print(httpError) //prints "(404, \"Not Found\")"
3  let (statusCode, statusMessage) = httpError
4  print(statusCode) //prints 404
5  print(statusMessage) //prints "Not Found"
6  let (statusCodeAgain, _) = httpError; //just read the
```

```

7 error number, not the type
8 //note: in the line above I have used the "_" symbol
9 meaning "I don't care about the second value"
10 print(statusCodeAgain) //prints 404
11 let statusCodeAgainAgain = httpError.0
12 print(statusCodeAgainAgain) //prints 404
13 let statusMessageAgain = httpError.1
14 print(statusMessageAgain) //prints "Not Found"

```

4.1.4 Valori opzionali

In Java l'eccezione che ci viene fuori più spesso è la `NullPointerException`. Per risolvere questo problema in Swift hanno detto che di default, una variabile non può assumere un valore nullo. Hanno distinto due tipi, per qualunque tipo esiste il tipo opzionale (può essere nullo). Ma una variabile e una variabile opzionale non sono la stessa cosa, non possiamo passare in input ad un metodo che prende una variabile non opzionale una opzionale.

```

1 var i:Int? = nil
2 i=5
3 var j = 5;
4 j = nil //Compilation time error: j is not
5 optional, cannot be set to nil
6 //-----
7 var s1 = "123"
8 //let's create a Int from a string
9 var iii = Int(s1)
10 //The Int() initializer returns a Int?
11 //because not every string can be converted to Int
12 //so s1 is a Int?
13 print (iii) //prints "Optional(123)"
14 s1="ciao"
15 iii=Int(s1)
16 print(iii) //prints "nil"

```

Quando devo usare le variabili opzionali il programmatore è obbligato ad estrarre un valore non nullo, abbiamo vari modi per farlo.

Force Unwrap

Il modo più naturale è farlo con un `force unwrap`, si usa il punto esclamativo dopo la variabile per forzare l'estrazione del valore non nullo da una variabile nulla, si va a leggere quel tipo come se non fosse opzionale. Attenzione se si

usa questa sintassi su una variabile che è nil, viene generata un'eccezione a runtime.

```
1 var a = 1 //non-optional
2 var b = Int("123") //optional
3 var c:Int
4 c=a+b //this is a compile-time error
5 //b cannot be used in an expression because it
6 could be nil
7 //-----
8 //solution:
9 if b != nil {
10     //we use "!" to unwrap the value
11     c = a+b!
12 }
13 //force unwrapping of a nil variable generates
14 a runtime exception
```

Optional Binding

Vediamo un metodo più elegante, l'optional binding (anche indicato con if let), costruisco una variabile non opzionale e ci metto dentro il valore della opzionale:

```
1 var possibleNumber = "123"
2 var numberOptional = Int(possibleNumber)
3 //next line: check if numberOptional is nil,
4 if not, assign it to numberNonOptional and run
5 the if condition. Otherwise run the else
6 condition.
7 if let numberNonOptional = numberOptional {
8     print(numberNonOptional) //prints 123
9 } else {
10     print("cannot convert")
11 }
```

È possibile fare anche l'optional binding senza cambiare il nome della variabile:

```
1 var a: Int?
2 a = 5
3 if let a {
4     print(a+1)
5 } else {
6     print("a is nil")
7 }
```

```
7 }
```

Implicitly Unwrapped Optionals

Implicitly Unwrapped Optionals, si possono definire delle variabili opzionali ma che posso usarle come non opzionali. Se definisco una variabile con ! può essere nulla ma la voglio usare come se non lo fosse:

```
1 var a: String!
2 var b: String = ""
3 b = a //generates a runtime error: I am
4 assigning a nil value to a non-optional
5 variable
6 a = "ciao"
7 //next line: I can use b as if it is not
8 optional (note, I am not using the !)
9 b=a;
```

Una variabile definita come a è concettualmente simile ad un oggetto Java, può sempre essere nullo, non lo sappiamo a tempo di compilazione, ma solo di esecuzione.

Optional Chaining

C'è un modo che viene chiamato optional chaining in modo tale da dire: prova a chiamare quel metodo o a prendere l'attributo, se è nullo invece che ritornare un'eccezione mi ritorna nullo:

```
1 var user: Person?
2 var n = "" //an empty non-optional string
3 n=user.name //compile time error: cannot use a property
4               for optional variable user
5 n=user!.name //run-time exception if user is nil
6 n=user?.name //compile time error: name is a
7               non-optional property, but when used with
8 optional chaining can return nil, so it cannot be
9 assigned to non-optional variable n
10 var m: String?
11 m = user?.name
```

Nil-Coalescing Operator

L'operatore ?? viene utilizzato per rendere più breve un'operazione comune: data una variabile opzionale a, restituisce il valore a se a non è nil, altrimenti

restituisce il valore b (l'idea è che b sia il valore di default da usare):

```
1 let defaultColorName = "red"
2 var userDefinedColorName: String? // defaults to nil
3 var colorNameToUse = userDefinedColorName ??
    defaultColorName
4 //meaning: if userDefinedColorName is not nil (which is
    not the case in this example),
5 then colorNameToUse is userDefinedColorName. Otherwise
    colorNameToUse is
6 defaultColorName
```

4.1.5 Gestione dell'overflow

Vediamo l'esempio in Java, c'è un overflow che (molto spesso) genera un bug:

```
1 int a = Integer.MAX_VALUE -1;
2 int b = Integer.MAX_VALUE -1;
3 int c = a+b;
4 System.out.println(c); //4
```

In Swift invece gli overflow generano un'eccezione a runtime:

```
1 var a = Int.max
2 var b = Int.max
3 let c=a+b //error: Execution was interrupted
4 print(c)
```

Esiste un operatore particolare &+ che dice di non generare delle eccezioni in casi di overflow. Tipicamente l'overflow è una situazione non prevista dal programmatore: in Java potrebbe generare situazioni inattese, anche difficili da risolvere perché l'effetto del bug (l'overflow) potrebbe apparire molto distante dal bug stesso. In SWIFT il fatto che l'overflow generi un'eccezione aiuta a risolvere il problema ed evita che l'errore si propaghi (es: porti a memorizzare dati inconsistenti). In questo caso SWIFT non previene un errore comune, ma ne limita gli effetti e lo rende più semplice da risolvere.

4.1.6 Range operator

Sono stati introdotti degli operatori che permettono di creare rapidamente dei gruppi di interi o di elementi contigui di un array. Si usano abbastanza perchè i for si definiscono solo con i range.

- a...b : tutti gli elementi tra a e b (estremi inclusi)

- `a..<b`: tutti gli elementi da `a` (incluso) a `b` (escluso)
- `...a`: elementi fino ad `a`
- `a...` : elementi a partire da `a`

```

1  for i in 1...5 {
2      print(i)
3  } //1, 2, 3, 4, 5
4
5  for i in 1..<5 {
6      print(i)
7  } //1, 2, 3, 4
8
9  let names = ["Anna", "Alex", "Brian",
10 "Jack"]
11 for i in names[1...2] {print(i)}//Alex,
12 Brian
13 for i in names[...1] {print(i)}//Anna, Alex
14 for i in names[2...] {print(i)}//Brian, Jack

```

4.1.7 Collections

Sono predefinite tre strutture dati principali:

- Array: liste ordinate di valori
- Set: insieme non ordinati di valori
- Dictionary: insieme non ordinati di chiave valore

Array

```

1  var someInts = [Int]()
2  someInts.append(5)
3  someInts.append(3)
4  print(someInts.count) //prints 2
5  print(someInts[1]) //prints 3
6  someInts[1]=10
7  print(someInts[1]) //prints 10
8  if someInts.isEmpty {
9      print("vuoto")
10 }

```

```

11 else {
12     print("pieno")
13 }
14 for myInt in someInts {
15     print(myInt) //prints 2 then 10
16 }

```

Set

```

1 var mySet = Set<Int>()
2 mySet.insert(5)
3 mySet.insert(3)
4 print(mySet.count)
5 if mySet.contains(5) {
6     print("5 is in!")
7 }
8 var myOtherSet = Set<Int>()
9 myOtherSet.insert(7)
10 print(mySet.union(myOtherSet))
11 // [7, 5, 3]

```

È comodo perchè definisce anche le operazioni insiemistiche come intersezione, unione.

Dictionary

```

1 var weekdays = [String: String]()
2 //key-value added
3 weekdays["Sunday"] = "Domenica"
4 //key-value added
5 weekdays["Monday"] = "Lun"
6 print(weekdays["Sunday"]!) //Note:
7 why do we use "!" ?
8 print(weekdays["Saturday"]!) //
9 Runtime exception, why?
10
11 weekdays["Monday"] = "Lunedì" //
12 value updated
13 for (eng, it) in weekdays {
14     print("\(eng) translates to \
15     (it)")
16 }

```

Leggendo un valore di un dictionary otteniamo sempre un opzionale

4.1.8 Costrutti di selezione

Lo switch è simile a Java e a C, ma c'è una differenza sostanziale, abbiamo automaticamente il break nei casi:

```
1 let approximateCount = 62
2 var naturalCount = ""
3 switch approximateCount {
4 case 0:
5     naturalCount = "no"
6 case 1..<5:
7     naturalCount = "a few"
8 case 5..<12:
9     naturalCount = "several"
10 case 12..<100:
11     naturalCount = "dozens of"
12 case 100..<1000:
13     naturalCount = "hundreds of"
14 default:
15     naturalCount = "many"
16 }
17 print("There are \(naturalCount) objects.")
18 // Prints "There are dozens of moons
19 orbiting Saturn."
```

Guard

Hanno introdotto un altro costrutto di selezione. Supponiamo di dover fare un metodo ed all'inizio del metodo vogliamo controllare una certa condizione, va messo un if con due rami diversi, se mettiamo if una certa condizione else rischiamo che tutta la struttura del codice si complica. La cosa più comoda da fare sarebbe prima verificare la condizione che non vogliamo e nel caso uscire dalla funzione:

```
1 //approccio sconsigliato
2 if (condition) {
3     //a lot of code in the if
4 statement
5 }
6 //altrimenti non faccio nulla
7 //-----
8 //così è meglio!
9 if (!condition) {return}
10 //the code is outside the if
```

```

11 statement
12 //-----
13 //il costrutto guard permette di
14 scrivere questo in modo piu
15 leggibile
16 guard condition else {return}

```

Il guard fa questa cosa, garantisci che il denominatore sia diverso da zero, se lo è ritorniamo ed esegui quello che c'è nell'else:

```

1 func divide(numerator: Int,
2 denominator: Int) {
3     guard denominator != 0 else {
4         print("non si puo")
5         return
6     }
7     print(numerator/denominator)
8 }
9 divide(numerator:10, denominator: 2)

```

Il costrutto guard viene spesso utilizzato per verificare che il valore di alcuni parametri non sia nil:

```

1 func prova(a: Int?) {
2     guard let b = a else {return}
3     print(b+1)
4 }
5
6 func prova(a: Int?) {
7     guard let a else {return}
8     print(a+1)
9 }

```

4.2 Funzioni e closures

Caratteristiche delle funzioni:

- possono avere più valori di ritorno
- i parametri possono avere dei valori di default
- le funzioni sono un tipo di dato, possono essere assegnate ad una variabile o passate come argomento di altre funzioni

Vediamo la sintassi:

```

1 func greet(person: String) -> String {
2     let greeting = "Hello, " + person + "!"
3     return greeting
4 }
5
6 print(greet(person: "Sergio"))

```

4.2.1 Valori di ritorno multipli

Se vogliamo valori di ritorno multipli possiamo ritornare una tupla:

```

1 func minMax(array: [Int]) -> (min:Int, max:Int) {
2     var currentMin=array[0]
3     var currentMax=array[0]
4     for i in array[1..

```

4.2.2 Augments lables e parameters name

Probabilmente chi ha progettato Swift ha pensato che un errore comune dei programmatori è scambiare i parametri. Se quando chiamiamo una funzione e invertiamo l'ordine dei parametri il compilatore non ha modo di accorgersi dell'errore. Per evitare questo in Swift hanno deciso che per ogni parametri si specifica:

- Il nome con cui chiamiamo il parametro nella funzione
- Il nome del parametro che deve usare chi chiama da fuori

Se ne indico solo uno viene usato come parameters name e argument lable, posso anche usare l'_' per evitare di dichiarare l'argument lable.

```

1 func printName1(name: String) {
2     print("ciao " + name)
3 }

```

```

4 printName1(name: "Sergio")
5 //-----
6 func printName2(name toPrint:
7 String) {
8     print("ciao " + toPrint)
9 }
10 printName2(name: "Sergio")
11 //-----
12 func printName3(_ toPrint: String) {
13     print("ciao " + toPrint)
14 }
15 printName3("Sergio")

```

Non è solo una questione di leggibilità e riduzione degli errori, l'argument label ha un valore sintattico. La signature di un metodo è un insieme di informazioni che ci permette di distinguerlo da tutti gli altri, in Java è fatta da il nome del metodo e dall'elenco ordinato dei tipi. In Swift la signature di un metodo è data dal nome e dalle coppie label e tipo, siccome abbiamo aggiunto la label possiamo avere due metodi che hanno lo stesso nome e lo stesso elenco dei tipi, basta che abbiano le label diverse, come vediamo in esempio:

```

1 func f(aLabel value: String) {
2     print("Ciao \(value)")
3 }
4
5 func f(anotherLabel value: String)
6 {
7     print("Hola \(value)")
8 }
9
10 //f(aLabel) and f(anotherLabel)
11 //are syntactically two distinct
12 functions
13 f(aLabel: "Sergio")
14 f(anotherLabel: "Mattia")

```

4.2.3 Parametri opzionali

Posso definire dei parametri opzionali dandogli un valore che poi quando andiamo a richiamare la funzione posso mettere o meno:

```

1 func f(a: Int, b: Int, c: Int=5)
2 -> Int {

```

```

3     return a+b+c
4 }
5
6 print(f(a:10, b:10, c:10)) //30
7 print(f(a:10, b:10)) //25

```

4.2.4 Passaggio dei parametri per riferimento

Di default i parametri passati ad una funzione sono costanti, all'interno della funzione stessa. Si può fare in modo che una funzione modifichi i parametri specificando la parola riservata `inout`. Quando si invoca la funzione, i parametri devono essere preceduti dal simbolo `&`:

```

1 func swap(_ a: inout Int, _ b: inout Int) {
2     let c = a;
3     a=b;
4     b = c;
5 }
6
7 var x=5, y=10;
8 swap(&x, &y);
9 print("\(x) \(y)") //Prints 10 5

```

4.2.5 Le funzioni come tipi

Le funzioni sono un tipo di dato, definito dal tipo dei parametri e del valore di ritorno. Questo rende possibile assegnare funzioni a variabili o passare le funzioni come argomenti di altre funzioni. Quindi dobbiamo dire che tipo di funzione è, il tipo è dato dai parametri e il tipo di ritorno.

```

1 func sum(_ a: Int, _ b: Int) -> Int {
2     return a+b;
3 }
4
5 func dif(_ a: Int, _ b: Int) -> Int {
6     return a-b;
7 }
8
9 func f(theFunction: (Int, Int)->Int, _ a: Int, _ b:
10     Int) -> Int {
11     return theFunction(a, b);
12 }

```

```

13 let a=5, b=3
14 var operation = sum //comoda l'inferenza di un tipo
    funzione
15 print(f(theFunction: operation, a, b)); //8
16 operation = dif
17 print(f(theFunction: operation, a, b)); //2

```

É possibile assegnare un nome nuovo ad un tipo. É utile quando si usano le funzioni come tipi:

```

1 func sum(_ a: Int, _ b: Int) -> Int {
2     return a+b;
3 }
4
5 func dif(_ a: Int, _ b: Int) -> Int {
6     return a-b;
7 }
8
9 typealias BinaryIntergerFunction = (Int, Int) -> Int
10
11 //confronta il metodo che segue con quello della
12 slide precedente: questo e piu semplice da leggere
13 func f(theFunction: BinaryIntergerFunction, _ a: Int,
14 _ b: Int) -> Int {
15     return theFunction(a, b);
16 }
17 }
18
19 let a=5, b=3
20 var operation = sum
21 print(f(theFunction: operation, a, b)); //8
22 operation = dif
23 print(f(theFunction: operation, a, b)); //2

```

4.2.6 Un esempio esteso

La programmazione funzionale è molto comoda, perchè di fatto abbiamo una variabile che rappresenta una funzione. Vediamo un esempio, vogliamo definire un algoritmo di ordinamento in Java hanno definito delle interfacce da implementare. Dal punto di vista di chi scrive un algoritmo di ordinamento interessa sapere tra due elementi quale viene prima e quale dopo, quindi basterebbe indicare quale viene prima e si può implementare la funzione. In programmazione funzionale si risolve facilmente, possiamo avere

un algoritmo di ordinamento che prende in input la struttura su cui fare l'ordinamento e la funzione con cui fare l'ordinamento. In questo caso abbiamo un array di nomi, prende in input una funzione e ritorna un booleano se l'ha ordinata:

```
1 func forward(first:String, second:String) -> Bool {
2     return first<second
3 }
4
5 func backward(first:String, second:String) -> Bool {
6     return first>second
7 }
8
9 func lengthAsc(first:String, second:String) -> Bool {
10    return first.count<second.count
11 }
12
13 func lengthDesc(first:String, second:String) -> Bool {
14    return first.count>second.count
15 }
16
17 let names = ["Carlo", "Daniele", "Ugo", "Bruno",
18             "Sergio", "Mattia"]
19 /*Array has a "sorted" method that takes in input a
20    function with two parameters
21    (same type as the array) and returns true if the first
22    should
23    stay before the second*/
24 print(names.sorted(by: forward))
25 //["Bruno", "Carlo", "Daniele", "Mattia", "Sergio",
26    "Ugo"]
27 print(names.sorted(by: backward))
28 //["Ugo", "Sergio", "Mattia", "Daniele", "Carlo",
29    "Bruno"]
30 print(names.sorted(by: lengthAsc))
31 //["Ugo", "Carlo", "Bruno", "Sergio", "Mattia",
32    "Daniele"]
33 print(names.sorted(by: lengthDesc))
34 //["Daniele", "Sergio", "Mattia", "Carlo", "Bruno",
35    "Ugo"]
```

4.2.7 Funzioni innestate

È possibile creare delle funzioni innestate, è comodo quando ho una funzione complicata e la visibilità della funzione è all'interno di questa:

```
1 func out() {
2     func in1() {print("ciao")}
3     func in2() {print("a tutti")}
4     in1()
5     in2()
6 }
7
8 out() //prints ciao a tutti
9 in1() //compile time error: in1
10 does not exist in this scope
```

4.2.8 Closures

Le closures sono un costrutto sintattico, intuitivamente simile alle lambda expression, che serve per definire nel punto esatto del codice una funzione. Le funzioni servono anche per dire come vogliamo risolvere il problema, come abbiamo visto il criterio di ordinamento prima. Servono per risolvere in modo più compatto alcuni problemi comuni:

- Come ad esempio la gestione degli eventi, cosa molto comune nella programmazione mobile
- Sono alla base della sintassi di SwiftUI

Sono un costrutto sintattico che mi permette di definire la funzione e tutto il codice la dove mi serve:

```
1 let names = ["Carlo", "Daniele", "Ugo", "Bruno",
2             "Sergio", "Mattia"]
3 print(names.sorted(by: {(first: String, second: String)
4     -> Bool in
5     return first < second
6 })))
7 //["Bruno", "Carlo", "Daniele", "Mattia", "Sergio",
8    "Ugo"]
```

Sintatticamente abbiamo la coppia di graffe, poi un insieme di parametri tra parentesi tonde, una freccia con il valore di ritorno e poi la parola riservata `in` che fa iniziare il codice vero e proprio della funzione.

Type inference

Siccome sono delle funzioni potremmo mettere variabile = closure e specificarla, in quel caso dovremmo specificare i tipi. Ma non è quello che facciamo di solito, spesso usiamo la closure come parametro della funzione quindi possiamo evitare di dire quali sono i tipo della closure:

```
1 let names = ["Carlo", "Daniele", "Ugo", "Bruno",  
  "Sergio", "Mattia"]  
2 print(names.sorted(by:{first, second in  
3   return first<second  
4 })))  
5 //["Bruno", "Carlo", "Daniele", "Mattia", "Sergio",  
  "Ugo"]
```

Implicit return

Se la closure ha una sola riga di codice il return è implicito:

```
1 let names = ["Carlo", "Daniele", "Ugo", "Bruno",  
  "Sergio", "Mattia"]  
2 print(names.sorted(by:{first, second in first<second}))  
3 //["Bruno", "Carlo", "Daniele", "Mattia", "Sergio",  
  "Ugo"]
```

Shorthand Argument Names

Se non specifichiamo i nomi dei parametri ma vengono usati i \$0,1...n:

```
1 let names = ["Carlo", "Daniele", "Ugo", "Bruno",  
  "Sergio", "Mattia"]  
2 print(names.sorted(by:{$0<$1}))  
3 //["Bruno", "Carlo", "Daniele", "Mattia", "Sergio",  
  "Ugo"]
```

Operation methods

Se l'unica operazione da compiere è applicare un operatore ai due parametri, possiamo omettere i parametri stessi e indicare solo l'operatore. In questo caso non indichiamo neanche le parentesi graffe:

```
1 let names = ["Carlo", "Daniele", "Ugo", "Bruno",  
  "Sergio", "Mattia"]  
2 print(names.sorted(by:<))
```

```
3 //["Bruno", "Carlo", "Daniele", "Mattia", "Sergio",
    "Ugo"]
```

Trailing Closure

Se una funzione ha un elenco di parametri e l'ultimo è una funzione e utilizziamo una closure per rappresentare quella funzione la posso mettere fuori dalle parentesi tonde e se la funzione rimane senza parametri posso non indicare neanche le parentesi tonde:

```
1 let names = ["Carlo", "Daniele", "Ugo", "Bruno",
    "Sergio", "Mattia"]
2 print(names.sorted {$0<$1})
3 //["Bruno", "Carlo", "Daniele", "Mattia", "Sergio",
    "Ugo"]
```

Esempio con closure

Questo è il modo in cui risolviamo l'ordinamento con le closures:

```
1 let names = ["Carlo", "Daniele", "Ugo", "Bruno",
    "Sergio", "Mattia"]
2 print(names.sorted {$0<$1})
3 //["Bruno", "Carlo", "Daniele", "Mattia", "Sergio",
    "Ugo"]
4 print(names.sorted {$0>$1})
5 //["Ugo", "Sergio", "Mattia", "Daniele", "Carlo",
    "Bruno"]
6 print(names.sorted {$0.count<$1.count})
7 //["Ugo", "Carlo", "Bruno", "Sergio", "Mattia",
    "Daniele"]
8 print(names.sorted {$0.count>$1.count})
9 //["Daniele", "Sergio", "Mattia", "Carlo", "Bruno",
    "Ugo"]
```

Cattura dei valori

Fino ad ora abbiamo visto questioni meramente sintattiche relative alle closure. Vediamo ora una questione sia sintattica che concettuale. L'implementazione delle closure in JS ha un comportamento completamente anti intuitivo, per farlo funzionare in Swift hanno dovuto implementare una funzionalità che si chiama cattura di valori.

Vediamo un esempio, qui abbiamo un ciclo, qui aspetta un secondo poi quando va a stampare tutto è arrivato a 9 e stampa 9 volte 9:

```
1 for (var i = 0; i < 9; i++) {
2     setTimeout(() => console.log(i), 1000)
3     //la funzione setTimeout esegue la funzione
4     //indicata come primo parametro dopo il
5     //numero di millisecondi indicato nel secondo
6     parametro
7 }
```

L'analogo codice swift stampa i numeri da 1 a 9:

```
1 for i in 1...9 {
2     setTimeout(delay:1, function: {() in return i})
3     //in forma piu compatta:
4     //setTimeout(delay:1) {i}
5 }
6
7 func setTimeout(delay:UInt32, function:()->Int) {
8     sleep(delay)
9     print(function())
10 }
```

Vediamo un altro esempio:

```
1 func creaSalutatore(nome: String) -> () -> String {
2     return {() in "Ciao \(nome)"}
3 }
4
5 let a = creaSalutatore(nome: "Sergio")
6 let b = creaSalutatore(nome: "Mascetti")
7 print(a())
8 //Ciao Sergio
9 print(b())
10 //Ciao Mascetti
11 print(nome)
12 //ERRORE! La variabile nome non esiste
```

La funzione creaSalutatore ritorna una funzione che a sua volta ritorna una stringa. La cosa strana è che quando invoco la funzione che ha ritornato creaSalutatore ritorna correttamente il nome, riesce a farlo perchè ha catturato i valori, si crea una copia dei valori e quando andiamo a richiamarla si ricorda dei valori che aveva quando è stata definita e non quando è stata richiamata.

4.3 Object Oriented Programming

Swift distingue tra tre costrutti sintattici:

- Classi (Class)
- Strutture (Struct)
- Enumeration (Enum)

Classi e strutture hanno cose in comune:

- Contengono attributi e metodi
- Possono avere costruttori
- Sfruttano l'ereditarietà

In generale si parla di istanze invece di oggetti.

Come si definiscono:

```
1 struct Resolution {
2     var width = 0
3     var height = 0
4 }
5
6 class VideoMode {
7     var resolution = Resolution()
8     var interlaced = false
9     var frameRate = 0.0
10    var name: String?
11 }
12
13 let vm = VideoMode()
14 print(vm.frameRate)
15 vm.resolution.height = 10
```

Non abbiamo la parola riservata new.

4.3.1 Costruttori (inizializzatori)

Si indica esplicitamente con la parola riservata `init`, che può prendere o meno in input dei parametri:

```

1 struct Celsius {
2     var temperatureInCelsius: Double
3     init(fromFahrenheit fahrenheit: Double) {
4         temperatureInCelsius = (fahrenheit - 32.0) / 1.8
5     }
6
7     init(fromKelvin kelvin: Double) {
8         temperatureInCelsius = kelvin - 273.15
9     }
10 }
11
12 let boilingPointOfWater = Celsius(fromFahrenheit:
13 212.0)
14 // boilingPointOfWater.temperatureInCelsius is 100.0
15 let freezingPointOfWater = Celsius(fromKelvin:
16 273.15)
17 // freezingPointOfWater.temperatureInCelsius is 0.0

```

C'è tutta una serie di regole sintattiche per dire quando viene creato un inzializzatore di default, l'idea base è questa: se abbiamo una classe che definisce valori di default per tutte le proprietà e non ha nessun inzializzatore esplicito viene definito in automatico un inzializzatore implicito.

```

1 class AClass {
2     var a = 5
3     var b = 5
4 }
5
6 print(AClass())

```

4.3.2 Copie e riferimenti

Le struct sono dette value type, cioè ogni volta che vengono associate ad una variabile ne viene fatta una copia. Le classi sono reference type, ogni volta che vengono associate ad una variabile viene fatto un puntatore a quell'istanza. I tipi primitivi in Java hanno una semantica uguale a quella dei value type, i reference type sono tutte le classi. In Swift possiamo definire dei costrutti con semantica value type personalizzati:

```

1 struct AStruct {
2     var aParam = 0
3 }
4 let aStruct = AStruct()
5 //anotherStruct is a copy!

```

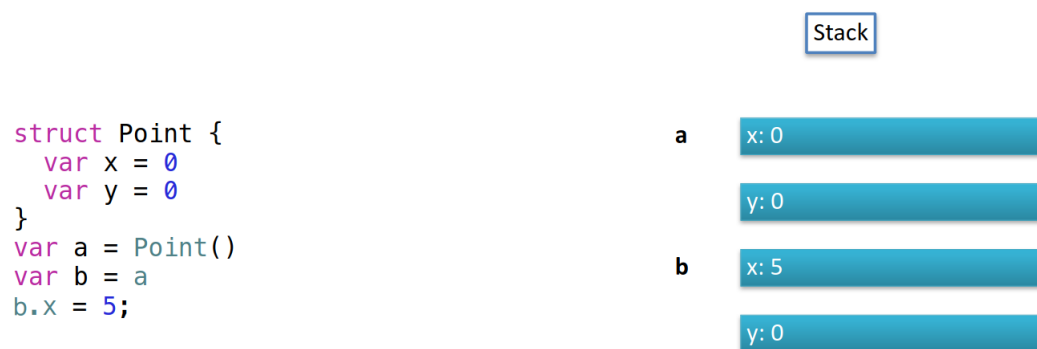
```

6 var anotherStruct = aStruct
7 anotherStruct.aParam = 5;
8 print(aStruct.aParam) //0
9 print(anotherStruct.aParam) //5
10
11 class AClass {
12     var aParam = 0
13 }
14 let aClass = AClass()
15 //this is a reference to the same instance
16 var anotherClass = aClass
17 anotherClass.aParam = 5;
18 print(aClass.aParam) //5
19 print(anotherClass.aParam) //5

```

Nel primo caso sono due copie, quando assegno a due variabili diverse ne sto facendo una copi, per esempio cambia il comportamento, se cambio anotherStruct cambiando l'attributo a 5 e poi li stampo, vedrò due valori diversi. Nel secondo caso invece è come Java, facciamo puntare una seconda variabile allo stesso oggetto, quando cambia uno cambia anche l'altro.

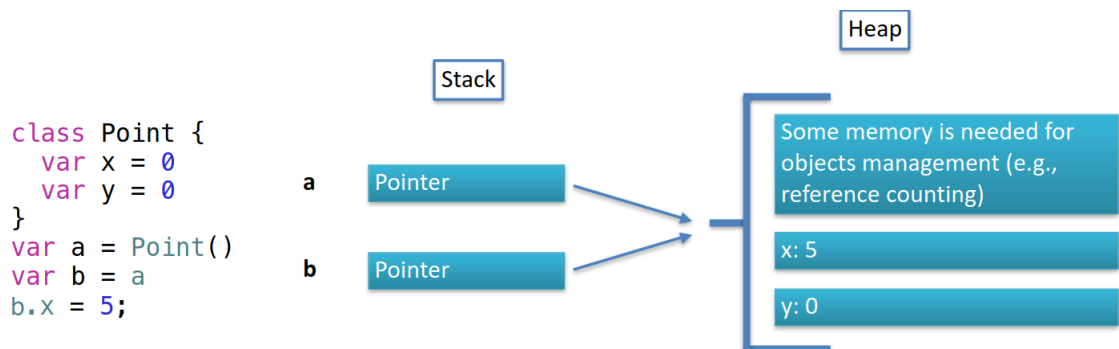
Ovviamente cambia la gestione della memoria, le struct essendo value type vengono allocate nello stack:



Il reference type è diverso, perchè abbiamo allocato nello heap i dati mentre nello stack abbiamo due puntatori:

Ovviamente questo maggiore controllo che abbiamo ci dà dei vantaggi e degli svantaggi, se non facciamo attenzione ci confondiamo. I value type hanno una serie di vantaggi, l'accesso alla memoria più rapido e la gestione della memoria è molto più semplice perchè allochiamo spazio nello stack e non occupiamo lo heap (evitando allocazione dinamica nello heap).

Non è detto che sia sempre così semplice, ci sono un sacco di oggetti che sono value type che al loro interno contengono una quantità di dati variabile,



quindi spesso si ha la struct che rappresenta la stringa e poi invece una classe dove vado a memorizzare i caratteri della stringa.

Ci sono molti casi in cui il reference type è più comodo, ad esempio se avessimo un'entità che deve essere acceduta da varie parti del codice, se abbiamo un'entità che contiene tanti dati alla quale (singola istanza) dobbiamo avere più riferimenti, non ha senso duplicarla più volte.

4.3.3 Cosa scelgo? Struct o class

Quello che consigliano le best practice consigliano di usare una struct, poi se vediamo che serve una classe la trasformiamo in classe.

4.3.4 Proprietà (attributi)

Esistono due tipi di proprietà:

- Stored properties: corrispondono agli attributi in Java
- Computed properties: sono sostanzialmente dei metodi che permettono di calcolare dei valori. Vengono però esposti all'esterno con una sintassi simile a quella delle stored properties, dunque usando l'istanza non si sa se siano valori memorizzati o calcolati al momento

Stored properties

```

1 struct AStruct {
2     var a = 5
3     let b: Int
4 }
5
6 var varStruct = AStruct(a: 10, b: 10)
7 varStruct.a = 20 //this is OK, a is var

```

```

8 varStruct.b = 20 //compile time error, b is let
9 let letStruct = AStruct(a: 10, b: 10)
10 letStruct.a = 20 //a is var, but the instance is let,
    so we cannot change its attributes. This is a
    compile time error

```

Un'istanza di una classe con `let` implica che anche gli attributi della struct non posso cambiarli, perchè è tutta la classe che è una costante.

Computer Properties

Sono un costrutto sintattico al quale si accede dall'esterno come se fossero attributi ma internamente hanno un comportamento come se fossero dei metodi:

```

1 struct Circle {
2     var radius: Double
3     var area: Double {
4         get {
5             return Double.pi*radius*radius
6         }
7         set(newArea) {
8             radius = (newArea /
9                 Double.pi).squareRoot()
10        }
11    }
12 }
13
14 var a = Circle(radius: 1)
15 print(a.area)
16 //3.141592653589793
17 a.area = 10
18 print(a.radius)
19 //1.7841241161527712

```

Abbiamo un modo per definire un attributo sul quale possiamo scrivere del codice quando viene acceduto o modificato.

4.3.5 Metodi

Sono simili a quello che avviene in Java. La parola riservata `self` è analoga a `this` e può essere sottintesa.

```

1 class MyClass {
2     var a = 5

```

```

3     func increment(by value: Int) {
4         a += value
5     }
6 }
7
8 var i = MyClass()
9 print(i.a) //5
10 i.increment(by: 5)
11 print(i.a) //10

```

4.3.6 Computed properties VS Metodi

Ci sono casi in cui non potrei usare le computed properties ad esempio se ci sono metodi con più parametri. Però ci sono dei casi in cui si possono usare o usare un metodo. È meglio usare una computed properties quando dal punto di vista di chi accede al metodo è come se stesse accedendo ad una variabile. Significa che il costo della computed property deve essere trascurabile e deve ritornare sempre lo stesso valore.

4.3.7 Ereditarietà

Classi e strutture hanno un approccio diverso all'ereditarietà.

- Una classe può estendere un'altra classe (al massimo una)
- Una classe/struttura può implementare un'interfaccia. In terminologia Swift interfaccia è protocollo e implementare un'interfaccia diventa adottare un protocollo
- Non esiste il concetto di classe "originale" come in java, le classe non ereditano da Object
- Abbiamo la parola riservata any che fa riferimento a una qualunque classe o struttura
- Le struct non si possono estendere

4.3.8 Estensione di classi

```

1 class Base {
2     var a = 5
3     func pippo() {

```

```

4         print("ciao")
5     }
6 }
7
8 class ExtendsBase: Base {
9     override func pippo() {
10         super.pippo()
11         print("hola")
12     }
13 }
14
15 var instance = ExtendsBase()
16 instance.a = 7
17 instance.pippo()
18 //prints ciao hola

```

Una classe che ne estende un'altra può definire nuove proprietà e metodi e può sovrascrivere quelli della superclasse:

- è obbligatorio esplicitare che si vuole fare un override
- se si vuole accedere ai metodi/proprietà della superclasse si usa la parola riservata `super`

Si può fare anche l'override delle proprietà. La parola `final` rende un metodo/proprietà non sovrascrivibile.

4.3.9 Controllo del tipo

A differenza di Java che usa `InstanceOf` abbiamo la parola riservata `is`.

Downcasting

Il problema del cast sbagliato genera delle eccezioni in Java, qui si è voluto limitare anche questo tipo di eccezioni. Quindi Swift introduce la parola riservata `as` e si può fare in due modi:

- `as?` converte se riesce il tipo di un oggetto, se non riesce ritorna `nil`. È comodo perchè solitamente lo associamo all'optional binding.
- `as!` converte il tipo e se non riesce viene generata un'eccezione

```

1 if let movieElem = elem as? Movie {
2     //qua uso movieElem sapendo che e di tipo Movie
3 }

```

4.3.10 Protocolli

I protocolli rappresentano l'analogo delle interfacce in Java, definiscono una serie di metodi e di proprietà. Però non dicono come devono essere implementati, dicono solo che se qualche istanza adotta quel protocollo allora deve implementare i metodi definite dal protocollo. Sia la classi che le struct possono aderire a uno o più protocolli, possono estendere una classe e implementare uno o più protocolli.

```
1 protocol CanJump {
2     func jumpNow()
3 }
4
5 protocol CanSwim {
6     func swimNow()
7 }
8
9 struct Cat: CanJump {
10     func jumpNow() {
11         print("salto quando voglio")
12     }
13 }
14
15 struct Dog: CanJump, CanSwim {
16     func jumpNow() {
17         print("Certo, adoro saltare")
18     }
19
20     func swimNow() {
21         print("Certo, adoro nuotare")
22     }
23 }
```

Protocolli e proprietà

A differenza di Java un protocollo può definire delle proprietà e definire come si può accedere a queste proprietà:

```
1 protocol FullyNamed{
2     var name: String {get}
3 }
4 //this struct conforms to the protocol:
5 //in addition to property read (i.e., get) it also
6 //provides property write(set).
```

```

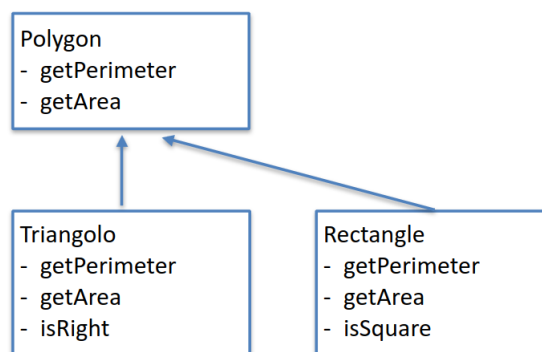
7 //Remember: protocols define
8 the minimum requirements
9 struct Person: FullyNamed {
10     var name: String
11 }

```

Concetto di tipo

Un protocollo essendo un tipo possiamo avere una funzione che prende in input un protocollo, sapendo che gli oggetti che passeremo aderiranno a quel protocollo.

Gerarchia



Ho una gerarchia, triangolo e rettangolo estendono poligono. Voglio aggiungere un comportamento a tutti questi, setColor, se modifichiamo polygon tutti ottengono quel metodo. Ci sono dei casi, non rari, in cui non ho accesso al codice di polygon. Dovremmo:

- Estendere tutte le classi creandone di nuove mettendoci il colore

4.3.11 Extension

In Swift è possibile estendere il comportamento di un tipo già esistente, anche nel caso in cui non disponiamo del sorgente del tipo originario. Tipicamente si fa in modo che quella classe o struttura adotti un protocollo in più rispetto a quelli che aveva:

```

1 protocol FullyNamed{
2     var name: String {get}
3 }

```

```

4
5 extension Int: FullyNamed {
6     var name: String {
7         return "The full
8         name for this int is \
9         (self)"
10    }
11 }
12
13 var a: Int = 5
14 print(a.name)

```

Nell'esempio di prima possiamo estendere Polygon:

Questo a volte rende più difficile la lettura, perchè a volte troviamo delle classi che hanno metodi che poi in documentazione non ci sono.

Esempio

Supponiamo di avere un nuovo formato di codifica dell'informazione e vogliamo fare in modo che alcuni classi già esistenti mi vengano stampate in questo nuovo formato. Prendiamo la classe Array, facciamo l'estensione al metodo e lo richiamiamo:

```

1 protocol PJSONable {
2     func toJSON() -> String
3 }
4
5 extension Array:PJSONable {
6     func toJSON() -> String {
7         return "JSON format \
8         (self)"
9     }
10 }
11
12 var a=[1, 2, 3]
13 print(a.toJSON())
14 //JSON format [1, 2, 3]

```

Estensione dei protocolli

Posso fare l'estensione anche di un protocollo, così facendo posso avere un protocollo in cui definisco una parte di comportamento:

```

1 protocol FullyNamed{

```

```

2     var name: String {get}
3     var initial: String {get}
4 }
5
6 extension FullyNamed {
7     var initial: String {
8         get {
9             return String(name.prefix(1))
10        }
11    }
12 }
13
14 struct Structure: FullyNamed {
15     var name: String
16 }
17
18 var a=Structure(name: "Sergio")
19 print(a.initial)

```

4.3.12 Gestione della memoria

In Java non dobbiamo gestire la memoria, perchè ogni tanto parte il garbage collector che costruisce un grafo i cui nodi sono gli oggetti (o degli oggetti) e gli archi sono dei puntatori a questi oggetti. Finchè ci sono delle variabili che puntano al sottografo (finchè siamo nello scope) il garbage collector non lo tocca, quando usciamo dallo scope il sottografo non è più puntato e elimina gli oggetti dalla memoria. Il vantaggio del garbage collector è che il programmatore può ignorare il problema della memoria, però ha un costo computazionale, che nei dispositivi mobili ha un costo anche per il consumo di batteria. Abbiamo quindi un impatto sulle prestazioni del sistema dovuto al fatto che per cancellare la memoria dobbiamo costruire il grafo della memoria e cancellare le parti disconnesse.

4.3.13 Manual Retain Release

Allora una soluzione potrebbe essere di rinunciare al garbage collector.

Partiamo da come si faceva a gestire la memoria in Objective-C fino a 5/6 anni fa, l'approccio era chiamato manual retain end release, ogni oggetto in memoria aveva un contatore che indicava il numero di archi entranti del grafo della memoria, cioè il numero di altre variabili che puntano a quell'oggetto. Poi c'erano due metodi:

- Retain: fa aumentare di uno quel contatore
- Release: fa due cose:
 - Diminuisce di uno il contatore
 - Se il contatore va a zero cancella l'oggetto

Un esempio di codice:

```

1 a = Person() //variabile punta oggetto
2 a.retain() //contatore a ++
3 h = House() //variabile punta oggetto
4 h.retain() //contatore h ++
5 h.owner = a //variabile punta oggetto
6 a.retain() //contatore a ++
7 h.owner.release() //contatore a --
8 //non posso senza sopra perche rimane puntatore ad a
9 h.release()
10 //non posso fare cosi senza sopra, perche rimane
    contatore
11 h = House()
12 a.release()
13 a = NIL

```

Un vantaggio è che non ho più il garbage collector, è il programmatore che mi dice che oggetti cancellare. Il programmatore è costretto a concentrarsi sulla gestione della memoria piuttosto che sul problema da risolvere, è una cosa terribilmente error prone. Possiamo fare due tipi di errore qui:

- usiamo un valore di contatore più basso di quello che è realmente, ci dimentichiamo una retain. Rischiamo che a un certo punto facendo una release il contatore vada a 0 anche se l'oggetto mi serve ancora, riusando la variabile trovo nullo
- dimentico di fare una release, abbiamo un memory leak, oggetto in memoria irraggiungibile. Il memory leak non ci dà un crash immediato, potremmo avere un memory leak in un ciclo oppure in una parte del codice che viene richiamata spesso avremo la memoria che nel tempo continua a crescere, se non c'è più memoria l'app crasha

4.3.14 Automatic Reference Counting

L'intuizione è di provare ad automatizzare questo codice. C'era un sistema che si chiama ARC (automatic reference counting) che automatizza quasi

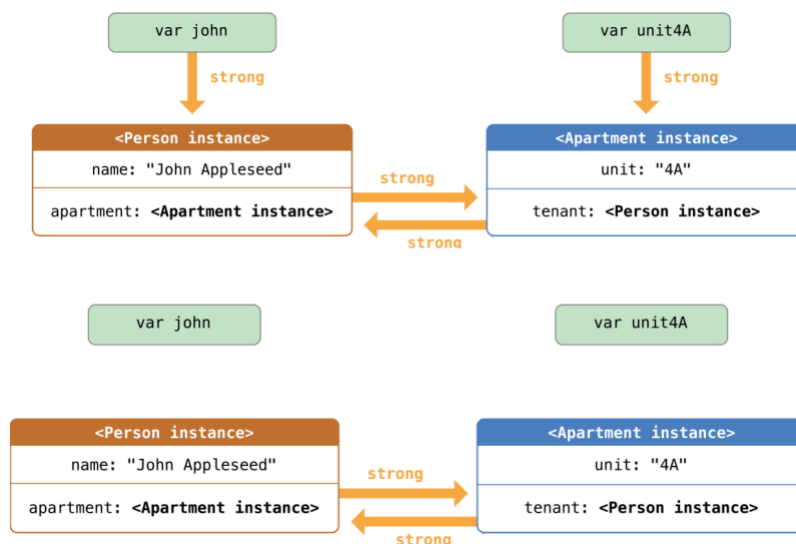
tutto questo, ma non tutto, rimane un piccolo problema e se non siamo consapevoli di questo problema non possiamo programmare in Swift.

Reference Cycle

La questione che non è stata ancora risolta è quella dei riferimenti circolari. Supponiamo di avere due oggetti A e B, A ha un riferimento a B, e viceversa, nessun altro oggetto ha un riferimento ad A o B. Intuitivamente A e B sarebbero da eliminare, non è più possibile accedervi, il garbage collector di Java li eliminerebbe. però il reference count di A e di B è ad uno, dunque non saranno eliminati da ARC.

Tutte le volte che abbiamo una variabile che punta ad un oggetto la possiamo dichiarare in due modi:

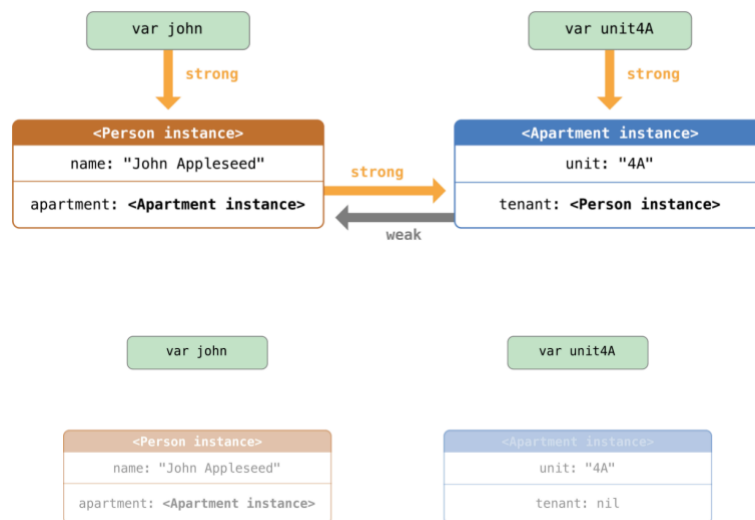
- Strong: mi aumenta il reference count, di default sono tutte così
- Weak: se mi rendo conto che c'è un riferimento circolare devo creare un collegamento come weak, questo collegamento non mi aumenta il reference count. Non posso mettere tutti i riferimenti a weak, perchè non è garantito che l'oggetto ci sia



Quando abbiamo un riferimento circolare, dobbiamo fare in modo che uno dei due puntatori (A o B o viceversa) sia weak. Esempio:

- supponiamo che il riferimento da A a B sia weak, quello da B ad A sia strong

- Nessun altro oggetto ha riferimenti ad A o B
- Il reference count di B è zero, quindi B viene eliminato
- Di conseguenza anche il reference count di A diventa zero, dunque anche A viene eliminato



Tutto quello che abbiamo visto funziona per le classi, non per le struct, perchè essendo nello heap quando finisce l'esecuzione vengono cancellate.

Swift introduce tutta una serie di proprietà di linguaggio a cui non eravamo abituati, lo hanno fatto principalmente per garantire delle prestazioni migliori.

4.4 iOS Programming

C'è una differenza notevole per quanto riguarda il problema della frammentazione rispetto ad Android. Android soffre del problema dovuto al fatto che ci sono tantissimi dispositivi supportati e tantissime versioni del sistema operativo. Per iOS è un problema molto meno marcato:

- Ci sono meno dispositivi
- Tipicamente i dispositivi iOS sono più recenti, quindi la maggior parte dei dispositivi usa la versione più recente

4.4.1 Xcode

É l'ambiente di sviluppo per tutte le piattaforme Apple. Storicamente era più stabile rispetto ad Android studio, è peggiorato nel tempo. Ci sono meno problemi con i simulatori e ci sono buoni strumenti di debugging e tuning. Storicamente la firma e la pubblicazione delle applicazioni è più complessa, anche se di recente la procedura è stata semplificata.

4.4.2 iOS vs Android

Chiaramente ci sono delle differenze tra i due ambienti, ad esempio:

- il linguaggio di programmazione
- le librerie di supporto
- l'ambiente di sviluppo

Tuttavia la maggior parte dei concetti relativi alla programmazione mobile rimane uguale:

- Singolo thread che modifica l'interfaccia grafica
- Come si gestiscono gli eventi
- Ciclo di vita

4.4.3 Gestione delle view

Ci sono due paradigmi principali di definizione delle UI:

- Imperativo: il programmatore indica i passaggi necessari per ottenere un comportamento:
 - Definizione statica dell'interfaccia
 - Programmazione dinamica dell'interfaccia
- Dichiarativa: un sottocaso comune è quello reattivo, cioè si reagisce al cambio dello stato

4.4.4 Approccio delle UI: approccio dinamico

Funzionava così java swing. Per ogni elemento da visualizzare dovevamo creare un oggetto e dargli tutte le caratteristiche e anche la struttura, il pulsante sta nel Panel (come un div). Era prolisso, error prone e soprattutto è più difficile focalizzarsi sugli aspetti di UI e UX.

```

public static void main(String[] args) {
    // Creating instance of JFrame
    JFrame frame = new JFrame("My First Swing Example");
    // Setting the width and height of frame
    frame.setSize(350, 200);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    /* Creating panel. This is same as a div tag in HTML
     * We can create several panels and add them to specific
     * positions in a JFrame. Inside panels we can add text
     * fields, buttons and other components.
     */
    JPanel panel = new JPanel();
    // adding panel to frame
    frame.add(panel);
    /* calling user defined method for adding components
     * to the panel.
     */
    placeComponents(panel);

    // Setting the frame visibility to true
    frame.setVisible(true);
}

```

4.4.5 Approccio statico

Abbiamo una rappresentazione statica dell'interfaccia, ad esempio in Android abbiamo un file XML che rappresenta l'interfaccia, poi quando esegue il codice in automatico per ogni elemento di interfaccia grafica crea un oggetto dinamico che rappresenta gli oggetti statici. Il limite è che bisogna creare un collegamento tra il codice dinamico e il linguaggio di markup. Ci si ritrova ad avere due versioni dell'interfaccia, quella definita staticamente e quella a seguito dell'interazione dell'utente.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is a TextView" />

    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is a Button" />

    <!-- More GUI components go here -->

</LinearLayout>

```

4.4.6 Esempio dichiarativo (con sistema reattivo)

L'idea dell'approccio dichiarativo è distinguere:

- lo stato
- il metodo che ridisegna l'interfaccia sul cambiamento dello stato

	<i>Imperativo</i>	<i>Dichiarativo</i>
<i>Web</i>	HTML+CSS+JS	React
<i>Android</i>	Nome non definito (Java o Kotlin)	Jatpack compose (Solo Kotlin)
<i>iOS</i>	UIKit (Objective C o Swift)	SwiftUI (Solo Swift)
<i>Cross Platform</i>	Xamarin	ReactNative

4.4.7 UIKit

É la libreria di definizione di interfacce grafiche. Viene anche chiamata impropriamente storyboard.

Scene

Definiscono una o più schermate e le transizioni tra queste schermate. Ogni scena ha il ciclo di vita.

ViewController

Rappresenta una singola schermata di una scena. Possiede i suoi eventi di ciclo di vita. Gestisce gli eventi di interfaccia. la creazione dell'interfaccia avviene in modalità WYSIWYG. L'associazione tra elementi statici e oggetti dinamici avviene tramite interfaccia grafica.

4.4.8 SwiftUI

Hanno introdotto una libreria per permettere l'implementazione imperativa delle interfacce grafiche. In un certo senso SwiftUI può essere considerato come un domain specific language, un linguaggio di programmazione specifico per la generazione di interfacce grafiche. UIKit a SwiftUI coesistono.

Quando parliamo di programmazione imperativa c'è questo concetto di componente, rappresenta una parte della schermata ed è fondamentale per la costruzione dell'interfaccia, sono una componente fondamentale della modularità e della riusabilità del codice.

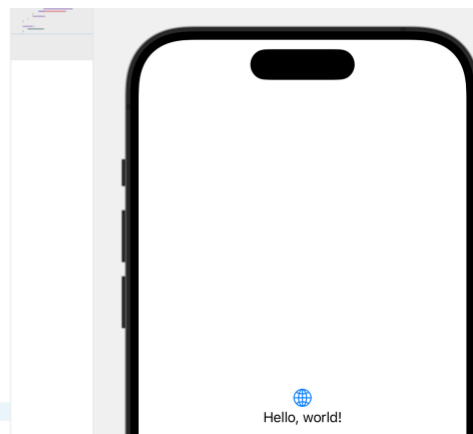
```
1 struct ContentView: View {  
2     var body: some View {  
3         VStack {  
4             Text("Hello")  
5             Text("World")  
6         }  
7     }  
8 }
```

ContentView è una struct che adotta il protocollo View e implementa una computed property (body). Come viene calcolata body, come quella cosa che non sembra normale codice. Il vantaggio è che sembra intuitiva.

Live Preview

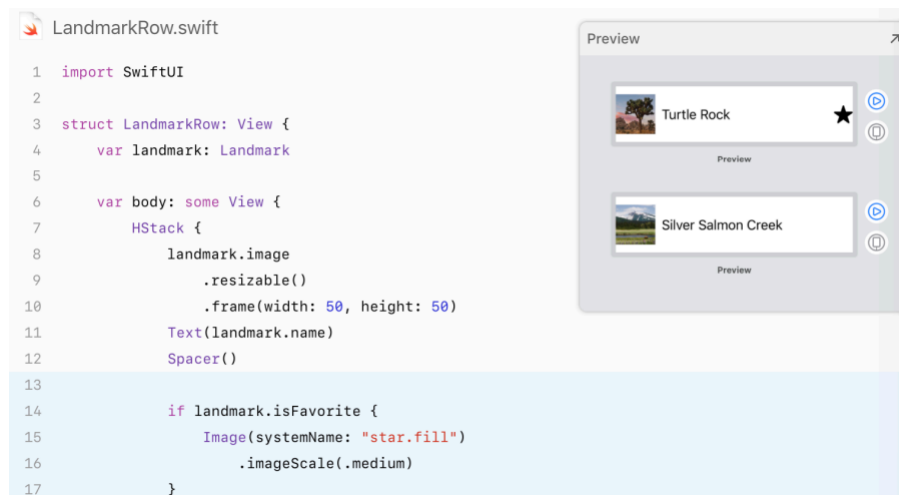
È disponibile uno strumento di preview che mostra come apparirà la view.

```
1 // BasicSwiftUIExample  
2 //  
3 // Created by Sergio Mascetti on 22/04/24.  
4 //  
5 //  
6 //  
7 //  
8 import SwiftUI  
9  
10 struct ContentView: View {  
11     var body: some View {  
12         VStack {  
13             Image(systemName: "globe")  
14                 .imageScale(.large)  
15                 .foregroundColor(.tint)  
16             Text("Hello, world!")  
17         }  
18         .padding()  
19     }  
20 }  
21  
22 #Preview {  
23     ContentView()  
24 }  
25 |
```



Costrutti in SwiftUI

Siccome noi definiamo l'interfaccia tramite codice possiamo usare costrutti di selezione e iterazione. Ci sono delle limitazioni che capiremo meglio più avanti. Body sembra il metodo che ridisegna l'interfaccia (come render in React).



Liste

Per lo stesso motivo, la gestione delle liste è enormemente semplificata. In questo esempio:

```
1 struct LandmarkList: View {
2     var body: some View {
3         List(landmarkData) { landmark in
4             LandmarkRow(landmark: landmark)
5         }
6     }
7 }
```

`LandmarkRow` è una view creata con SwiftUI. `List` accetta due parametri, l'elenco dei dati e una funzione che definisce come ciascun elemento debba essere mostrato, viene passato come closure con la tecnica del trailing closure.

Demo pulsante

Vediamo dall'esempio che cambiando il valore della variabile `showCiao` annotata come state viene ridisegnata tutta l'interfaccia:

```

1 struct ContentView: View {
2     @State private var showCiao = false
3     var body: some View {
4         VStack {
5             Text("Hello, world!")
6             Button(action: {
7                 self.showCiao = true
8             }) {
9                 Text("Button")
10            }
11            if (showCiao) {
12                Text("ciao")
13            }
14        }
15        .padding()
16    }
17 }

```

4.5 Approfondimento su SwiftUI

Come visto tramite SwiftUI riusciamo a generare l'interfaccia tramite codice dinamico. Velocizza lo sviluppo perchè ha una forma comoda ed espressiva da usare.

Questo codice rappresenta un pulsante, al tocco cambiamo il testo cambiando la variabile annotata come @State:

```

1 struct ContentView: View {
2     @State var text = "Ciao"
3     var body: some View {
4         VStack {
5             Text(text)
6             Button("Change text") {
7                 text="hola"
8             }
9         }
10    }
11 }

```

Come è possibile che cambiare il valore di una stringa venga aggiornata la vista? inoltre la parola some cosa significa? perchè possiamo scrivere text e button così uno sotto l'altro?

4.5.1 Property Wrappers

Hanno introdotto un costrutto sintattico chiamato property wrapper, sono modificatori di proprietà. Possiamo crearli noi, o meglio in linea di principio, quello che succede è che usiamo quelli già creati.

Vediamo un esempio in cui non li usiamo, abbiamo una struct che ha un valore privato (non viene esposto), poi abbiamo una variabile esposta all'esterno con un get e set. Questa struct logga ogni volta che viene modificata la variabile.

```
1 struct MyStruct {
2     private var _x = 0
3
4     var x: Int {
5         get { _x }
6         set {
7             _x = newValue
8             print("New value is \$(newValue)")
9         }
10    }
11 }
12
13 var bar = MyStruct()
14 bar.x = 1 // Prints New value is 1
```

Se volessimo farlo per più variabili non vorremmo scrivere questo codice tante volte, ma potremmo usarlo con i Generics. Abbiamo una variabile di un tipo che ogni volta che cambio quel valore stampa a log.

```
1 struct ConsoleLogged<Value> {
2     private var value: Value
3
4     init(wrappedValue: Value) {
5         self.value = wrappedValue
6     }
7     var wrappedValue: Value {
8         get { value }
9         set {
10             value = newValue
11             print("New value is \$(newValue)")
12         }
13     }
14 }
15
16 struct MyStruct {
```

```

17 private var _x = ConsoleLogged<Int>(wrappedValue: 0)
18
19 var x: Int {
20     get { _x.wrappedValue }
21     set { _x.wrappedValue = newValue }
22 }
23 }
24
25 var bar = MyStruct()
26 bar.x = 1 // Prints 'New value is 1'

```

In alternativa possiamo rendere la variabile `x` di `MyStruct` visibile all'esterno. Però quando andiamo a modificare il valore, dobbiamo usare l'attributo `wrappedValue`:

```

1 struct ConsoleLogged<Value> {
2     private var value: Value
3
4     init(wrappedValue: Value) {
5         self.value = wrappedValue
6     }
7     var wrappedValue: Value {
8         get { value }
9         set {
10             value = newValue
11             print("New value is \(newValue)")
12         }
13     }
14 }
15 struct MyStruct {
16     var x = ConsoleLogged<Int>(wrappedValue: 0)
17 }
18 var bar = MyStruct()
19 bar.x.wrappedValue = 1 // Prints 'New value is 1'

```

Questo pattern è molto comune e Swift introduce i property wrappers proprio per renderne l'implementazione più compatta. Sintatticamente serve:

- La dichiarazione `@propertyWrapper`
- Una property `wrappedValue`

```

1 @propertyWrapper
2 struct ConsoleLogged<Value> {
3     private var value: Value

```

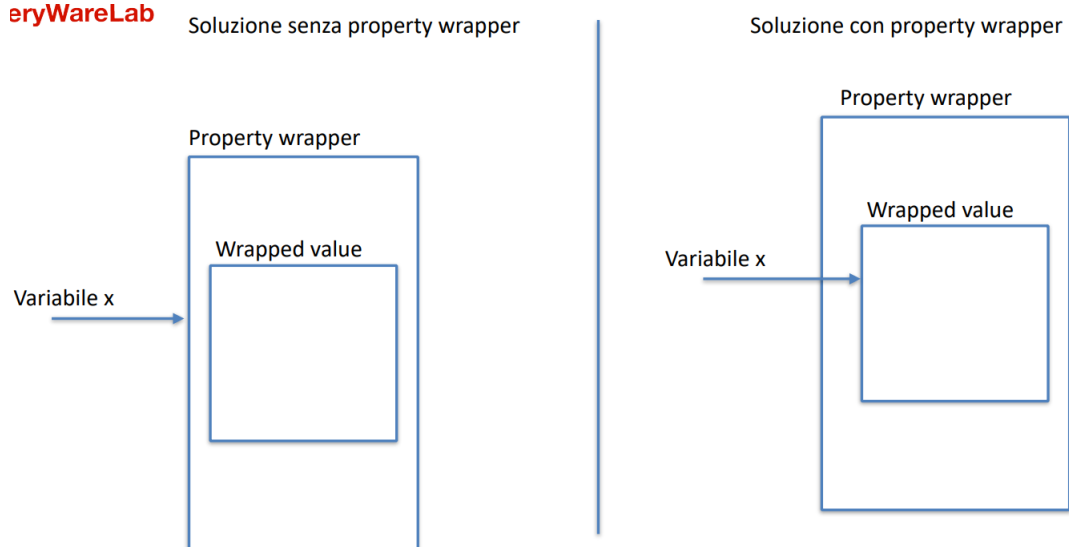
```

4
5     init(wrappedValue: Value) {
6     self.value = wrappedValue
7     }
8
9     var wrappedValue: Value {
10        get { value }
11        set {
12            value = newValue
13            print("New value is \(newValue)")
14        }
15    }
16 }
17
18 struct MyStruct {
19     @ConsoleLogged var x = 0
20 }
21
22 var bar = MyStruct()
23 bar.x = 1 // Prints 'New value is 1'

```

La morale è che abbiamo creato noi questo attributo. Ora dovrebbe essere chiaro cosa fa @State. Abbiamo una variabile x che è di tipo wrapped value non più del tipo contenitore. Prima avevamo due oggetti contenitore e contenuto, noi avevamo riferimento al contenitore e il contenitore ha un riferimento al contenuto. Ora abbiamo solo il riferimento al contenuto, però ereditiamo i comportamenti del contenitore.

eryWareLab



4.5.2 @State

Il property wrapper `@State` sovrascrive il metodo di scrittura di una variabile per far sì che quando il valore di quella variabile viene aggiornato la view viene ridisegnata. È possibile cambiare lo stato delle variabili `@State` da qualsiasi thread, ci viene garantito che l'interfaccia viene ridisegnata con il thread principale, questo semplifica tantissimo la scrittura del codice.

4.5.3 Altri property wrapper

Ci sono diversi property wrapper che si usano comunemente in SwiftUI:

- `@Binding`
- `@ObservedObject`
- `@Published`
- `@EnvironmentObject`
- `@Environment`

Quasi tutti i property wrapper definiti in SwiftUI (o almeno quelli di uso più comune) hanno lo scopo di definire quali sono i dati sulla base dei quali deve cambiare la UI. Questi dati sono chiamati “Single source of truth”, il concetto è che non dobbiamo duplicare l'informazione, la componente figlia deve avere un riferimento al padre.

4.5.4 Projected values e @Binding

Quando si definisce un property wrapper possiamo definire una property che si chiama `projectedValue`, che può essere di qualunque tipo, può ritornare quello che vogliamo, nell'esempio ritorna un riferimento a se stessa, o in un caso specifico ad esempio ritornare quante volte ho stampato il valore. Come faccio ad accedere al `projectedValue`? `NSStringOfA` è una stringa, possiamo accedere con un'eccezione sintattica mettendoci il `$` davanti, in questo modo se mettiamo il dollaro davanti ad un `propertyWrapper` ci ritorna il `projectedValue`:

```
1 @propertyWrapper
2 struct ConsoleLogged<Value> {
3     private var value: Value
4     private var logCount = 0
5 }
```

```

6  init(wrappedValue: Value) {
7  self.value = wrappedValue
8  }
9  var wrappedValue: Value {
10 get { value }
11 set {
12 value = newValue
13 print("New value is \$(newValue)")
14 logCount += 1
15 }
16 }
17
18 var projectedValue: ConsoleLogged<Value> {
19 return self
20 }
21
22 var logsNumber: Int {
23 logCount
24 }
25 }
26 func example() {
27 @ConsoleLogged var aStringOfA = "a"
28 for _ in 1...5 {
29 aStringOfA = aStringOfA + "a"
30 }
31 print(\$aStringOfA.logsNumber) //prints 5

```

Nell'esempio di prima ritornava un riferimento a se stesso, possiamo fargli ritornare lo stato:

```

1  @propertyWrapper
2  struct ConsoleLogged<Value> {
3  private var value: Value
4  private var logCount = 0
5
6  init(wrappedValue: Value) {
7  self.value = wrappedValue
8  }
9  var wrappedValue: Value {
10 get { value }
11 set {
12 value = newValue
13 print("New value is \$(newValue)")
14 logCount += 1

```

```

15 }
16 }
17
18 var projectedValue: Int {
19     return logCount
20 }
21 }
22 func example() {
23     @ConsoleLogged var aStringOfA = "a"
24     for _ in 1...5 {
25         aStringOfA = aStringOfA + "a"
26     }
27     print($aStringOfA) //prints 5
28 }

```

Binding

Il `projectedValue` di `@State` è un property wrapped di tipo `@Binding`. Il codice di seguito funziona

```

1 struct ContentView: View {
2     @State var p = Person(name: "Andrea", surname: "Andrei")
3     var body: some View {
4         VStack {
5             HStack{
6                 Text(p.name)
7                 Text(p.surname)
8             }
9             Button("Hide name") {
10                 p.name=""
11             }
12         }
13     }
14 }

```

Supponiamo che la nostra View principale diventi molto complicata e che vogliamo delegare parte della sua gestione ad una sotto-componente `Person-View`. L'idea è di passare una `person` alla sotto componente e cambiare il valore lì.

C'è però un errore sintattico. In SWIFT, un metodo di una istanza `value type` non può modificare i propri attributi. Per fare in modo che un metodo possa modificare gli attributi di una `struct`, è necessario dichiararlo come `mutating`. Tuttavia, se un metodo in un protocollo è definito come `non-mutating`, non si può implementare con un metodo `mutating`.

```

struct StructDiProva {
    var person: Person
    func prova() {
        person.name = "ciao"
    }
}

```

Cannot assign to property: 'self' is immutable

```

struct StructDiProva {
    var person: Person
    mutating func prova() {
        person.name = "ciao"
    }
}

```

Sintatticamente corretto

```

protocol ProtocolloDiProva {
    func prova()
}
struct StructDiProva : ProtocolloDiProva {
    var person: Person
    mutating func prova() {
        person.name = "ciao"
    }
}

```

Type 'StructDiProva' does not conform to protocol 'Prot...

Al di là di questo errore sintattico l'errore concettuale è che non vogliamo due copie di person, perchè se lo modifichiamo deve cambiare ovunque.

@Binding permette ad una view figlia di utilizzare un riferimento alle istanze @State definite nel padre. Quindi passiamo alla view figlia un propertyWrapped con il dollaro davanti, che significa estrai il projectedValue. Quando si apportano modifiche al dato passato con un Binding, il valore nel padre viene modificato e dunque la view viene ridisegnata.

```

1 struct ContentView: View {
2     @State var p = Person(name: "Andrea",
3     surname: "Andreii")
4     var body: some View {
5         VStack {
6             PersonView(p: $p)
7             //immaginate ci sia molto altro codice qua
8         }
9     }
10 }
11 struct PersonView: View {
12     @Binding var p: Person
13     var body: some View {
14         VStack {
15             HStack{
16                 Text(p.name)
17                 Text(p.surname)
18             }
19             Button("Hide name") {

```

```

20         p.name = ""
21     }
22 }
23 }
24 }

```

4.5.5 Observable objects

In tutti gli esempi che abbiamo fatto abbiamo memorizzato il dato nella classe della View, tipicamente questa è una cattiva idea. Ci capita spesso di avere un model che contiene i dati ed un view model che ci aiuta a preparare i dati che devono essere mostrati, pensiamo ad un applicazione con diverse schermate che mostrano gli stessi dati. Si crea un view model che aiuta a creare i dati per la view avendo una sola copia dei dati.

Per questo si usa il protocollo ObservableObject che permette di specificare che una classe di Model (o meglio ViewModel) è una single source of truth. Dichiariamo una variabile con il property wrapper @Published che intendiamo rendere "single source of truth":

```

1 class Model: ObservableObject {
2     @Published var people: [Person] = []
3     init() {
4         people.append(Person(
5             name: "Andrea", surname: "Andrei"))
6         people.append(Person(
7             name: "Bruno", surname: "Bruni"))
8         //...

```

Possiamo definire una istanza del model a livello di app. Poi passiamo il riferimento del model alle varie view che lo modificano/visualizzano, usando il property wrapper @ObservedObject. Eventuali modifiche del model causano il ridisegno dell'interfaccia grafica.

```

1 struct SwiftUIListExampleApp: App {
2     var model = Model()
3     var body: some Scene {
4         WindowGroup {
5             ContentView(model: model)
6         }
7     }
8 }
9
10 struct ContentView: View {
11     @ObservedObject var model: Model

```

```

12  var body: some View {
13      VStack {
14          PeopleView(model: model)
15          DeleteOneView(model: model)
16      }
17  }
18 }
19
20 struct PeopleView: View {
21     @ObservedObject var model: Model
22     var body: some View {
23         Text("\(model.people.count)")
24     }
25 }
26
27 struct DeleteOneView: View {
28     @ObservedObject var model: Model
29     var body: some View {
30         Button(action: {model.people.removeFirst()}) {
31             Text("remove first")
32         }
33     }
34 }

```

4.5.6 I tipi opachi

Servono per spiegare la parola riservata `some`, si utilizza per i valori di ritorno di funzioni metodi e proprietà. La proprietà ritorna un tipo particolare che aderisce ad un protocollo, ma il chiamante non sa che tipo verrà ritornato. Per come è spiegata così anche `var body: View` ritorna una classe che aderisce ad un protocollo ma non sappiamo che tipo, quindi perchè usare `some`? La differenza è che nel primo caso sappiamo a tempo di esecuzione quale valore ci viene ritornato, quando invece usiamo `some View` ci viene ritornato sempre lo stesso tipo e lo sappiamo a tempo di compilazione ma non indichiamo quale View.

Supponiamo di avere un protocollo `P`, con due strutture che adottano `P` con un metodo che ritorna un istanza di protocollo `P`, so cosa ritorno a tempo di esecuzione:

```

1  protocol P {}
2  struct S1: P {}
3  struct S2: P {}
4  func getP(a: Bool) -> P {

```

```

5  if (a) {
6  return S1()
7  } else {
8  return S2()
9  }
10 }

```

Quindi in questo caso non si potrebbe usare `some`. Perchè se diciamo che un metodo ritorna `"some P"` vuol dire che ritorna sempre lo stesso tipo, mentre in questo codice a volte si ritorna `S1`, altre `S2`, dunque c'è un errore a tempo di compilazione. Abbiamo mostrato cosa NON si può fare con i tipi opachi, ma non abbiamo detto COSA si può fare.

Associated types

Dato un protocollo questo può avere un tipo associato, che è un segna posto, non mi interessa quale sia il tipo di quel tipo associato, a patto che valgano certi vincoli, ad esempio (nel codice) per implementare il protocollo dobbiamo usare lo stesso tipo nei metodi `append` e `subscript`:

```

1  protocol Container {
2      associatedtype Item
3      mutating func append(_ item: Item)
4      var count: Int { get }
5      subscript(i: Int) -> Item { get }
6  }
7
8  struct IntStack: Container {
9      // original IntStack implementation
10     var items = [Int]()
11     mutating func push(_ item: Int) {
12         items.append(item)
13     }
14     mutating func pop() -> Int {
15         return items.removeLast()
16     }
17     // conformance to the Container protocol
18     mutating func append(_ item: Int) {
19         self.push(item)
20     }
21     var count: Int {
22         return items.count
23     }
24     subscript(i: Int) -> Int {

```

```

25         return items[i]
26     }
27 }

```

Il problema del protocollo con associated types non posso ritornarlo come tipo, perchè swift non sarebbe in grado di fare controlli sintattici, questo esempio non compila:

```

1 protocol MobileOS {
2     associatedtype Version
3     var version: Version { get }
4     init(version: Version)
5 }
6
7 struct iOS: MobileOS {
8     var version: Float
9 }
10
11 struct Android: MobileOS {
12     var version: String
13 }
14
15 func buildPreferredOS() -> MobileOS {
16     return iOS(version: 13.1)
17 }

```

Ma se ci metto some allora compila, la funzione buildPreferredOS() ritorna sempre una istanza di iOS. Il compilatore sa quindi che tipo ritorna perchè ne ritorniamo sempre uno di MobileOS:

```

1 protocol MobileOS {
2     associatedtype Version
3     var version: Version { get }
4     init(version: Version)
5 }
6
7 struct iOS: MobileOS {
8     var version: Float
9 }
10
11 struct Android: MobileOS {
12     var version: String
13 }
14
15 func buildPreferredOS() -> some MobileOS {

```

```

16     return iOS(version: 13.1)
17 }

```

In swiftUI potremmo specificare il tipo di View che viene ritornato ma è un casino se il valore di ritorno è complesso, quindi è comodo usare some così lui prende il valore di ritorno della funzione:

```

1 struct ContentView: View {
2     var body: some View {
3         Text("Hello, World!")
4     }
5 }

```

```

1 struct ContentView: View {
2     var body: Text {
3         Text("Hello, World!")
4     }
5 }

```

L'esempio sopra è corretto, tuttavia se il programmatore cambia il contenuto della computed property body, deve specificare a mano quale è il valore di ritorno. I tipi opachi servono per semplificare la sintassi di scrittura del codice, questo è un aspetto centrale di SwiftUI, in realtà ci sono anche dei motivi di performance che giustificano l'uso dei tipi opachi, ma non approfondiamo questo aspetto.

4.5.7 Result builders

Come può funzionare questo esempio? Cosa ritorna la computer property body? il codice tra le parentesi della View è una funzione, ha una sola riga di codice, il return è implicito.

```

1 struct ContentView: View {
2     var body: some View {
3         Text("Hello, World!")
4     }
5 }

```

Ma questo? ci sono due righe di codice:

```

1 struct ContentView: View {
2     var body: some View {
3         VStack{
4             Text("Hello, world!")
5             Text("Hello, world!")
6         }

```

```

7     }
8 }

```

Questo dovrebbe essere ancora più strano, perchè sintatticamente è impossibile sapere il tipo di ritorno:

```

1 struct ContentView: View {
2     @State var a = true
3     var body: some View {
4         VStack {
5             Text("Hello")
6             if (a) {
7                 Text("Andrea")
8             } else {
9                 Text("Bruno")
10            }
11        }
12    }
13 }
14 }

```

Dobbiamo introdurre i result builder, che sono un costrutto sintattico che in linea di principio potremmo usare con qualunque cosa. Sono molto utili se vogliamo creare un domain specific language (un adattamento di un linguaggio di programmazione specifico per un particolare dominio). Sono una funzione che prende un elenco di istanze e restituisce un risultato, ma quindi è una normale funzione? L'idea è di fare in modo che gli argomenti di questa funzione possano essere passate come righe indipendenti e queste righe possano contenere delle righe di selezione e iterazione, in modo che la normale sintassi swift guardi prima i costrutti e poi come ritorno mi da una funzione result builder.

Definiamo del codice per creare delle stringhe con "formattazione":

```

1 protocol Drawable {
2     func draw() -> String
3 }
4
5 struct Line: Drawable {
6     var elements: [Drawable]
7     func draw() -> String {
8         return elements.map { $0.draw()
9     }.joined(separator: " ")
10 }
11 }

```

```

12 struct Text: Drawable {
13     var content: String
14     init(_ content: String) { self.content = content }
15     func draw() -> String { return content }
16 }
17
18 struct Space: Drawable {
19     func draw() -> String { return " " }
20 }
21
22 struct Stars: Drawable {
23     var length: Int
24     func draw() -> String { return String(repeating:
25         "*", count: length) }

```

Questo è un parallelo di cosa avviene con le interfacce. Se vogliamo usare questo codice:

```

1 let manualDrawing = Line(elements: [
2     Stars(length: 3),
3     Text("Hello"),
4     Space(),
5     Text("Andrea"),
6     Space(),
7     Stars(length: 2),
8 ])
9
10 print(manualDrawing.draw())
11 // Prints "***Hello Andrea**"

```

Questo codice funziona, ma non è facilmente leggibile, inoltre se vogliamo aggiungere dei costrutti di selezione e iterazione non possiamo farlo nell'array e di conseguenza il codice diventa sempre più difficile da leggere:

```

1 var manualDrawing3: Line
2 var drawableArray = [Drawable]()
3 for _ in 1 ... 4 {
4     drawableArray.append(Text("Hello"));
5     drawableArray.append(Space());
6     drawableArray.append(Text("Andrea"));
7     drawableArray.append(Stars(length: 2));
8 }
9
10 manualDrawing3 = Line(elements: drawableArray)

```

```

11 print(manualDrawing3.draw())
12 //Hello Andrea**Hello Andrea**Hello Andrea**Hello
    Andrea**

```

```

1 var manualDrawing2: Line
2 let a = true
3 if (a) {
4     manualDrawing2 = Line(elements: [
5         Stars(length: 3),
6         Text("Hello"),
7         Text("Andrea"),
8         Space(),
9         Stars(length: 2),
10    ])
11 } else {
12     manualDrawing2 = Line(elements: [
13         Stars(length: 3),
14         Text("Hello"),
15         Text("Bruno"),
16         Space(),
17         Stars(length: 2),
18    ])
19 }

```

l'idea è introdurre un costrutto sintattico in modo tale da poter indicare cosa c'è all'interno di una linea specificandolo in modo più facile per il programmatore, creiamo una struttura specifica per aiutarci a scrivere del codice che riguardi il disegno. Usiamo l'attributo `@resultBuilder` che ci serve associato ad una struct, dove questa struct ha un metodo che si chiama `buildBlock`, il quale prende in input `tot Drawable` e ci restituisce un `Drawable`:

```

1 @resultBuilder
2 struct DrawingBuilder {
3     static func buildBlock(_ components: Drawable...)
4     -> Drawable {
5         return Line(elements: components)
6     }
7 }

```

Il trucco è come va a leggere questo, possiamo usare il result builder come un attributo dei parametri di una funzione. Nell'esempio la closure viene passata al result builder dando in input, come se fossero elementi di un array, l'elenco delle righe di codice che seguono. Ma in più sono gestiti:

- Costrutti indentati

- Costrutti di selezione

```

1 func createDraw(@DrawingBuilder content:
2 () -> Drawable)-> Drawable {
3     return content()
4 }
5
6 let myDrawable = createDraw {
7     Stars(length: 3)
8     Text("Hello")
9     Space()
10    Text("World!")
11    Stars(length: 4)
12 }
13
14 print(myDrawable.draw())
15 //returns: ***Hello World!***

```

Possiamo definire due metodi aggiuntivi nel result builder che definiscono i due casi di un costrutto di selezione. Come costruire il risultato in un caso (if verificato) e nell'altro caso:

```

1 static func buildEither(first: Drawable) -> Drawable {
2     return first
3 }
4 static func buildEither(second: Drawable) -> Drawable {
5     return second
6 }

```

Così posso fare questo:

```

1 let bool = false
2 let myDrawable = createDraw {
3     Stars(length: 3)
4     Text("Hello")
5     Space()
6     if (bool) {
7         Text("World!")
8     } else {
9         Text("mondo")
10    }
11    Stars(length: 4)
12 }

```

La logica del costrutto di selezione viene interpretata dal compilatore swift, mentre cosa fare in un caso o nell'altro viene delegato al result builder.

In modo analogo possiamo definire come il result builder deve gestire i costrutti di iterazione. Aggiungiamo il metodo: `buildArray` che definisce come gestire il risultato di un costrutto di iterazione, che è rappresentato come un array:

```
1 static func buildArray(_ components: [Drawable]) ->
   Drawable {
2     return Line(elements: components)
3 }
```

A questo punto possiamo usare i costrutti di iterazione:

```
1 let lotOfStars = createDraw {
2     for length in 1...4 {
3         Space()
4         Stars(length: length)
5     }
6 }
7
8 print(lotOfStars.draw())
9 //prints: * ** *** ****
```